

Nothing Is Magic

Machine learning mathematics

Yulan Galagoda

2026-07-23

Table of contents

Preface	1
1 Vectors: Arrows and Lists	3
The itch	3
The picture	4
The math, built up	6
Build it yourself	9
Where it lives in ML	12
Common misunderstandings	14
Check your intuition	16
2 Adding, Scaling, and Span	19
The itch	19
The picture	20
The math, built up	22
Build it yourself	25
Where it lives in ML	27
Common misunderstandings	29
Check your intuition	30
3 The Dot Product	33
The itch	33
The picture	34
The math, built up: the formula	36
What the dot product contains	38
Build it yourself	40

Table of contents

Where it lives in ML	43
Common misunderstandings	45
Check your intuition	47
4 Length, Distance, and Norms	51
The itch	51
The picture	52
The math, built up	55
Build it yourself	56
Where it lives in ML	58
Common misunderstandings	60
Check your intuition	61
5 Matrices as Transformations	65
The itch	65
The picture	66
The math, built up	69
Build it yourself	71
Where it lives in ML	73
Common misunderstandings	75
Check your intuition	77
6 Matrix Multiplication as Composition	79
The itch	79
The picture	80
The math, built up	82
Build it yourself	83
Where it lives in ML	86
Common misunderstandings	87
Check your intuition	88
7 Independence, Basis, and Rank	91
The itch	91
The picture	92

Table of contents

The math, built up	94
Build it yourself	95
Where it lives in ML	97
Common misunderstandings	99
Check your intuition	100
8 The Determinant	103
The itch	103
The picture	104
The math, built up	106
Build it yourself	107
Where it lives in ML	109
Common misunderstandings	110
Check your intuition	111
9 Systems of Equations, Geometrically	113
The itch	113
The picture	114
The math, built up	116
Build it yourself	117
Where it lives in ML	119
Common misunderstandings	121
Check your intuition	122
10 Eigenvectors and Eigenvalues	125
The itch	125
The picture	126
The math, built up	128
Build it yourself	129
Where it lives in ML	132
Common misunderstandings	133
Check your intuition	134

Table of contents

11 The Singular Value Decomposition	137
The itch	137
The picture	138
The math, built up	140
Build it yourself	141
Where it lives in ML	144
Common misunderstandings	146
Check your intuition	147
12 High-Dimensional Space	151
The itch	151
The picture	152
The math, built up	153
Measure it yourself	154
Where it lives in ML	156
Common misunderstandings	157
Check your intuition	158

Preface

Most people do not meet the mathematics behind machine learning. They meet its symbols. A formula appears, already assembled, and the task is to accept it and move on. The reason anyone would ever have written that formula down, is left as something you are supposed to absorb later, or never. This book is an attempt at the opposite: to arrive at every formula only once you already feel why you want it.

The idea is simple. Before a symbol appears, there is a picture. Before the picture, there is a problem you can actually feel, and is always something concrete enough to be irritating. We start from the itch, build the picture that resolves it, and only then write down the mathematics, so that the formula reads as the inevitable end of a story rather than the start of one. Nothing is asserted that has not first been made to feel obvious, and nothing is left as magic.

Three commitments run through every chapter. No notation arrives before the reader wants it. Every major concept is implemented from scratch, in plain code you can run, with no library quietly doing the real work. And every idea is tied back to where it actually lives in machine learning, so you always know not just what a thing is, but what it is *for*, and what breaks when you get it wrong.

The book is being written in the open, one part at a time, based on my own notes that I composed when I first started studying Machine Learning, and it will always be free to read. **Part 1, on linear algebra, is complete:** its twelve chapters carry you from what a vector is through to the singular value decomposition, the single most useful construction in the subject,

Preface

and end by showing where the geometric intuition the whole part rests on quietly stops being reliable. Part 2 (calculus), Part 3 (probability and statistics), and Part 4, where all three come together into the mathematics of learning itself, are on the way.

You do not need much to begin. School algebra, and the memory of having once seen a derivative, is enough. Beginners can lean on the warm-ups; readers who already know a section can skip ahead from its clearly marked opening. The book is meant to be read start to finish, but a finished Part 1 is already a complete resource on its own, so wherever you stop, nothing is wasted.

A note on why this exists. Good mathematical intuition is not evenly distributed, and neither are the resources that build it: the best of them are often priced out of reach for exactly the people who would most benefit. This book stays free and online for that reason. If it helps you see why the formula works, it has done its job.

1 Vectors: Arrows and Lists

The itch

It is a calm, quiet night, and Spotify plays us a song we have never heard before. It is by a band we could not name, from a country we have never visited, sung in a language we do not speak. And somehow it fits. It settles into the evening so well that we do not reach for the phone to skip it. We just let it play.

Something chose that song for us. But that something has never heard a song in its life. It has no ears. It has never felt a bassline in its chest, never had a chorus stuck in its head on the walk home, never tied a track to a person it misses. It has no idea what music even is.

And yet it was right.

So whatever “these two songs are similar” means to that machine, it cannot mean what it means to us. It is not recognising a mood or remembering a feeling. It is doing something far more mechanical, something it can actually carry out step by step. And whatever that something is, it worked well enough that we sat there and let the song play.

That leaves us with one question, and it is the only question this chapter is about:

What would we have to turn a song into, so that a machine could measure how close it is to another song?

1 *Vectors: Arrows and Lists*

Notice that the machine does not need to describe the song, and it does not need to understand it. It only needs to measure it: to get a number out, compare that number to another, and act on the result. So whatever a song becomes in the machine, it has to be the kind of thing we can hold a ruler against and read off a value.

Turning a thing into something we can measure is the first move in all of machine learning. Every recommendation we have ever been given, every time a phone has unlocked at the sight of a face, every sentence a model has finished for us, all of it starts here. And the idea we need is smaller than we might expect.

The picture

Let us take a single song and, just to keep things simple while we build the idea, pretend there are only two things about it we can measure: how fast it is, and how loud. Call these tempo and loudness. We write each one down as a number, and the song becomes a pair of numbers, something like 128 and 7.

That pair of numbers is a **list**, and the order is part of the deal. We agree once and for all that tempo comes first and loudness comes second, and we hold to that for every song we ever describe. Without that agreement the numbers would be ambiguous, so we treat it as a fixed rule. With the rule in place, a list is simply a set of numbers lined up in order, each sitting in a slot that means something.

Now let us draw the same song. We put tempo along a horizontal axis and loudness along a vertical one. Starting from the corner where the axes meet, we walk 128 steps to the right and then 7 steps up, and we mark where we end up. If we draw an **arrow** from the starting corner to that mark, we have turned the two numbers into something with a shape.

Look at what that arrow gives us. It points in a particular direction, and it has a particular length. Neither of those is new information we added; both came straight out of the two numbers. Walk the same steps again and we land in exactly the same place, every time.

Here is the idea the whole chapter turns on: the list and the arrow are the same object looked at in two ways. The list tells us how far to walk along each axis. The arrow is where that walk leaves us. Neither one is more real than the other, and once we have either, we can always recover the other.

If they are the same object, it is fair to ask why we bother keeping both. The answer is that each is good at something the other is not. The list is what a computer can hold, because it is nothing but numbers in memory, and numbers are all a machine has ever had to work with. The arrow is what we can think with, because no one looks at 128 and 7 and feels anything, but anyone can look at two arrows and see at a glance whether they point the same way. We keep the list for the machine and the arrow for ourselves.

Two things are worth pinning down before we move on.

The first is that the order of the numbers is not decoration. If we swap them and write 7 and 128, we now walk 7 steps right and 128 steps up, which lands us somewhere completely different and describes a song crawling along at 7 beats a minute at deafening volume. The slots carry meaning, so moving a number from one slot to another changes what the list says. This is exactly why the order has to be a rule we fix at the start.

The second is that nothing about this stops at two numbers. Add a third, say danceability, and we have three numbers and an arrow that now lives in a room rather than on a flat page. Add a fourth and we can no longer picture the arrow at all. That is a limit of our eyes, though, not a limit of the idea or of the computer. The list does not mind in the slightest; it simply gets longer. Everything we build in this book keeps working out there in the space we cannot see, which matters because that is where real

1 Vectors: Arrows and Lists

data actually lives. A song on Spotify is not two numbers. It is a few hundred.

So let us return to the itch we started with. Two songs become two arrows. When the songs are alike, their arrows point in much the same direction. When they have nothing in common, their arrows point off in different directions. And how close two arrows come to pointing the same way is the kind of question that has a number for an answer, which is precisely what we were looking for.

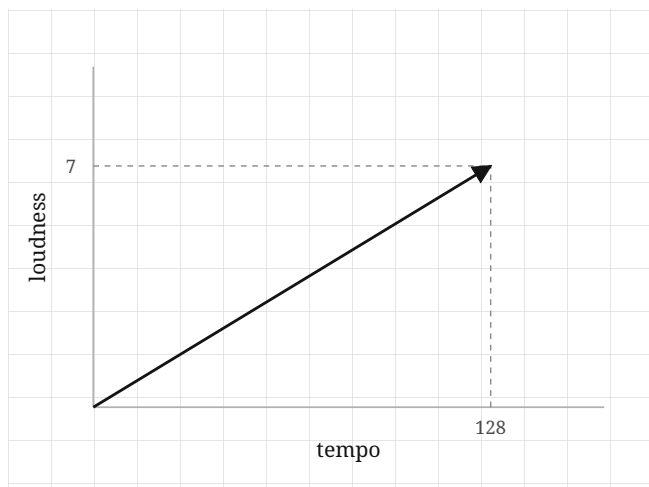


Figure 1.1: One song, two numbers, one arrow.

The math, built up

The object we have been describing, the list that is also an arrow, has a name. We call it a **vector**. That is the whole of the word. Every vector in this book is exactly what we just built by hand: some numbers sitting in slots, and an arrow we can picture.

To work with a vector we need a way to write it down, so we give it a name and put its list in brackets:

$$\mathbf{v} = [128, 7]$$

The name here is $\mathbf{v}$, written in bold to remind us it stands for a whole vector and not a single ordinary number. The brackets hold the list, which we read as tempo 128, loudness 7. Nothing has been calculated yet; this is just notation, a tidy way of naming what we already have. It is also a convention shared by more or less everyone who works with vectors, so once we are comfortable reading it, we can read it anywhere.

With the notation in place, we can answer the first real question: how long is that arrow?

Let us look again at the drawing. We have the arrow itself, and we have the two dashed lines we dropped down to the axes. Those three lines together form a right-angled triangle. The two dashed lines are its short sides, 128 across and 7 up, and the arrow is its long side, running from the corner out to the tip.

And the long side of a right-angled triangle is something we already know how to find. It is the one piece of school geometry most of us still carry: square the two short sides, add them together, and take the square root. That is Pythagoras, and it gives us the length of our arrow directly:

$$\|\mathbf{v}\| = \sqrt{v_1^2 + v_2^2}$$

The left-hand side is read as “the length of $\mathbf{v}$.” The double bars are chosen deliberately. A single bar, as in $|x|$, means the size of a single ordinary number, while the double bars mean the size of a whole vector, where several numbers fold together into one. Different job, different symbol.

The right-hand side is Pythagoras and nothing more. Here v_1 is the first number in the list and v_2 is the second, so we square each of them, add

1 Vectors: Arrows and Lists

the results, and take the root. We did not invent a new formula for the length of a vector. We noticed that the arrow was already the long side of a triangle, and used what we had.

It is worth checking this on a case we can verify with our own eyes. Take $\mathbf{v} = [3, 4]$, three across and four up:

$$\|\mathbf{v}\| = \sqrt{3^2 + 4^2} = \sqrt{9 + 16} = \sqrt{25} = 5$$

The length comes out as five. Draw that arrow on squared paper and measure it with a ruler, and it is five squares long. The formula is not telling us anything we could not have found with the ruler. What it is doing is telling us how to find the length without the ruler, which is the only way a machine could ever find it.

Now let us add a third number, danceability again, so the vector is $[v_1, v_2, v_3]$ and the arrow lives in a room. How long is it?

$$\|\mathbf{v}\| = \sqrt{v_1^2 + v_2^2 + v_3^2}$$

It is the same shape as before: square everything, add it all up, take the root. A fourth number puts four squares under the root, and three hundred numbers put three hundred squares under it:

$$\|\mathbf{v}\| = \sqrt{v_1^2 + v_2^2 + \cdots + v_n^2}$$

This is the moment worth pausing on. Our picture ran out at three numbers, because we cannot see four axes and we never will. The formula, though, does not run out. It has no reason to stop and no way of knowing it should. It simply keeps squaring and summing and rooting, and it keeps handing back a sensible number, and that number keeps behaving in every way like a length. Once we understand how the length works in the two or three

dimensions we can see, we can trust it to keep meaning the same thing in the hundreds of dimensions we cannot.

That is what we gain by turning the picture into arithmetic. The arithmetic is free to go where our eyes cannot follow.

So we can now take a single song and get a number out of it, which is real progress. But it is not yet the thing we set out to find. A length is a fact about one arrow on its own, and the itch was about two: two songs, two arrows, and how close they come to pointing the same way. One arrow cannot answer that. We need something that takes two vectors in and gives a single number back, and that something turns out to be the most useful object in this entire book. It is where we go next.

Build it yourself

We have now found the length of an arrow in two ways, once with a ruler on squared paper and once with Pythagoras on paper. The point all along was to find it without the ruler, so let us hand the job to a machine and watch it do the same thing.

First we make the vector:

```
import numpy as np

v = np.array([3.0, 4.0])
print(v)
```

```
[3.  4.]
```

It is worth pausing on what we actually gave the computer. We did not give it an arrow; we gave it a list, three then four, in that order. The

1 Vectors: Arrows and Lists

machine has no picture of this and never will. It has the two numbers, and as we are about to see, the two numbers are all it needs.

Now we compute the length exactly the way the formula reads, squaring each number, adding the results, and taking the root:

```
squared = v ** 2
total = np.sum(squared)
length = np.sqrt(total)

print(squared)
print(total)
print(length)
```

```
[ 9. 16.]
25.0
5.0
```

Each line here is one piece of $\sqrt{v_1^2 + v_2^2}$. The squares come out as 9 and 16, they add up to 25, and the root of 25 is 5.0. We have done the arithmetic ourselves, step by step, with nothing hidden.

NumPy also offers the length as a single ready-made function:

```
print(np.linalg.norm(v))
```

```
5.0
```

The answer is the same. The word **norm** is just the mathematician's term for length, and this function is doing precisely what we did by hand a moment ago: squaring, summing, and rooting. It is not doing anything cleverer than that. Since that claim is easy to check, let us have the machine check it for us:


```
print(np.sqrt(np.sum(v ** 2)) == np.linalg.norm(v))
```

True

It comes back **True**, which tells us the library and the formula really are the same thing. (We will find, a little later, that this kind of exact check is not always as trustworthy as it looks here. For now it holds, and it makes the point.)

Now for the part that matters. Here is a vector with three hundred numbers in it, which is much closer to what a real song actually is. We cannot draw it, we cannot picture it, and we would not know where to begin if we tried:

```
rng = np.random.default_rng(0)
big = rng.random(300)

print(big.shape)
print(np.linalg.norm(big))
```

```
(300,)
10.683214066915752
```

A number came back.

It is worth sitting with that for a moment, because it is the whole reason this chapter exists. We asked for the length of an arrow in three hundred dimensions. No person has ever seen such an object or ever will. And the machine answered in a fraction of a millisecond, using nothing but the formula we worked out from a triangle on a page.

The code did not change to make this happen. `np.linalg.norm` has no special case for dimensions too high to imagine. It squares, it sums, it roots,

1 Vectors: Arrows and Lists

which is all it ever did. In everyday work the single line `np.linalg.norm(v)` is all we would write, but now when we write it we know exactly what it is doing underneath, because we built it ourselves out of a triangle we could see.

Where it lives in ML

Everything a model has ever learned from arrived as a vector. Not something like a vector, but a vector exactly: a list of numbers in fixed slots. It is the only kind of thing a machine can be handed in the first place.

A house is a vector. Three bedrooms, ninety square metres, built in 1974, becomes `[3, 90, 1974]`, and a model that predicts prices sees that and nothing else. No front door, no neighbourhood, no afternoon light through the kitchen window. Just the list.

An image is a vector too. We read off the brightness of every pixel in a fixed order and lay those values end to end, and a small photo turns into a list of a few hundred thousand numbers. To us the photo is a face. To the machine it is a very long list, and the face was never part of what it received.

A word is a vector as well, and this one is the strangest of the three. There is nothing about the word *coffee* we can measure the way we measured tempo, no natural quantity to read off. So the numbers are not measured at all; they are learned. A model reads enough text to work out that *coffee* belongs near *tea* and far from *bulldozer*, and it invents a few hundred numbers that place the word accordingly. None of those numbers means anything on its own. What matters here is only the shape of the answer: even for something as unmeasurable as a word, what the machine ends up holding is a list of numbers.

What all three share is worth noticing. In every case, somebody had to decide what each slot would hold. Bedrooms or bathrooms? Brightness or

colour? That decision is not a small piece of setup before the real work begins. In most machine learning projects, that decision is most of the work.

So where does the length, $\|\mathbf{v}\|$, actually earn its keep? Two places, both of which we will meet properly further on. The first is **normalising**. If we divide a vector by its own length, we get an arrow exactly one unit long that still points the same way, which lets us compare two things by direction alone while ignoring how big they are. It is how we stop a loud song from looking special simply because it is loud. The second is **gradient clipping**. During training, the signal a model uses to correct itself is a vector, and now and then its length grows out of control. So we measure that length, and if it has grown too large we scale it back before it damages the model. In both cases, measuring a length is the first step.

Now the part that quietly costs people weeks.

The obvious way to get this wrong is to put numbers in the wrong slots: tempo first for some songs and loudness first for others. Do that, and every length and every distance we compute is nonsense. The machine will not warn us, because it cannot. It never saw the arrow, only the list, and the list looked perfectly fine.

The subtler failure is worse, precisely because the arithmetic is completely correct. Go back to [128, 7]. Tempo lives in the hundreds and loudness lives in single digits, so when we square them we are comparing 16,384 against 49. Tempo was about eighteen times larger than loudness to begin with, but squaring stretches that gap into a factor of more than three hundred. The result is that $\|\mathbf{v}\|$ is no longer really the size of the song. It is very nearly just the tempo, with a faint trace of loudness left over where the real contribution should have been.

Nothing failed here and no error was raised. The formula did exactly what we derived it to do. The answer is useless anyway, because we fed it numbers living on wildly different scales and it had no way of knowing that one of them was being drowned out.

1 *Vectors: Arrows and Lists*

This is the first time in the book we run into the fact that being right about the maths is not enough on its own, and it will not be the last. Getting the formula correct is the easy half. Understanding what the numbers mean before we feed them in is the other half, and it is the half that decides whether anything works at all. This is exactly why normalising exists, and it is why the next section is about the ways this chapter can quietly mislead us.

Common misunderstandings

A vector is not a point. We drew $[3, 4]$ as an arrow ending at the spot three across and four up, and it is tempting to decide the vector simply is that spot. It is not. The vector is the journey: three to the right, then four up. It happened to end on that particular spot only because we chose to start drawing from the origin. Start the same journey somewhere else and it finishes somewhere else, even though the vector, the three-right-four-up, has not changed. This can feel like splitting hairs until the next section, where we add two vectors together. Adding two journeys makes plain sense, because we can do one and then the other. Adding two locations makes no sense at all. Keeping vectors and points separate now is what will let the next chapter work.

The numbers describe the arrow, not the other way round. Our song was $[128, 7]$ when we measured tempo in beats per minute. Measure tempo in beats per second instead and the very same song becomes $[2.13, 7]$. Every number in every vector in the dataset has shifted, and yet not a single song has changed. The arrow is the real thing; the list is only a description of it under a convention we happened to pick, and the instant we pick a different convention, the numbers move. It is worth remembering this the next time some numbers look strange: suspect the convention before you suspect the data.

Set aside “magnitude and direction.” Anyone who met vectors in school physics was told a vector is a thing with a magnitude and a direction, like a force or a velocity. That is a perfectly good picture for arrows in the physical world, but it can mislead us here. It puts direction and length first, as though they were what the vector truly is, and treats the numbers as a readout that comes second. In machine learning the order is reversed. The numbers come first. They are handed to us hundreds at a time, and most of them measure nothing we could ever point to. Direction and length are not what such a vector is; they are things we compute from its numbers when we happen to need them. A word embedding has no direction that anyone chose. It has a few hundred numbers, and a direction falls out of them if we ask for one.

The machine’s arithmetic is not exact. A little earlier we asked the machine whether the formula and the library agreed:

```
import numpy as np
v = np.array([3.0, 4.0])
print(np.sqrt(np.sum(v ** 2)) == np.linalg.norm(v))
```

True

It answered **True**, and we should admit that this was partly luck. The numbers were clean and the two calculations happened to land on exactly the same value. In general, computers store most numbers as very close approximations rather than exact values, so two different routes to what should be “the same” answer can end up a hair apart. Ask for exact equality and you will sometimes get **False** for two quantities that are equal in every way you actually care about. The honest question is whether they are close enough:

```
print(np.isclose(np.sqrt(np.sum(v ** 2)), np.linalg.norm(v)))
```

1 Vectors: Arrows and Lists

True

A reasonable habit is to use `==` only for whole numbers we are certain about, and to reach for `np.isclose` the moment a square root, a division, or a long sum is involved. It will save us a confusing afternoon at some point.

Higher dimensions are not simply “more of the same.” This is the important one, and it is a trap this chapter sets almost by accident. We spent two sections celebrating the fact that the formula follows the numbers into dimensions we cannot see, which is true and worth celebrating. The danger is walking away believing our intuition follows just as easily, as though three hundred dimensional space were ordinary flat space with more elbow room. It is not. Things happen out there that never happen on a page. Pick two vectors at random in three hundred dimensions, for instance, and they will almost always come out very nearly at right angles to each other, again and again, for reasons that feel wrong right up until we see why. The formulas will keep working perfectly. The instincts we built on flat paper will quietly stop being reliable. That gap between working formulas and failing intuition has a name, the curse of dimensionality, and Chapter 12 is devoted entirely to it.

Check your intuition

Try to reason each of these out before opening the answers. None of them is asking us to recall the formula. Each is asking us to use it.

1. What does the arrow for the vector $[0, 0]$ look like? How long is it, and which way does it point?
2. Two songs, where every number describing the second is exactly double the first: the first is $[60, 4]$ and the second is $[120, 8]$. If we draw both arrows, how do they compare?

Check your intuition

3. Two songs are measured as $[1000, 2]$ and $[1000, 9]$. If the only thing we are allowed to look at is the length of each vector, can we tell the two songs apart?
4. Is the vector $[3, 4]$ the same length as $[4, 3]$? Are they the same vector?
5. What is the length of $[1, 1, 1, 1]$? We cannot draw this one, so we will have to compute it instead.

 Answers

1. Its length is zero, and it does not point anywhere at all. It is not really an arrow, just a dot resting on the origin, having gone nowhere. That sounds like a technicality, but it matters later. Normalising a vector means dividing it by its length, and the zero vector is the one vector we cannot normalise, because that would mean dividing by zero. It is the exception we will always have to check for by hand.
2. They point in exactly the same direction, and the second is simply twice as long as the first. This is the whole idea of scaling: multiply every number in a vector by the same amount and the arrow stretches without ever turning. We will build on this in the next section, and it is also a first hint of why “which direction” and “how long” are two separate questions we can ask about the same arrow.
3. Only barely, and that is the lesson. The first length is $\sqrt{1000^2 + 2^2} = \sqrt{1000004} \approx 1000.002$, and the second is $\sqrt{1000^2 + 9^2} = \sqrt{1000081} \approx 1000.040$. The two songs differ by more than a factor of four in the quantity we actually care about, and yet their lengths agree to four decimal places. The 1000 drowns everything else out. Length alone is nearly blind here, for the very same different-scales reason we ran into a few sections ago.
4. They are the same length, since both give $\sqrt{9 + 16} = 5$, but they are not the same vector. The vector $[3, 4]$ walks three across and four up, while $[4, 3]$ walks four across and three up, so the two arrows point in different directions and end in different places. Length is one

1 Vectors: Arrows and Lists

fact about a vector. It is not the whole vector, and two genuinely different arrows are perfectly free to share it.

5. $\sqrt{1^2 + 1^2 + 1^2 + 1^2} = \sqrt{4} = 2$. There is no picture to draw and no page large enough to hold one, and the answer still comes out clean. Getting 2 without drawing anything is a sign that the central move of this chapter has taken hold: the formula does not care that we cannot see four dimensions, and by now, neither do we.

2 Adding, Scaling, and Span

The itch

By the end of the last chapter a song was a vector, a fixed arrow sitting at one spot in space, and we could measure how long that arrow was. That is enough to talk about one song at a time. It is not yet enough to talk about the things we actually care about, which are almost never single songs.

Think about what a playlist really is. Not a list of songs, but a kind of region: a mood, a texture, a stretch of the space where the songs we like tend to fall. A genre is the same, only larger. When we say two playlists “overlap,” or that one song sits “between” two others, or that some track is “what you’d get if you crossed these two artists,” we are talking about space in a way that a single fixed arrow cannot reach.

So the question for this chapter is about what we can do with vectors once we have more than one. Given a handful of songs, what other songs can we describe using only them? Can we build a point that sits halfway between two tracks? Can we take a direction we like and follow it further than any single song goes? Can we start with two or three vectors and map out the whole region they open up?

All of that turns out to rest on two operations, and only two. We can add vectors together, and we can scale a vector up or down. Everything in this chapter, up to and including the idea that a few vectors can lay out a whole region of space, is built from just those two moves. They are simple enough to define in a sentence each, and between them they are most of what linear algebra ever does.

The picture

Start with adding, because it is the one we already set up. At the end of the last chapter we said a vector is a journey, not a place: $[3, 4]$ means three across and four up, and it lands where it lands only because we started at the origin. If a vector is a journey, then adding two of them should mean the obvious thing. Do the first journey, and from wherever it leaves us, do the second.

Picture two vectors, $\mathbf{v}$ and $\mathbf{w}$. Walk $\mathbf{v}$ first: from the origin to the tip of $\mathbf{v}$. Now, without going back, walk $\mathbf{w}$ starting from where $\mathbf{v}$ ended. We finish somewhere new, and the arrow from the origin straight to that finishing point is the sum $\mathbf{v} + \mathbf{w}$. Laid out this way, tip of the first to tail of the second, the picture almost draws itself.

It does not matter which journey we take first. Walk $\mathbf{w}$ and then $\mathbf{v}$ and we land at exactly the same place, having traced the other two sides of the same parallelogram. Two vectors and their sum always close up into that shape, which is worth holding onto: adding vectors is not a single path but a parallelogram, and the sum is its far corner.

Scaling is the second move, and it is simpler. Take one vector and stretch it. If $\mathbf{v}$ is a journey, then $2\mathbf{v}$ is the same journey walked twice as far in the same direction, and $\frac{1}{2}\mathbf{v}$ is half of it. The arrow keeps pointing exactly where it did; only its length changes. Scaling never turns a vector, it only lengthens or shortens it.

One case is worth seeing before it surprises us. Scaling by a negative number, say -1 , does not break the rule so much as extend it. The arrow keeps to the same line it was always on, but now it points backwards along that line, to the far side of the origin. Negative scaling is still “the same direction” in the sense that matters here: the same line through space, walked the other way.

Now put the two moves together, because that is where the chapter is really going. Take two vectors, scale each one by however much we like,

The picture

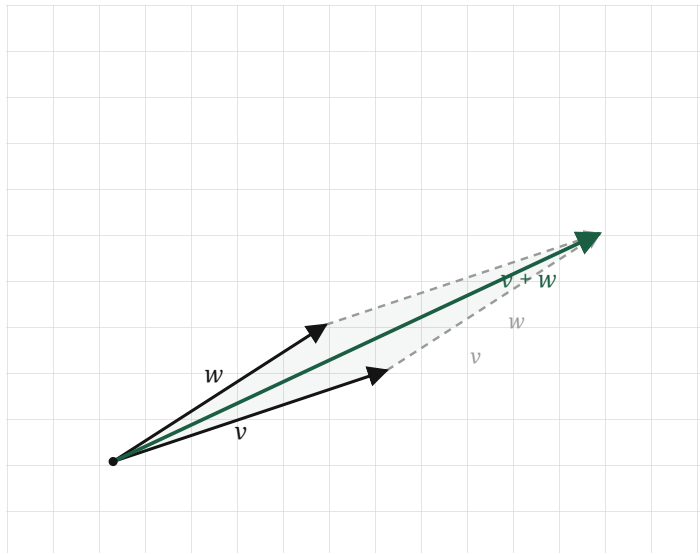


Figure 2.1: Two journeys, either order, same destination. The sum is the diagonal of the parallelogram they close up.

2 Adding, Scaling, and Span

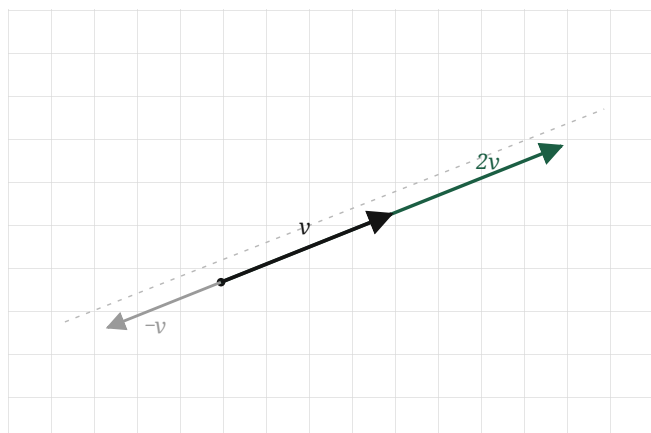


Figure 2.2: Scaling moves a vector along one fixed line. Stretching lengthens it, a negative factor sends it back through the origin, but it never leaves the line.

and add the results. Something like $2\mathbf{v} + 3\mathbf{w}$, or $\frac{1}{2}\mathbf{v} - \mathbf{w}$. A recipe of that shape, some amount of one vector plus some amount of another, is called a **linear combination**, and it is the single most important construction in this part of the book. Almost everything ahead is linear combinations wearing different clothes.

Once we can build linear combinations, one last question opens up on its own. If we are allowed every possible combination of $\mathbf{v}$ and $\mathbf{w}$, every amount of each, positive or negative, whole or fractional, what is the full set of places we can reach? That set has a name, the **span** of the two vectors, and picturing it is where we are headed next.

The math, built up

We have two moves, adding and scaling, and so far we only have them as pictures. To hand them to a machine we need them as arithmetic. Both

turn out to be about as simple as arithmetic gets.

Take addition first. We have $\mathbf{v} = [v_1, v_2]$ and $\mathbf{w} = [w_1, w_2]$, and we want the single arrow that is $\mathbf{v}$ followed by $\mathbf{w}$. Walking $\mathbf{v}$ moves us v_1 across and v_2 up. Walking $\mathbf{w}$ from there moves us a further w_1 across and w_2 up. So in total we have moved $v_1 + w_1$ across and $v_2 + w_2$ up, and that total is the sum:

$$\mathbf{v} + \mathbf{w} = [v_1 + w_1, v_2 + w_2]$$

That is the whole rule. Add vectors by adding the numbers that sit in the same slot: first with first, second with second. The tip-to-tail picture and this line of arithmetic are the same fact. One is what we see, the other is what we compute.

Scaling is even shorter. To walk $\mathbf{v}$ twice as far we double each step, and to walk it by any amount c we multiply each step by c :

$$c \mathbf{v} = [c v_1, c v_2]$$

Multiply every number in the list by the same amount. That is scaling, in full. Notice the negative case falls out for free: $c = -1$ multiplies each number by -1 , flipping the arrow to the far side of the origin exactly as the picture promised, with no new rule required.

Both operations extend to longer lists without a single change to the idea. A vector with three hundred numbers is added slot by slot, three hundred additions, and scaled by multiplying all three hundred numbers at once. Same rule, longer list, which is the pattern the whole book runs on.

Now we can write down the construction the chapter has been circling. Scale some vectors, add the results, and the recipe

$$c_1 \mathbf{v}_1 + c_2 \mathbf{v}_2 + \cdots + c_k \mathbf{v}_k$$

2 Adding, Scaling, and Span

is a **linear combination** of the vectors $\mathbf{v}_1$ through $\mathbf{v}_k$. The numbers c_1 through c_k are the amounts, one per vector, and they are allowed to be anything: positive, negative, fractional, zero. Every linear combination is just the two moves we already have, applied together and then read off as one arrow.

That last freedom, that the amounts can be *anything*, is what opens up a whole region rather than a few scattered points. Fix two vectors $\mathbf{v}$ and $\mathbf{w}$ and let their amounts range over every possible pair of numbers. The set of all the arrows we can build this way is called the **span** of $\mathbf{v}$ and $\mathbf{w}$.

For two vectors that point in different directions, the span is the entire flat plane. Every point on the page is reachable, because any spot can be arrived at by walking some amount of $\mathbf{v}$ and some amount of $\mathbf{w}$. Dial the two amounts and we can steer anywhere.

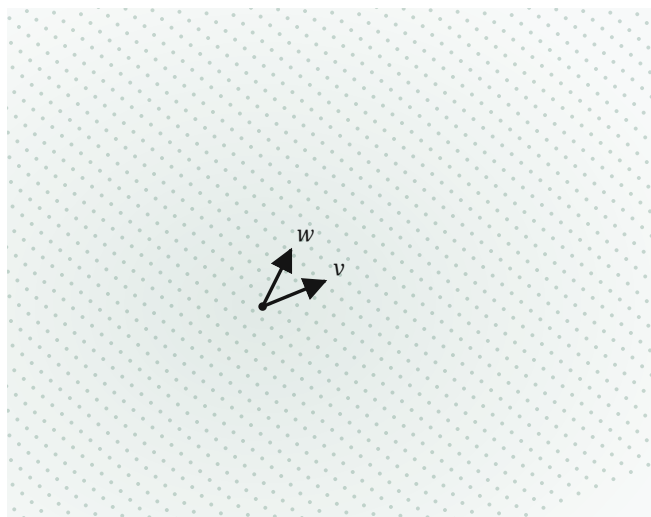


Figure 2.3: Every combination of two vectors that point different ways fills the whole plane. That filled region is their span.

Build it yourself

Both operations were simple enough on paper. In NumPy they are simpler still, because the componentwise rule is exactly how NumPy already thinks.

Start with two vectors and add them:

```
import numpy as np

v = np.array([2.0, 1.0])
w = np.array([1.0, 3.0])

print(v + w)
```

```
[3.  4.]
```

The result is `[3.0, 4.0]`, which is $[2+1, 1+3]$, first slot with first, second with second. We did not loop over the numbers or add them one at a time. We wrote `v + w` and NumPy added each slot to its partner on its own. That is the componentwise rule from the last section, and it is built into the `+` sign.

Scaling is the same story. Multiply a vector by a number and every slot is multiplied at once:

```
print(2 * v)
print(0.5 * v)
print(-1 * v)
```

```
[4.  2.]
[1.  0.5]
[-2. -1.]
```

2 Adding, Scaling, and Span

Three scalings, no loops. $2 * v$ doubles both numbers, $0.5 * v$ halves them, and $-1 * v$ flips their signs, which is the arrow pointing back through the origin exactly as we drew it.

Now put both moves together into a linear combination. Here is $2v + 3w$:

```
print(2 * v + 3 * w)
```

```
[ 7. 11.]
```

That single line scaled two vectors and added the results, and NumPy read it the way we read it on paper: scale, scale, add. Every linear combination we will ever write, however many vectors and whatever the amounts, is this same line made longer.

It is worth checking once that NumPy's $+$ really is doing the slot-by-slot rule and nothing else. We can spell the addition out by hand, building the result one slot at a time, and compare:

```
by_hand = np.array([v[0] + w[0], v[1] + w[1]])  
  
print(by_hand)  
print(v + w)  
print(np.array_equal(by_hand, v + w))
```

```
[3. 4.]
```

```
[3. 4.]
```

```
True
```

The hand-built version and $v + w$ are the same, and `np.array_equal` confirms it. There is no hidden cleverness in the $+$. It is the rule we derived, applied to each slot.

One last thing, the same one that mattered for length. None of this cared that our vectors had two numbers. Give them three hundred and every line above works unchanged: $\mathbf{v} + \mathbf{w}$ adds three hundred slots, $2 * \mathbf{v}$ scales three hundred numbers, and a linear combination stirs them together without a single edit. The picture ran out at three numbers. The arithmetic, and the code, did not.

Where it lives in ML

Adding and scaling look almost too simple to be the point. They are the point. Nearly everything a neural network does, from the first layer to the last, is linear combinations, the exact move we just built.

Picture a single artificial neuron, the basic unit of a network. It receives some numbers, the outputs of the neurons before it, as a vector. It holds its own vector of the same length, called its **weights**. To produce its answer it multiplies each incoming number by the matching weight and adds the results, then does one small extra step we will meet later. That multiply-each-and-add is a linear combination, with the weights as the amounts. A neuron is a linear combination with an opinion about which inputs matter, and that opinion is what the weights are.

A layer is many neurons at once, so a layer is many linear combinations of the same inputs, each with its own weights. A network is layers stacked on layers. Almost the whole structure, the part that gets trained, the part that holds what the model has learned, is linear combinations repeated at scale. When we say a model has seven billion parameters, we are mostly counting the amounts in a very large number of linear combinations.

Because the operations are this simple, something surprising becomes possible: we can do arithmetic on meaning. Recall that a word is a vector, a few hundred learned numbers. It turns out those vectors can be added and subtracted and the results still land on sensible words. The celebrated example is

2 Adding, Scaling, and Span

$$\text{king} - \text{man} + \text{woman} \approx \text{queen}$$

Read as vectors, this is only adding and scaling. Start at the vector for *king*, subtract the vector for *man*, which is scaling by -1 and adding, then add the vector for *woman*. The arrow that results lands nearest, of all the word vectors, to the one for *queen*. The direction from *man* to *woman* turns out to be roughly the same direction as from *king* to *queen*, so walking it moves us the way we would hope. Nobody built that in. It fell out of numbers a model chose on its own, and we reach it with nothing but the two moves from this chapter.

Now the failure worth knowing, and it is a quiet one again. Because a linear combination is only scaling and adding, everything a single layer can do stays within reach of those two moves, and that is a real limit. Scaling and adding can stretch space, flip it, and slide it around, but they cannot bend it. Stack a hundred layers that each only add and scale, and the whole stack still only adds and scales, no matter how tall it is. A network built purely from linear combinations, however many layers deep, can only draw straight lines. It could never tell two tangled-together classes apart, because no amount of stretching and sliding will separate them. And this straightness, lines staying lines and space never bending, is what the word *linear* has been pointing at all along. It is why the subject is called linear algebra.

This is why the “one small extra step” each neuron takes is not a detail. That step, a simple bending applied after the linear combination, is the whole reason stacking layers buys us anything. Without it, depth is an illusion and the network collapses back to a single linear combination. We will build that bend, and see exactly why it rescues us, when we reach neural networks in Part 4. For now the thing to hold onto is that the linear combination is the engine, and it is this chapter’s two operations, running at enormous scale.

Common misunderstandings

Span is not always the whole space. We ended the span section with two vectors filling the entire plane, which is the usual case but not the only one. Suppose the two vectors point along the same line, say $\mathbf{v} = [1, 1]$ and $\mathbf{w} = [2, 2]$. The second is just the first scaled by two, so it opens up no new direction. Every combination of them, whatever amounts we choose, lands somewhere on that one line through the origin. The span has collapsed from a plane to a line. Two vectors reach a plane only when they genuinely point different ways; if one is a scaled copy of the other, they are really one direction wearing two coats, and one direction spans only a line.

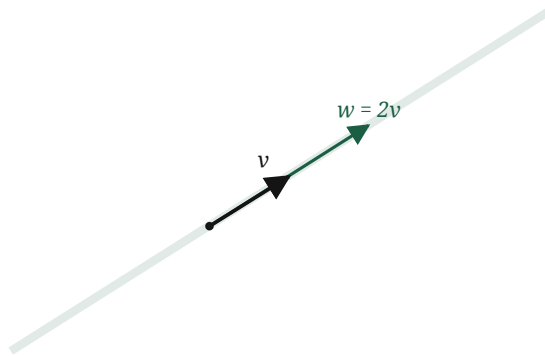


Figure 2.4: When one vector is a scaled copy of the other, every combination lands on a single line. The span collapses.

This is not a curiosity. It is the whole reason the next chapters care about words like independence and rank. The question “do these vectors actually point in different directions, or are some of them redundant” turns out to decide how much space a set of vectors can reach, and we will make it

2 Adding, Scaling, and Span

precise before long.

Scaling by a negative number is not the same as subtracting. It is tempting to read $-2\mathbf{v}$ as “take away two of $\mathbf{v}$,” as if we were removing something. Nothing is removed. Scaling by -2 builds a brand new arrow, twice as long as $\mathbf{v}$ and pointing the opposite way, and it exists in its own right. Subtraction only enters when we then add that arrow to another. Keep scaling and subtracting separate: scaling reshapes one arrow, subtracting is really adding a negatively scaled one.

Adding vectors does not average them. When we add $\mathbf{v}$ and $\mathbf{w}$ we land at the far corner of the parallelogram, which is further out than either vector alone. That is not the point “between” them. The midpoint is a different construction, $\frac{1}{2}\mathbf{v} + \frac{1}{2}\mathbf{w}$, a particular linear combination with both amounts set to a half. Plain addition and averaging feel similar because both mix two vectors, but they land in different places, and confusing them will put a point where you did not mean to put one.

A longer vector is not a more important one. After the last chapter it is easy to read length as weight, as though the longest vector in a combination should dominate. Length and role are separate. A short vector scaled by a large amount contributes more than a long vector scaled by almost nothing. What a vector brings to a combination is set by its amount, its c , not by how long the arrow happened to be to begin with. This is the same scales problem from the last chapter looked at from the other side: raw length is not the same as influence.

Check your intuition

As before, try each one before opening the answers. None of these asks you to recall a rule. Each asks you to use the two moves.

1. We have $\mathbf{v} = [2, 1]$ and $\mathbf{w} = [1, 2]$. What is $\mathbf{v} + \mathbf{w}$, and what is $\mathbf{v} - \mathbf{w}$? Draw both if it helps.

Check your intuition

2. Take a single vector $\mathbf{v} = [3, 1]$. Describe, in words, the full set of points you can reach using only scalings of $\mathbf{v}$, every possible amount. What shape is it?
3. We have two vectors $\mathbf{v} = [1, 0]$ and $\mathbf{w} = [2, 0]$. Between them, can we reach the point $[0, 1]$? Why or why not?
4. Find the amounts a and b that make $a[1, 0] + b[0, 1]$ equal to $[5, 3]$. Now do the same for any target $[x, y]$.
5. Is the midpoint between $\mathbf{v}$ and $\mathbf{w}$ a linear combination of them? If so, what are the two amounts?

 Answers

1. Adding slot by slot, $\mathbf{v} + \mathbf{w} = [2 + 1, 1 + 2] = [3, 3]$. Subtracting, $\mathbf{v} - \mathbf{w} = [2 - 1, 1 - 2] = [1, -1]$. Notice the sum leans up and to the right, halfway in feel between the two originals in direction though longer than either, while the difference points down and to the right, off toward where $\mathbf{v}$ sits relative to $\mathbf{w}$. Subtraction is just adding $-\mathbf{w}$, and $-\mathbf{w} = [-1, -2]$ points the opposite way to $\mathbf{w}$, which is why the result swings below the axis.
2. Every scaling of $\mathbf{v} = [3, 1]$ is some amount c times it, $[3c, c]$. As c runs through all values, positive, negative, and zero, these points trace the entire straight line through the origin that passes along $\mathbf{v}$'s direction. The shape is a line: one vector spans a line, exactly the collapsed case from the misunderstandings. Positive c gives the half heading out along $\mathbf{v}$, negative c the half heading back through the origin, and $c = 0$ the origin itself.
3. No. Both $\mathbf{v} = [1, 0]$ and $\mathbf{w} = [2, 0]$ have a second number of zero, and $\mathbf{w}$ is simply $2\mathbf{v}$, so any combination $a\mathbf{v} + b\mathbf{w}$ has the form $[a + 2b, 0]$. The second slot is always zero, no matter what a and b we pick. The point $[0, 1]$ has a second slot of one, so it can never be reached. These two vectors point the same way and span only the

2 Adding, Scaling, and Span

horizontal line, and $[0, 1]$ is off that line.

4. Here $a[1, 0] + b[0, 1] = [a, b]$, so to hit $[5, 3]$ we need $a = 5$ and $b = 3$. For a general target $[x, y]$ the same reading gives $a = x$ and $b = y$ directly. These two vectors are special: the amounts you need are just the target's own numbers. That is not a coincidence, and it is the first glimpse of why $[1, 0]$ and $[0, 1]$ are the natural reference vectors for the plane, a thread we pick up when we reach basis.

5. Yes. The midpoint is $\frac{1}{2}\mathbf{v} + \frac{1}{2}\mathbf{w}$, a linear combination with both amounts equal to a half. It sits squarely between the two arrow tips, which is exactly what averaging two things should give, and it is a different point from $\mathbf{v} + \mathbf{w}$, the far corner of the parallelogram. Halving both amounts pulls the sum back to the middle.

3 The Dot Product

The itch

Two chapters ago we asked how a machine could tell that two songs are similar, and we have been carrying that question ever since. We turned each song into a vector, an arrow in space. We learned to measure how long one arrow is. We learned to add two arrows and to scale them. But we never answered the actual question. Length tells us about one arrow on its own. Adding and scaling let us build new arrows. None of it tells us how *alike* two arrows are.

Look at what “alike” should mean, now that songs are arrows. Two similar songs became two arrows pointing much the same way. Two songs with nothing in common became two arrows pointing off in different directions. So similarity is about the *angle* between two arrows. Small angle, alike. Wide angle, different. Pointing opposite ways, as unlike as it gets.

That reframes the whole problem into something almost embarrassingly concrete. We do not need to understand music. We need to measure the angle between two arrows, using nothing but the lists of numbers the machine actually holds. If we can turn two lists of numbers into a single number that says “these point the same way” or “these point apart,” we are done. The recommendation, the face unlock, the search result, all of it comes down to that one number.

There is a single operation that produces it. It is the most useful tool in this part of the book, the one the first chapter promised and the last one kept pointing at. It takes two vectors and returns one number, and

3 The Dot Product

packed inside that number are the angle between them, the shadow one casts on the other, and the similarity we have been chasing since the first page. It is called the dot product, and everything in this chapter is that one operation seen from three sides.

The picture

We want the angle between two arrows, but angle is an awkward thing to measure directly from two lists of numbers. So we come at it sideways, through a picture that turns out to carry the angle inside it: the shadow one arrow casts on another.

Take two vectors, $\mathbf{v}$ and $\mathbf{w}$, drawn from the same origin. Now imagine light coming straight down onto $\mathbf{w}$, at right angles to it, and look at the shadow $\mathbf{v}$ casts along the line of $\mathbf{w}$. That shadow has a length, and it is this shadow that quietly knows the angle.

Watch what the shadow does as we swing $\mathbf{v}$ around. When $\mathbf{v}$ points the same way as $\mathbf{w}$, it lies flat along $\mathbf{w}$ and its shadow is as long as $\mathbf{v}$ itself. Nothing is lost, because nothing is off to the side. As $\mathbf{v}$ tilts away, the shadow shortens. When $\mathbf{v}$ stands at a right angle to $\mathbf{w}$, straight up from it, it casts no shadow along $\mathbf{w}$ at all. The shadow has shrunk to nothing. Tilt $\mathbf{v}$ further, past the right angle, and it begins to cast a shadow the other way, pointing backwards along $\mathbf{w}$, which we read as a negative length.

So the shadow's length is a quiet report on the angle. Full-length shadow means the arrows are aligned. Shrinking shadow means the angle is opening. Zero shadow means a right angle, arrows completely unrelated in direction. Negative shadow means they lean opposite ways. We have turned “what is the angle” into “how long is the shadow,” and length is something we already know how to compute from lists of numbers.

There is one wrinkle to be honest about now, because it shapes the formula ahead. The shadow depends on two things at once, not one. It depends

The picture

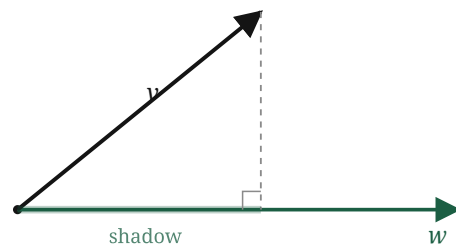


Figure 3.1: As $\mathbf{v}$ swings away from $\mathbf{w}$, the shadow it casts along $\mathbf{w}$ shrinks. Aligned, the shadow is full length. At a right angle, it vanishes. Past that, it points backwards.

3 The Dot Product

on the angle, which is what we are chasing, but also on how long $\mathbf{v}$ is to begin with, since a longer arrow casts a longer shadow at the same angle. And when we build the actual operation, it will fold in the length of $\mathbf{w}$ too. The dot product does not hand us the angle cleanly on its own. It hands us the angle tangled together with both lengths, and part of the work in this chapter is learning to separate them back out when we want the angle alone.

The math, built up: the formula

We have a shadow that knows the angle, and we need it as arithmetic on two lists. Rather than reach for a formula, let us build it from the simplest possible case and let the general rule assemble itself.

Start with the two reference vectors from the last chapter, $[1, 0]$ and $[0, 1]$, pointing straight along the axes. Ask for the shadow of $[1, 0]$ on $[0, 1]$. One points along the ground, the other straight up, at a perfect right angle, so from the picture the shadow is zero. Now the shadow of $[1, 0]$ on itself: the arrow lies flat along its own direction, so the shadow is its full length, one.

Those two facts are the whole engine:

- along the same axis, the shadow is full,
- across a right angle, the shadow is nothing.

Now take two general vectors, $\mathbf{v} = [v_1, v_2]$ and $\mathbf{w} = [w_1, w_2]$, and split each into its along-the-axes pieces. The v_1 part of $\mathbf{v}$ points purely along the first axis, the v_2 part purely along the second, and the same for $\mathbf{w}$. To ask how much of $\mathbf{v}$ falls along $\mathbf{w}$, we add up the contributions piece by piece, and by the two facts above, only the matching pieces survive.

First axis meets first axis, and those align, so they contribute $v_1 w_1$. Second meets second, contributing $v_2 w_2$. First axis meets second axis sits at a

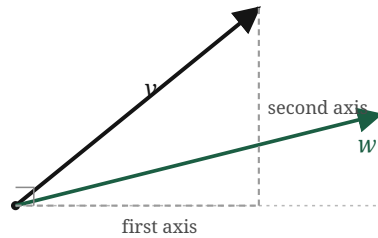


Figure 3.2: Each vector splits into an along-first-axis piece and an along-second-axis piece. Matching pieces line up and contribute; the crossing pair sits at a right angle and contributes nothing.

right angle and contributes nothing. The crossing terms vanish, and what remains is a sum of matching products:

$$\mathbf{v} \cdot \mathbf{w} = v_1 w_1 + v_2 w_2$$

That is the **dot product**. Multiply the numbers in matching slots and add the results. The little dot is the whole notation. It takes two vectors and returns a single number, and that number is the shadow relationship from the picture, now written purely in the lists the machine holds.

It is the same slot-by-slot pattern as length and addition, and like them it does not care how long the lists are. Three hundred numbers means three hundred products summed into one:

$$\mathbf{v} \cdot \mathbf{w} = v_1 w_1 + v_2 w_2 + \cdots + v_n w_n$$

What the dot product contains

Before we open it up, one thing needs saying plainly, because it is the weld holding this whole chapter together. We built the dot product two ways and have not yet admitted they are the same thing. One way was a picture: the shadow one arrow casts on another. The other was arithmetic: multiply matching slots and add. These are not two facts about the dot product. They are one number reached by two roads.

The number you get from multiplying and adding is exactly the length of $\mathbf{w}$ times the length of $\mathbf{v}$'s shadow on $\mathbf{w}$. That equality is the bridge. On one side sits arithmetic the machine can run on two lists. On the other sits geometry we can see. The dot product is the single thing standing on both sides at once, which is why one small sum of products can tell us about shadows, lengths, and angles all together.

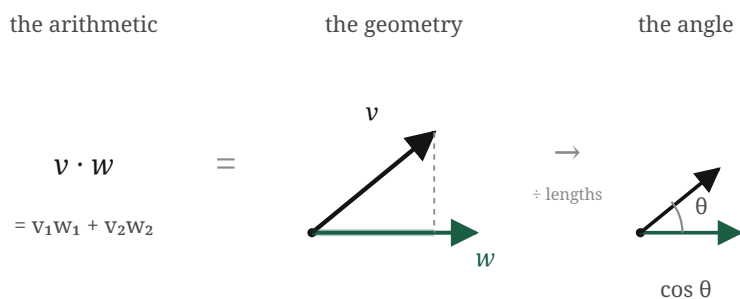


Figure 3.3: One number, three views. The arithmetic (multiply matching slots, add) equals the geometry (how much of one arrow lies along the other, scaled by its length), and stripping the two lengths out of that leaves the angle alone.

What the dot product contains

Read the three faces below as three consequences of that one bridge, not three separate discoveries.

It contains length. Take the dot product of a vector with *itself*. Every slot multiplies by itself, giving $v_1^2 + v_2^2 + \dots$, which is exactly what sat under the square root when we measured length. So

$$\mathbf{v} \cdot \mathbf{v} = \|\mathbf{v}\|^2.$$

The dot product of a vector with itself is its length squared. Length was never a separate idea; it was the dot product looking in a mirror.

It contains the angle. This is the relation the chapter turns on, tying the arithmetic back to the shadow:

$$\mathbf{v} \cdot \mathbf{w} = \|\mathbf{v}\| \|\mathbf{w}\| \cos \theta,$$

where θ is the angle between the arrows. It says exactly what the picture warned it would: the dot product is the angle, through $\cos \theta$, tangled together with both lengths. Not the angle alone, but the angle wearing the two lengths as coats. We will not derive this from scratch here; it is the law of cosines in vector dress, and the derivation waits in the appendix for anyone who wants it. What matters is reading it.

When the arrows align, θ is zero, $\cos \theta$ is one, and the dot product is just the two lengths multiplied, as large as it can be. At a right angle, $\cos \theta$ is zero and the whole thing collapses to zero, however long the arrows are. Pointing opposite ways, $\cos \theta$ is -1 and the dot product is as negative as it gets. The sign carries the headline: positive means the arrows lean together, zero means perpendicular, negative means they lean apart.

It contains a test for right angles. That middle case is the one we will lean on hardest, so it gets its own line. Two vectors are perpendicular exactly when their dot product is zero:

3 The Dot Product

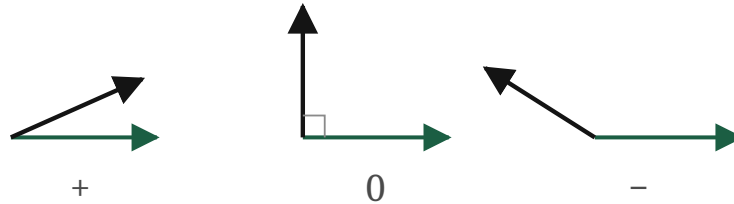


Figure 3.4: The same two arrows at three angles. Aligned, the dot product is large and positive. At a right angle it is zero. Pointing apart, it is negative. The sign alone tells the basic story.

$$\mathbf{v} \cdot \mathbf{w} = 0 \iff \mathbf{v} \text{ and } \mathbf{w} \text{ are perpendicular.}$$

No angle measured, no trigonometry. Multiply the slots, add them, and if the answer is zero the arrows stand at a right angle. It is the cleanest test in linear algebra, and it falls out of a sum of products for free.

Build it yourself

The dot product is a sum of matching products, and that is exactly as short in NumPy as it sounds.

Start with two vectors and form it by hand, the way the formula reads: multiply matching slots, then add:

```
import numpy as np
```

```
v = np.array([2.0, 1.0])
w = np.array([1.0, 3.0])

products = v * w
print(products)
print(np.sum(products))
```

```
[2. 3.]
5.0
```

`v * w` multiplies slot by slot, giving `[2.0, 3.0]`, and summing those gives 5.0. That is $2 \cdot 1 + 1 \cdot 3$, the dot product spelled out in two steps. NumPy also has it as one built-in:

```
print(np.dot(v, w))
```

```
5.0
```

Same 5.0. `np.dot` multiplies the matching slots and adds them, nothing more, exactly what we did above.

Now let us confirm the three faces from the last section, in code, one at a time.

Length. The dot product of a vector with itself should be its length squared:

```
print(np.dot(v, v))
print(np.linalg.norm(v) ** 2)
```

```
5.0
5.0000000000000001
```

3 The Dot Product

Both give 5.0. The self-dot and the squared length are the same number, as promised.

The right-angle test. Two perpendicular vectors should give a dot product of zero. Take $[1, 0]$ and $[0, 1]$, which point straight along the two axes:

```
a = np.array([1.0, 0.0])
b = np.array([0.0, 1.0])
print(np.dot(a, b))
```

0.0

Zero, with no angle measured and no trigonometry. The sum of products came out to zero on its own, and that is the whole test.

The angle. We can run the tangled relation backwards to pull the actual angle out. Since $\mathbf{v} \cdot \mathbf{w} = \|\mathbf{v}\| \|\mathbf{w}\| \cos \theta$, we divide the dot product by the two lengths to get $\cos \theta$, then undo the cosine:

```
cos_theta = np.dot(v, w) / (np.linalg.norm(v) * np.linalg.norm(w))
theta = np.arccos(cos_theta)

print(cos_theta)
print(np.degrees(theta))
```

0.7071067811865475
45.000000000000001

The angle between $\mathbf{v}$ and $\mathbf{w}$ comes out to about 45 degrees. Notice what we did: dividing by the two lengths stripped the coats off, leaving $\cos \theta$ alone, and `arccos` turned that back into an angle. Separating the angle

from the lengths, which sounded abstract in the last section, is three lines of arithmetic.

And, as ever, none of this depended on the vectors being short. What is worth noticing here is not just that the dot product survives three hundred numbers, but that all three faces survive together. `np.dot` still sums the products, the self-dot is still the length squared, the zero test still finds a right angle, and the angle still separates out, all from the same one line, all in a space no one can picture. One operation carried the whole toolkit up there at once.

Where it lives in ML

We can finally answer the question we opened the book with. How does a machine know two songs are similar? It takes their two vectors and computes one dot product, then divides out the two lengths. What is left is $\cos \theta$, a single number between -1 and 1 that says how nearly the two arrows point the same way. Close to 1 , nearly aligned, nearly the same song. Close to 0 , unrelated. Below zero, opposite. That number has a name, **cosine similarity**, and it is the dot product with the lengths stripped off, exactly the move we made three lines into the last section.

$$\text{cosine similarity} = \frac{\mathbf{v} \cdot \mathbf{w}}{\|\mathbf{v}\| \|\mathbf{w}\|} = \cos \theta$$

Dividing the lengths out is not a technicality. It is the whole reason cosine similarity is used instead of the raw dot product. Remember the loud-song problem from the first chapter: a track that is loud on every measure has a long vector, and a raw dot product rewards it for being long, not for being similar. Stripping the lengths cancels that. Two songs count as similar when they point the same way, whatever their volumes, which is what we wanted “similar” to mean all along. The angle is the honest measure; the length was never the point.

3 *The Dot Product*

This one operation, dot product then normalise, is not a magic trick. It is how similarity is measured almost everywhere in machine learning.

Search runs on it. Your query becomes a vector, every document becomes a vector, and the engine returns the documents whose vectors point most nearly the same way as your query's. "Relevant" means small angle.

Recommendation runs on it. Your taste is a vector, every item is a vector, and what gets recommended is whatever points your way. The whole idea of "people like you liked this" is a search for small angles in a space of vectors.

Retrieval in modern language systems runs on it. When a model needs to find the passage most relevant to a question, it embeds both as vectors and ranks by cosine similarity. The reason a chatbot can pull the right paragraph out of a thousand-page manual is a great many dot products, each one a sum of matching products.

And then there is attention, the operation at the heart of the models behind the current wave of AI. A model reading a sentence needs to decide, for each word, which other words matter to it. It does this by giving every word two vectors, a query and a key, and taking the dot product of one word's query with every other word's key. A large dot product means "this word is relevant to that one, attend to it." That is the entire mechanism in one line: relevance is a dot product, and the model steers its attention by where those products come out large. The operation you just built from a shadow on the floor is, run billions of times, most of what a transformer does when it reads.

Now the failure, and in this chapter it is loud rather than quiet, because it is the one that breaks recommendation systems in practice. The dot product only means "similarity" when the slots line up. If document vectors and query vectors were built by different recipes, so that slot 7 means one thing in one and something else in the other, the dot product still returns a number, confidently, and the number is noise. Everything in this chapter assumed both vectors live in the same space with the same meaning in

every slot. Break that assumption and the arithmetic keeps working while the meaning quietly drains out, which is the hardest kind of bug to see: the one that never raises an error.

Common misunderstandings

The dot product is simple to compute and easy to misread. These are the misreadings worth heading off.

The dot product is not vector multiplication. It is tempting to see $\mathbf{v} \cdot \mathbf{w}$ as “multiplying two vectors,” the way we multiply two numbers. It is not, and the giveaway is the output. Multiply two numbers and you get a number. Multiply two vectors slot by slot, with $\mathbf{v} * \mathbf{w}$, and you get a *vector*, the list of matching products. The dot product takes that vector and adds its entries, collapsing it to a single number. So the dot product is not multiplication; it is multiplication *followed by adding up*, and that final sum is what changes its character entirely. Watch for this in code especially: $\mathbf{v} * \mathbf{w}$ and `np.dot(v, w)` are different operations, one returning a vector and one a number, and reaching for the wrong one is a common bug.

A dot product of zero does not mean “unrelated” in meaning. We built the clean fact that a zero dot product means the arrows are perpendicular. It is easy to slide from “perpendicular” to “nothing to do with each other,” but those are not the same claim. Perpendicular is a statement about direction in the vector space, nothing more. Two word vectors might come out perpendicular, dot product zero, and still be deeply related in meaning in ways the particular vectors simply do not capture. The dot product reports on the geometry we built, faithfully. Whether that geometry captures the meaning we care about is a separate question, and a much harder one.

Stripping the lengths is not always what you want. Cosine similarity divides out both lengths to get the pure angle, and for comparing songs or

3 The Dot Product

documents that is right, because length was contaminating the answer. But the length is not always noise. Sometimes how *much* of something there is genuinely matters, and throwing the lengths away throws real information away with them. Cosine similarity is the right tool when direction is what carries meaning and size does not. Deciding which of those you are in is a judgement about your data, not something the formula makes for you.

arccos can hand you a nan. When you recover an angle by dividing the dot product by the two lengths and calling `arccos`, the arithmetic can bite. In exact maths, $\cos \theta$ for two vectors always lands between -1 and 1 . In floating point it can drift a hair outside that range from rounding alone, and the drift goes either way. Take a vector, scale it to length one, and dot it with itself. The true answer is exactly one, a vector makes a zero angle with itself, but the machine does not always land exactly there:

```
import numpy as np

u = np.array([1.0, 1.0, 1.0])
u = u / np.linalg.norm(u)           # scale to length 1

cos_theta = np.dot(u, u)
print(cos_theta)                     # 1.0000000000000002, just over the limit
```

```
1.0000000000000002
```

That last digit is the whole problem. Feed a value above one to `arccos`, which is undefined there, and it does not raise an error. It returns `nan`, the quiet not-a-number that then poisons everything downstream:

```
print(np.arccos(cos_theta))          # nan
```

```
nan
```

Check your intuition

```
/tmp/ipykernel_28586/334907105.py:1: RuntimeWarning: invalid value encountered in arccos
print(np.arccos(cos_theta))          # nan
```

This is the same approximate-arithmetic issue from the first chapter, surfacing in a new place. The standard guard is to clip the value back into range before taking the inverse cosine:

```
theta = np.arccos(np.clip(cos_theta, -1.0, 1.0))
print(np.degrees(theta))
```

0.0

Clipping forces the value back to exactly one before `arccos` sees it, and the angle comes out as zero, which is correct. Reach for `np.clip` any time you feed a computed cosine into `arccos`.

Check your intuition

Try each before opening the answers. As always, these ask you to use the dot product, not recite it.

1. Compute $\mathbf{v} \cdot \mathbf{w}$ for $\mathbf{v} = [3, 0]$ and $\mathbf{w} = [0, 5]$. What does the answer tell you about the angle between them, without computing the angle?
2. Two vectors have a positive dot product. Then we replace $\mathbf{w}$ with $-\mathbf{w}$. What is the sign of the new dot product, and what has happened to the angle?
3. We have $\mathbf{v} = [1, 1]$ and $\mathbf{w} = [3, 3]$. Compute the dot product, and compute the cosine similarity. Why is one of these large and the other exactly 1?

3 The Dot Product

4. Without computing anything, which pair is more similar in direction: two vectors with a dot product of 50, or two vectors with a dot product of 10?
5. A vector's dot product with itself is 0. What can you say about the vector?

Answers

1. The dot product is $3 \cdot 0 + 0 \cdot 5 = 0$. A zero dot product means the two arrows are perpendicular, so they sit at a right angle, without our measuring any angle to find out. This matches the picture: $[3, 0]$ points along the first axis and $[0, 5]$ straight up the second, and the axes meet at a right angle.
2. Flipping $\mathbf{w}$ to $-\mathbf{w}$ negates every slot, so every product in the sum flips sign, and the dot product flips from positive to negative. Geometrically, reversing $\mathbf{w}$ swings it to point the opposite way, which opens the angle past a right angle: two arrows that leaned together now lean apart. Positive to negative is exactly the crossing from acute to obtuse.
3. The dot product is $1 \cdot 3 + 1 \cdot 3 = 6$. The cosine similarity divides that by the two lengths: $\|\mathbf{v}\| = \sqrt{2}$ and $\|\mathbf{w}\| = \sqrt{18}$, so the similarity is $6/(\sqrt{2} \cdot \sqrt{18}) = 6/6 = 1$. The dot product is large partly because $\mathbf{w}$ is long, three times the length of $\mathbf{v}$. Cosine similarity strips that length away and reports pure direction, and since $\mathbf{w}$ is just $\mathbf{v}$ scaled up, the two point in exactly the same direction: similarity 1. This is the loud-song problem in miniature. The raw dot product is inflated by length; the cosine sees only direction.
4. You cannot tell. The raw dot product mixes direction together with both lengths, so a dot product of 50 might come from two barely-aligned but very long vectors, while a 10 might come from two short vectors pointing almost exactly the same way. Without dividing out the lengths, the raw number does not rank similarity.

Check your intuition

This is precisely why cosine similarity exists, and why the question has no answer as asked.

5. The dot product of a vector with itself is its length squared, so if that is zero, the vector has length zero. The only vector with length zero is the zero vector, all slots zero. It is the same exception we met when normalising: the one vector that is its own perpendicular, because it has no direction at all.

4 Length, Distance, and Norms

The itch

Back at the start we measured the length of an arrow with Pythagoras, squared the numbers, summed them, took the root, and called it the length. It was the length, and nothing in this chapter takes that back. But there was a quiet assumption hiding inside it, one we never said out loud, and it is worth dragging into the light now because it turns out we had a choice we did not know we were making.

Here is the assumption. When we measured the distance from the origin to the tip of the arrow, we measured it as the crow flies: a single straight line, cutting diagonally across, ignoring everything in the way. That is one honest way to measure distance. It is not the only one.

Think about actually getting somewhere in a city laid out in blocks. You cannot cut diagonally through the buildings. You walk so far along one street, then turn and walk so far along another, and the distance you actually travel is the sum of those two walks, not the diagonal a bird would fly. Same two points, same start and finish, and a completely different, equally real number for how far apart they are. The bird and the pedestrian disagree, and neither is wrong. They are answering different questions.

So “how far apart are these two vectors” does not have one answer. It has several, depending on what kind of travel we have in mind, and each kind gives a different notion of length. The straight-line length from the first chapter is one member of a family. This chapter is about the family, what the other members measure, and the one question in machine learning

4 Length, Distance, and Norms

where picking the right member changes everything: why one of them quietly forces a model to throw away features it does not need.

The picture

Put two points on a grid: the origin, and a point three streets across and four streets up. We already know what the straight-line distance is, because we computed exactly this in the first chapter. Pythagoras gives $\sqrt{3^2 + 4^2} = 5$. The bird flies five.

Now walk it as a pedestrian. Three blocks across, four blocks up, and it does not matter which order or how we zigzag, every legal path that only moves along streets covers the same total: $3 + 4 = 7$. The pedestrian walks seven. Two honest distances for the same two points, five and seven, and the only difference is whether we are allowed to cut the corner.

These two ways of measuring have plain names. The straight-line distance, the one from chapter one, is the **L2 norm**. The city-block distance, the sum of how far we move along each axis, is the **L1 norm**. The names will make more sense once we see the arithmetic, but the pictures are the thing to hold: L2 cuts across, L1 keeps to the grid.

There is a second way to see the difference, and it is the one that matters most later, so it is worth setting up carefully now. Ask a simple question of each norm: what does “distance exactly one from the origin” look like? Collect every point that sits at distance one, and see what shape they trace.

For the L2 norm, the straight-line one, every point at distance one is exactly a circle. That is what a circle *is*, the set of points a fixed straight-line distance from a centre. No surprise there.

For the L1 norm, the city-block one, the shape is not a circle. The points at city-block distance one from the origin trace a diamond, a square balanced on its corner, with its points sitting out on the axes and its flat sides

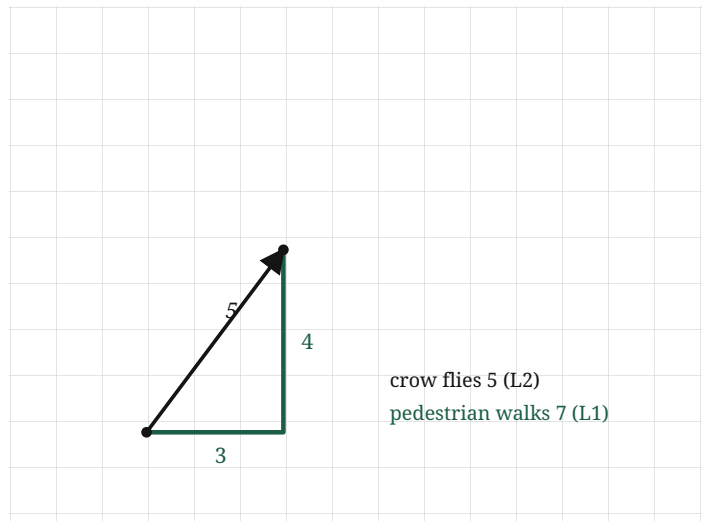


Figure 4.1: The same two points, two distances. The straight diagonal is the crow's five. The path along the streets is the pedestrian's seven. Neither is wrong; they permit different moves.

4 Length, Distance, and Norms

cutting across the diagonals. It looks strange until you check a corner against a side. The point one step out along an axis, straight out and nowhere else, is city-block distance one. But a point out on the diagonal has to spend its budget of one on *both* axes at once, so it cannot reach as far along either, and the boundary pulls in. Full reach along the axes, pinched on the diagonals: that is a diamond.

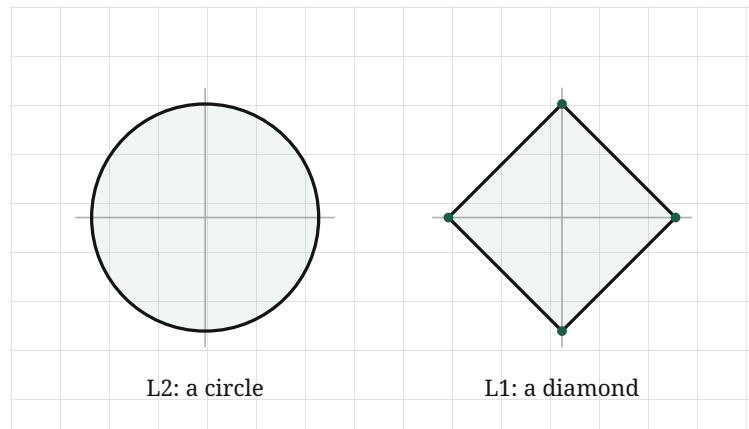


Figure 4.2: Distance one from the origin, drawn for each norm. Under L2 it is a circle. Under L1 it is a diamond, reaching furthest along the axes and pinched in on the diagonals.

Hold onto that diamond. The fact that L1's shape has sharp corners sitting exactly on the axes, while L2's is smooth all the way round, looks like a geometric curiosity right now. It is not. When we reach machine learning it will turn out to be the entire reason L1 does something L2 cannot, but we are not ready to say why yet. For now, just notice that the corners are on the axes, and that the circle has no corners at all.

The math, built up

Both norms are easy to write down, and setting them side by side shows exactly what they share and where they part.

Start with the one we already have. The L2 norm squares each number, sums, and takes the root:

$$\|\mathbf{v}\|_2 = \sqrt{v_1^2 + v_2^2 + \cdots + v_n^2}$$

The only new thing is the small 2 tucked under the bars. In the first chapter we wrote $\|\mathbf{v}\|$ with no subscript, because there was only one length in the room and it needed no label. Now that there is more than one, the subscript says which. The bare $\|\mathbf{v}\|$ from here on still means the L2 norm; it is the default, and the subscript is there for when we need to be explicit.

The L1 norm is simpler, and in a way more honest about what it does. No squaring, no root. Take the size of each number, ignoring its sign, and add them up:

$$\|\mathbf{v}\|_1 = |v_1| + |v_2| + \cdots + |v_n|$$

That is the city-block distance in symbols. Each $|v_i|$ is how far we walk along one street, and the norm is the total walk. The absolute value bars are doing the “how far, never mind which way” that a pedestrian cares about: walking three blocks west is three blocks of walking, not minus three.

Look at what the two formulas share. Both take the numbers of the vector, do something to each one that throws the sign away, and add the results. L2 throws the sign away by squaring, which also stretches big numbers out; L1 throws it away with the absolute value, which treats every unit of distance the same. That single difference, squaring versus not, is the whole

4 Length, Distance, and Norms

reason their unit shapes came out so differently, a smooth circle against a cornered diamond.

The naming is not an accident either. There is a whole family here, one norm for each choice of a number p : raise each $|v_i|$ to the power p , sum, and take the p -th root. Set $p = 2$ and the squares and square root give L2. Set $p = 1$ and the powers do nothing, leaving the plain sum of absolute values, L1. Other values of p exist and occasionally matter, but two members of this family carry almost all the weight in machine learning, and they are the two we have. We will not need the general machine again; it is enough to know the two norms are cousins, not strangers, and that the subscript is pointing at which cousin.

One value of p is worth naming just so it does not seem mysterious later. As p grows without bound, the norm comes to care only about the single largest entry, and in the limit it *is* the largest absolute entry, called the L-infinity norm. We will meet it only in passing. The two that earn their keep are L1 and L2.

Build it yourself

Both norms are short in NumPy, and building each by hand first shows there is nothing to either one but the formula.

Take a vector and compute its L2 norm the long way, squaring, summing, rooting, exactly as we did in the first chapter:

```
import numpy as np

v = np.array([3.0, -4.0])

l2_by_hand = np.sqrt(np.sum(v ** 2))
print(l2_by_hand)
```

5.0

Five, the crow's distance, and notice it did not care that the second number was negative: squaring removed the sign along the way. Now the L1 norm by hand, taking the size of each number and adding:

```
l1_by_hand = np.sum(np.abs(v))
print(l1_by_hand)
```

7.0

Seven, the pedestrian's distance. `np.abs` throws the signs away, three and four, and the sum is seven. Two numbers from the same vector, five and seven, the same pair we drew.

NumPy gives both through one function, `np.linalg.norm`, steered by an argument called `ord` that picks which norm. Left alone it gives L2, the default we have been leaning on:

```
print(np.linalg.norm(v))          # L2, the default
print(np.linalg.norm(v, 2))       # L2, asked for explicitly
print(np.linalg.norm(v, 1))       # L1
```

5.0

5.0

7.0

The first two both print five, the same number, one by default and one by request. The third prints seven. The single argument is the whole difference between the crow and the pedestrian, and it lines up with the subscript from the last section: `ord=1` is $\|\mathbf{v}\|_1$, `ord=2` is $\|\mathbf{v}\|_2$.

It is worth confirming, once, that the built-in really is doing our two hand formulas and nothing more:

4 Length, Distance, and Norms

```
print(np.linalg.norm(v, 2) == l2_by_hand)
print(np.linalg.norm(v, 1) == l1_by_hand)
```

```
True
True
```

Both `True`. No cleverness hiding in the library: `ord=2` squares-sums-roots, `ord=1` sums absolute values, exactly what we wrote out.

And, as with every operation so far, none of this depends on the vector being short. Give it three hundred numbers and both norms still come back, each on its one line, `ord` still choosing which kind of distance we mean in a space we could never draw.

Where it lives in ML

The two norms do different jobs in machine learning, and the L1 norm does one strange and valuable thing that the diamond has already half-shown us. This is where that shape pays off.

Start with the ordinary job, the one both norms share. A trained model is a big pile of numbers, its weights, and left unchecked those numbers tend to grow large during training, which makes the model brittle: it fits the training data too tightly and stumbles on anything new. A standard fix is to add a penalty for large weights, so the model is pushed to keep them small while it learns. The size of the weights is measured by a norm, and now the choice of norm matters. Penalise with the L2 norm and the model is pushed to make all its weights small. Penalise with the L1 norm and something different happens: the model is pushed to make many of its weights exactly zero. Not small. Zero.

A weight of exactly zero is not a small contribution. It is no contribution. The feature it multiplies has been switched off entirely, dropped from the

model as if it were never there. So an L1 penalty does not just shrink the model, it *selects*: it decides which features are worth keeping and sets the rest to nothing. A model with mostly-zero weights is called **sparse**, and sparsity is prized, because a model that uses ten features out of a thousand is one you can inspect, trust, and run cheaply. L1 gives sparsity almost for free. L2 does not. To see why, we go back to the diamond.

Picture the penalty as a budget. The model wants to reach the weights that fit the data best, but the penalty holds it on a leash, keeping it within a fixed distance of the origin. Under L2 that leash traces a circle; under L1, the diamond. The best-fitting weights sit somewhere out beyond the leash, and the model settles at the point on the leash's boundary that gets it as close as possible to that ideal.

Here is the difference, and it is entirely about the shapes. A circle is smooth everywhere, so the closest point on it to some outside target is almost always a generic point, a little of every coordinate, nothing exactly zero. But the diamond is not smooth. It has sharp corners, and those corners stick out further than the flat sides, and, crucially, the corners sit exactly on the axes. A point on an axis is a point where the other coordinates are zero. Because the corners reach out furthest and land on the axes, they are the points the leash most often settles on, and settling on a corner means setting coordinates to zero. The sharp corners of the diamond are sparsity. The smooth circle has no corners to catch on, so it never zeroes anything out.

That is the whole reason the diamond had to be a diamond, and why we were told to hold onto it two sections ago. The geometry we noticed as a curiosity, corners on the axes, is a working feature-selection mechanism. It is the same fact seen twice: once as a shape, once as a model deciding what to ignore.

This shows up all over practical machine learning. It is the heart of a method called the lasso, the standard tool when you have far more features than you can use and need the model itself to tell you which ones matter. It is why L1 penalties appear whenever someone wants a model they can

4 Length, Distance, and Norms

actually read. And the choice between L1 and L2 is exactly the choice from the start of this chapter, crow or pedestrian, made where it has real consequences: the same two ways of measuring distance, now deciding whether a model keeps all its features quietly or throws most of them away.

The failure to watch for is reaching for sparsity when you did not want it, or missing it when you did. Adding an L1 penalty to a model whose features all genuinely matter will zero out useful ones and quietly cost you accuracy. Reaching for L2 when you needed a short, inspectable model will leave you with a thousand tiny non-zero weights and nothing switched off. Neither norm is better. They measure different things, and the diamond and the circle are the picture of the difference.

Common misunderstandings

The two norms are simple, but the ideas around them invite a few wrong turns.

The L1 norm is not an approximation of the L2 norm. Because the two often come out close, and both measure something called distance, it is tempting to treat L1 as a rougher, cheaper stand-in for the “real” L2 distance. It is not. Neither is more real. They answer different questions, crow or pedestrian, and for the same vector they generally give different numbers on purpose. When they disagree, that is not error to be corrected; it is the two norms measuring the two different things they were built to measure.

A bare norm means L2, but only by convention. We settled that $\|\mathbf{v}\|$ with no subscript means the L2 norm, and that convention holds throughout this book. But it is a convention, not a law, and other authors sometimes default differently or leave it genuinely ambiguous. When you read a norm in someone else’s work with no subscript, the safe move is to

Check your intuition

check what they mean rather than assume L2. In your own writing, pick a default and hold to it, which is exactly what the subscript is for.

Sparsity is not the same as smallness. This is the one that costs people the most. An L2 penalty makes weights small; an L1 penalty makes many of them exactly zero. It is easy to blur these into “both keep the weights under control,” but the gap between small and zero is the whole point. A weight of 0.0001 is still a feature the model is using, still a coordinate that has to be stored, computed, and reasoned about. A weight of 0 is a feature switched off. Small is quieter. Zero is gone. If you expected an L2 penalty to hand you a short list of features and it gave you a thousand tiny non-zero ones instead, this is why.

A zero norm does not mean a zero vector for every notion of “size.” For both L1 and L2, a norm of zero does mean the vector is the zero vector, all entries zero, and this is a genuine, useful property. The trap is assuming everything anyone calls a “norm” behaves this way. There is a widely used quantity, confusingly nicknamed the “L0 norm,” that simply counts how many entries are non-zero. It is what people often actually mean by sparsity, it is not truly a norm in the strict sense, and it does not follow the rules L1 and L2 do. You do not need it yet. It is flagged only so that the word “norm,” met in the wild, is not assumed to always mean what it means here.

Check your intuition

Try each before opening the answers. As always, these ask you to use the two norms, not just recall their formulas.

1. For $\mathbf{v} = [3, -4]$, compute both $\|\mathbf{v}\|_1$ and $\|\mathbf{v}\|_2$. Which is larger, and will that ordering always hold?

4 Length, Distance, and Norms

2. A vector points straight along one axis, say $\mathbf{v} = [5, 0]$. What are its L1 and L2 norms? Why are they equal here, when they were different for $[3, 4]$?
3. Two weight vectors come out of training. One is $[0.5, 0.5, 0.5, 0.5]$, the other is $[1, 0, 0, 0]$. Compare them under each norm. Which norm sees them as the same size, and which tells them apart?
4. You add a penalty to a model and it returns weights with most entries at exactly zero. Which norm did you almost certainly use, and what has it done for you beyond shrinking the weights?
5. Under the L1 norm, what does the set of points at distance one from the origin look like, and which points on it are furthest out along the axes? Tie your answer back to sparsity.

Answers

1. The L1 norm is $|3| + |-4| = 7$. The L2 norm is $\sqrt{3^2 + (-4)^2} = \sqrt{25} = 5$. L1 is larger, and that ordering always holds: for any vector, $\|\mathbf{v}\|_1 \geq \|\mathbf{v}\|_2$. The pedestrian, forced to keep to the streets, can never travel less than the crow who cuts across. The two are equal only in the special case where there is no corner to cut, which the next question is about.
2. Both norms give five. L1 is $|5| + |0| = 5$, and L2 is $\sqrt{5^2 + 0^2} = 5$. They agree because the vector lies flat along a single axis, so there is no diagonal to cut. The pedestrian and the crow walk the very same path when the destination is straight down one street. The norms differ only when a vector spreads across more than one axis at once, giving the crow a corner to cut and the pedestrian a longer way round. That is exactly the case $[3, 4]$: two axes in play, so five versus seven.
3. Under L1, both are size one: $0.5 \times 4 = 2$... wait, check it. $\|[0.5, 0.5, 0.5, 0.5]\|_1 = 2$, and $\|[1, 0, 0, 0]\|_1 = 1$. So L1 tells them apart, two against one. Under L2, $\|[0.5, 0.5, 0.5, 0.5]\|_2 = \sqrt{4 \times 0.25} =$

$\sqrt{1} = 1$, and $\|[1, 0, 0, 0]\|_2 = 1$. L2 sees them as the same size. This is the mechanism of the last section in miniature: L2 is indifferent between spreading the weight across all four coordinates and piling it onto one, while L1 actively prefers the concentrated, sparse vector, giving it the smaller norm. Under an L1 budget, $[1, 0, 0, 0]$ is the cheaper place to be.

4. You almost certainly used an L1 penalty. Beyond making the weights small, it has performed feature selection: every weight it drove to exactly zero is a feature switched off, dropped from the model entirely. You have not just a smaller model but a shorter one, using only the features L1 judged worth keeping. An L2 penalty would have left every feature in play with a small non-zero weight.

5. It is a diamond, a square standing on its corner, with its four corners sitting out on the axes and its flat sides cutting across the diagonals. The corners are the points furthest out along the axes, and a point on an axis has zeros in its other coordinates. That is the whole link to sparsity: the corners of the L1 unit shape stick out furthest and land exactly where coordinates are zero, so a model held on an L1 budget tends to settle on them, switching those coordinates off. The diamond's corners and the model's zeros are the same fact.

5 Matrices as Transformations

The itch

Everything we have done so far happens to one vector at a time. We measure a vector, we add two of them, we compare their directions. The vector sits still and we inspect it. But almost nothing interesting in machine learning leaves the vectors sitting still. The whole business is *moving* them: taking every data point and shifting it somewhere more useful, over and over, until points that were tangled together come apart.

Think about what a model actually does to its input. A face, as we said, arrives as a long list of pixel numbers, a vector in a space with far too many dimensions to picture. That raw vector is useless as it stands; the numbers are just brightnesses. What a network does, layer after layer, is *transform* that vector, reshaping the space it lives in so that by the end faces of the same person land near each other and different people land far apart. Nothing was added to the photo. The space was bent and stretched until the answer became easy to read off.

So we need to talk about transformations: not single vectors, but operations that take a whole space of vectors and move all of them at once. And we need the operation to be something a machine can store and apply, which means, as always, it has to come down to numbers.

Here is the surprise, and it is the heart of this part of the book. An enormous and useful class of these space-moving operations can each be captured completely by a small grid of numbers. Not approximately, completely: the entire behaviour of the transformation, what it does to

every one of the infinitely many vectors in the space, is pinned down by a handful of numbers arranged in a box. That box is called a matrix, and this chapter is about the one idea that makes all of linear algebra click: a matrix is not a grid of numbers that happens to sit there. It is a thing that *moves space*, and the grid is only how we write the motion down.

The picture

“Moving space” sounds like it should take an enormous amount of information. Space is full of vectors, infinitely many, and a transformation sends every one of them somewhere. To pin the operation down it seems we would have to list where each vector goes, and there is no end to them. But we do not have to, and the reason why is the whole trick.

Cast your mind back to the two reference vectors, $[1, 0]$ and $[0, 1]$, the ones pointing one step along each axis. We saw two chapters ago that every vector is built from them: the vector $[3, 2]$ is just three of the first plus two of the second, three steps east and two steps north. Every vector in the plane is some amount of $[1, 0]$ and some amount of $[0, 1]$. They are the two building blocks the whole space is assembled from, and they have a name for that reason, the **basis** vectors.

Now here is the move. Suppose our transformation is of the well-behaved kind we care about, the kind that keeps grid lines straight and evenly spaced and keeps the origin fixed. For a transformation like that, if we know only where the two basis vectors land, we know where *everything* lands. Watch why. The vector $[3, 2]$ was three of the first basis vector plus two of the second. After the transformation, it is still three of *wherever the first basis vector went* plus two of *wherever the second went*. The recipe, three of one and two of the other, survives the move untouched. Only the ingredients changed.

That is the entire idea, and it is worth saying slowly because everything rests on it. To describe a transformation of the whole plane, we do not

The picture

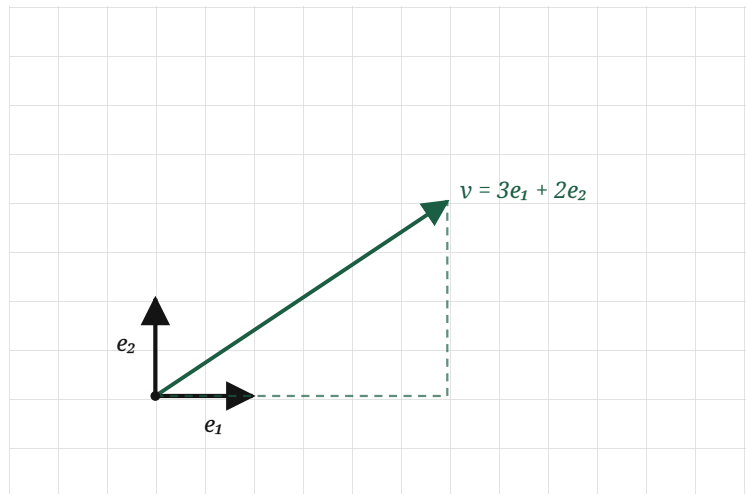


Figure 5.1: A transformation is fixed by where the two basis vectors land. Every other vector is carried along by the same recipe that built it from them.

5 Matrices as Transformations

list where infinitely many vectors go. We list where two vectors go. Two vectors, the images of $[1, 0]$ and $[0, 1]$, and the transformation is completely determined, because every other vector just rides along on the linear combination that already defined it.

So watch what a transformation does by watching what it does to the grid. Take the transformation that sends $[1, 0]$ to $[1, 0]$, leaving it alone, and sends $[0, 1]$ to $[1, 1]$, tilting the up-direction over to the right. The basis vectors have moved, and the whole grid moves with them: every square shears into a slanted parallelogram, the way a stack of paper slides when you push the top. No vector was moved by hand. They all followed the two that changed.

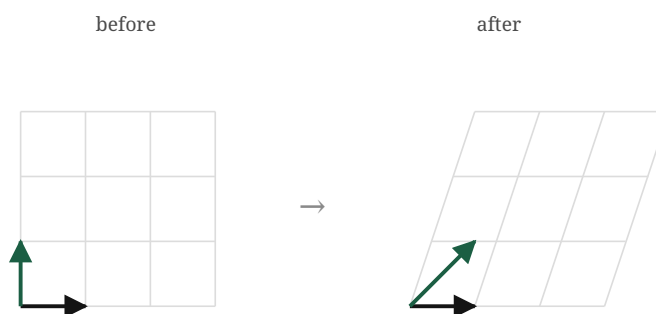


Figure 5.2: A shear. The first basis vector stays put, the second tilts right, and every vector in the plane follows the same recipe, sliding the whole grid into slanted parallelograms.

Notice the restriction we slipped in: grid lines stay straight and evenly spaced, and the origin stays put. That is not a small print. It is the exact meaning of the word we have been circling since the second chapter. A transformation that keeps the grid straight and the origin fixed is precisely a **linear** one, and it is the reason lines stayed lines back when we first met the

word. These well-behaved, grid-preserving, origin-fixing transformations are the only ones a matrix can describe, and they are the ones this whole part of the book is about.

The math, built up

We said a transformation is pinned down by where the two basis vectors land. So to write a transformation as numbers, we just write down those two landing spots. That is all a matrix is.

Take the shear from the last section. It sent $[1, 0]$ to $[1, 0]$ and sent $[0, 1]$ to $[1, 1]$. Stack those two landing spots as columns, the image of the first basis vector on the left, the image of the second on the right:

$$A = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}$$

Read this the right way and it holds no mystery. The first column, $[1, 0]$ read downward, is where $[1, 0]$ went. The second column, $[1, 1]$, is where $[0, 1]$ went. The matrix is not a grid of anonymous numbers. It is a record of what happened to the basis, one landing spot per column. Anyone who hands you a matrix is really handing you the two arrows the basis turned into, written side by side.

Now applying the transformation to some vector, which we write as $A\mathbf{v}$, has to do the thing we already know: carry $\mathbf{v}$ along by its own recipe. If $\mathbf{v} = [x, y]$, then $\mathbf{v}$ was x of the first basis vector plus y of the second. After the transformation it is x of the first landing spot plus y of the second. So the result is a linear combination of the columns, with the entries of $\mathbf{v}$ as the amounts:

$$A\mathbf{v} = x \begin{bmatrix} 1 \\ 0 \end{bmatrix} + y \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

5 Matrices as Transformations

That is the whole operation, and it is worth pausing on how little it is. Multiplying a matrix by a vector is scaling each column of the matrix by the matching entry of the vector, and adding. It is the linear combination from the second chapter, wearing new notation. The matrix supplies the ingredients, its columns, and the vector supplies the amounts.

Let us carry it through with $\mathbf{v} = [3, 2]$:

$$A\mathbf{v} = 3 \begin{bmatrix} 1 \\ 0 \end{bmatrix} + 2 \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 3 \\ 0 \end{bmatrix} + \begin{bmatrix} 2 \\ 2 \end{bmatrix} = \begin{bmatrix} 5 \\ 2 \end{bmatrix}$$

The vector $[3, 2]$ landed at $[5, 2]$. The shear pushed it to the right by an amount equal to its height, two, which is exactly what a shear should do, and we read that straight off the columns without memorising a single rule.

There is a mechanical way to write the same calculation, the one usually taught first: the top entry of the result is $1 \cdot 3 + 1 \cdot 2 = 5$, the bottom is $0 \cdot 3 + 1 \cdot 2 = 2$. Each entry of the result is a dot product of a row of the matrix with the vector. That row-by-row recipe is correct, it gives the same $[5, 2]$, and it is often the fastest way to compute by hand. But learn it as the *meaning* and the matrix goes dead: it becomes a grid of numbers you shuffle, and the space stops moving. The columns-as-landing-spots view is the one that keeps the geometry alive. The row view is for computing; the column view is for understanding.

None of this is confined to two dimensions. A transformation of three-dimensional space is fixed by where three basis vectors land, so its matrix has three columns. A transformation carrying a three-hundred-dimensional space somewhere is three hundred landing spots, a matrix three hundred columns wide, and $A\mathbf{v}$ is still the same thing: scale each column by the matching entry of $\mathbf{v}$ and add. The picture ended at two or three columns. The rule did not.

Build it yourself

A matrix in NumPy is an array of rows, and applying it to a vector is a single operator. But the point of this chapter is the meaning, so we will compute the transformation both ways, by the column recipe and by the built-in, and watch them agree.

First, build the shear matrix. NumPy takes it row by row, so we type the two rows even though we have been thinking in columns:

```
import numpy as np

A = np.array([[1, 1],
              [0, 1]])
print(A)
```

```
[[1 1]
 [0 1]]
```

Keep the columns in mind as you read it. The first column, top-to-bottom, is $[1, 0]$, where the first basis vector landed. The second column is $[1, 1]$, where the second landed. NumPy stores it as rows on the page, but the transformation lives in the columns.

Now apply it to $[3, 2]$ the meaningful way, as a linear combination of the columns, scaling each column by the matching entry of the vector and adding:

```
v = np.array([3, 2])

col1 = A[:, 0]      # first column: where [1,0] went
col2 = A[:, 1]      # second column: where [0,1] went
```

5 Matrices as Transformations

```
result = v[0] * col1 + v[1] * col2
print(result)
```

[5 2]

The answer is [5, 2], the landing spot we worked out by hand. `A[:, 0]` pulls out the first column and `A[:, 1]` the second, and the line `v[0] * col1 + v[1] * col2` is the recipe read aloud: three of the first column plus two of the second. Nothing here is matrix magic. It is the linear combination from the second chapter, run on the columns of `A`.

NumPy has the operation built in, written with the `@` symbol:

```
print(A @ v)
```

[5 2]

Same [5, 2]. The `@` is matrix application, and it is doing exactly the column recipe we just wrote out by hand, however it organises the arithmetic internally. We can put the two side by side to be sure:

```
print(np.array_equal(v[0] * A[:, 0] + v[1] * A[:, 1], A @ v))
```

True

True. The meaning and the machine agree: applying a matrix is combining its columns.

To feel the transformation as motion rather than a single answer, apply the same matrix to several vectors and watch where they go:

```
points = np.array([[1, 0],
                   [0, 1],
                   [1, 1],
                   [2, 1]])

for p in points:
    print(p, "->", A @ p)
```

```
[1 0] -> [1 0]
[0 1] -> [1 1]
[1 1] -> [2 1]
[2 1] -> [3 1]
```

Read the arrows. The first basis vector $[1, 0]$ stays put, as a shear should leave it. The second, $[0, 1]$, moves to $[1, 1]$, tilted right. The corner $[1, 1]$ slides to $[2, 1]$, and $[2, 1]$ to $[3, 1]$. Every point shifts right by an amount equal to its height, which is the shear, seen now not as one landing spot but as a whole set of points sliding in step.

And, as ever, none of this needed two dimensions. Give A three hundred columns and v three hundred entries, and $A @ v$ still combines the columns by the entries of v , moving a vector through a space we could never picture, on the same one line.

Where it lives in ML

Two chapters ago we said a neural network layer takes many linear combinations of its inputs, each with its own weights. We can now say what that pile of linear combinations actually is. It is a matrix.

A layer receives a vector, the outputs of the neurons before it. Each neuron in the layer forms one linear combination of that vector, using its own row

5 *Matrices as Transformations*

of weights. Stack those rows up and you have a grid of numbers, one row per neuron, which is exactly a matrix. The whole layer, every neuron at once, is a single matrix applied to the incoming vector: $A\mathbf{v}$. The thing we built by hand in this chapter, scaling the columns and adding, is what runs when a network processes its input, billions of times over, at every layer of every model. When people say a model “has weights,” the weights are the numbers in these matrices. Training a model is searching for the matrices that move the data the right way.

This reframes the whole picture of what a network is doing. Each layer is a transformation, a matrix that moves the space of vectors. The input starts as a point in some raw space, pixel brightnesses or word counts, where the thing we want is tangled up and hard to read. Layer by layer, matrix by matrix, the network carries that point through a sequence of transformations, each one bending and stretching the space a little more, until the point lands somewhere the answer is obvious. A network is a chain of the space-moving operations from this chapter, applied one after another.

Beyond neural networks, matrix transformations are everywhere the same way. Rotating an image, scaling it, flipping it, all of these are matrices applied to the vectors of pixel positions. Rotating a point cloud of data to line it up with its most important directions, which is the heart of a technique we will build later called PCA, is a matrix. Whenever data is reshaped, reoriented, or projected down to fewer dimensions, a matrix is doing it, and it is always the same operation underneath: combine the columns by the entries of the vector.

Now the limit, and it is the one we promised back when we first met linear combinations. A single matrix can only perform a linear transformation. It can stretch space, rotate it, shear it, flip it, but it cannot bend it. Straight grid lines stay straight, no matter what numbers we put in the matrix. And we saw two chapters ago that stacking linear steps changes nothing: apply one matrix, then another, and the combined effect is still a single linear transformation, still unable to bend. This is the same wall from

before, now stated in matrices. A network that was nothing but matrices, however many layers deep, could only ever move space in straight-line ways, and most of the interesting structure in real data cannot be untangled that way.

Which is why, between one matrix and the next, a real network does one small non-linear step, the bending we have twice now set aside for later. The matrices do the moving; the bending step, applied between them, is what lets the moves accumulate into something a straight transformation never could. The matrix is the engine of the network, and it is exactly the operation from this chapter. The bend is what we build on top of it, and it is nearly the last thing this book assembles.

Common misunderstandings

Matrices attract more confusion than anything so far, mostly because the notation invites you to forget they are transformations.

In NumPy, `*` is not matrix application. This is the direct cousin of the trap from the dot product chapter, and it bites just as often. The matrix operator is `@`. The star, `A * v`, does something else entirely: it multiplies elementwise, matching up entries and broadcasting, and it will happily return a wrong answer without any error. Watch the two disagree:

```
import numpy as np

A = np.array([[1, 1],
              [0, 1]])
v = np.array([3, 2])

print(A @ v)      # matrix application: [5, 2]
print(A * v)      # elementwise, NOT what we want
```

5 Matrices as Transformations

```
[5 2]  
[[3 2]  
 [0 2]]
```

The first is the transformation. The second is a grid of elementwise products that means nothing here. Reach for `@` when you want to apply a matrix, and treat a bare `*` between a matrix and a vector as a bug until proven otherwise.

A matrix is not its grid of numbers; it is what they do. It is tempting, once the numbers are on the page, to think of a matrix as the numbers. But two matrices with entries that look nothing alike can be nearly the same transformation, and two that look similar can do wildly different things. The numbers are a description of a motion, written in the language of where the basis lands. When you read a matrix, do not read the entries, read the columns, and ask where the basis went. That is the transformation; the grid is just its handwriting.

Order matters: A then B is not B then A. When we apply two transformations in turn, the order almost always changes the result. Rotate a vector and then stretch it, and you get one thing. Stretch it first and then rotate, and you generally get another. This is not a quirk of the arithmetic. It is a fact about actions in the world: putting on socks then shoes is not the same as shoes then socks. Because matrices are actions, they inherit this, and it means the order in which we multiply them is not free to shuffle. This is the single biggest difference between matrix arithmetic and the ordinary number arithmetic we grew up with, where order never mattered, and it is the thing the next chapter has to take seriously.

A transformation fixes the origin, so a matrix cannot move it. Every linear transformation leaves the origin where it is, because the origin is zero of every basis vector, and zero of anything is still zero. This has a consequence people trip on: a matrix alone cannot shift space sideways, cannot slide everything two steps to the right. That kind of move, a shift

Check your intuition

that does not fix the origin, is not linear and no matrix can express it on its own. It is a real and common operation, and when we need it we will have to add something to the matrix, not just choose different entries. For now the thing to hold is that a bare matrix pins the origin, always.

Check your intuition

Try each before opening the answers. These ask you to read matrices as transformations, not to grind the arithmetic.

1. A matrix has columns $[1, 0]$ and $[0, 1]$, so $A = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$. Where does it send the vector $[7, -3]$? What is this transformation doing to space?
2. A matrix sends the first basis vector to $[2, 0]$ and the second to $[0, 2]$. Write the matrix, and describe the transformation in one phrase.
3. A matrix sends $[1, 0]$ to $[0, 1]$ and sends $[0, 1]$ to $[-1, 0]$. Without computing anything further, what motion is this? Try applying it to $[1, 0]$ and picturing where it points afterwards.
4. Apply $A = \begin{bmatrix} 2 & 0 \\ 0 & 3 \end{bmatrix}$ to $[1, 1]$ using the column recipe: scale each column by the matching entry and add. What lands where, and why is this transformation not a rotation?
5. Someone claims a single matrix can take every point in the plane and slide it three units to the right, moving $[0, 0]$ to $[3, 0]$. Are they right?

💡 Answers

1. It sends $[7, -3]$ to $[7, -3]$, unchanged. The columns are the untouched basis vectors, so the transformation leaves every vector exactly where it is. This is the identity, the transformation that does

nothing, and its matrix is the one with ones down the diagonal and zeros elsewhere. Applying it is the matrix equivalent of multiplying a number by one.

2. The matrix is $\begin{bmatrix} 2 & 0 \\ 0 & 2 \end{bmatrix}$, the two landing spots as columns. It doubles the length of every vector while keeping its direction, so it is a uniform scaling, stretching the whole plane out from the origin by a factor of two. Every point moves twice as far from the centre, and the grid squares double in size without tilting.

3. It is a rotation by a quarter turn, ninety degrees counterclockwise. The first basis vector, pointing east, is sent to $[0, 1]$, pointing north. The second, pointing north, is sent to $[-1, 0]$, pointing west. Each basis vector has been swung a quarter turn counterclockwise, and since everything follows the basis, the whole plane turns with them. Reading the columns told us the motion without a single multiplication.

4. The columns are $[2, 0]$ and $[0, 3]$. Applying to $[1, 1]$ gives $1 \cdot [2, 0] + 1 \cdot [0, 3] = [2, 3]$. The transformation stretches the first axis by two and the second by three. It is not a rotation because it does not turn vectors rigidly; it stretches different directions by different amounts, distorting shapes rather than spinning them. A rotation keeps every length the same, and this changes them, so the two are different kinds of motion entirely.

5. No. Sliding every point three units right would move the origin from $[0, 0]$ to $[3, 0]$, but every linear transformation fixes the origin, because the origin is zero of the basis and no combination of the columns can escape zero when both amounts are zero. A bare matrix cannot perform this slide. It is a genuine and useful operation, but it needs more than a matrix, and building that “more” is a job we have deferred to later.

6 Matrix Multiplication as Composition

The itch

A single matrix moves space once. But we ended the last chapter noticing that a network does not move space once. It moves it again and again, matrix after matrix, each transformation handed the output of the one before. So the natural question is what happens when we chain two transformations together. Apply one matrix, then apply a second to the result. Two moves, back to back.

We could always do this the slow way: take a vector, apply the first matrix, take the answer, apply the second matrix, and read off where it ends up. That works, but it treats the two-step process as something we have to walk through in full every time. It would be far better if the *pair* of transformations were itself a single transformation, one matrix that does in one move what the two did in two. Then a whole stack of layers could collapse, at least in principle, into one operation we understand all at once.

It turns out the combination of two matrices is always itself a matrix, and finding it is what matrix multiplication actually is. Not a strange rule to memorise, but the answer to a concrete question: if I do this transformation and then that one, what single transformation have I done? That single combined transformation is the product of the two matrices, and this short chapter is about why it works the way it does.

The picture

We already have everything we need to find the combined transformation, because we know the one fact that pins any transformation down: where the basis vectors land. A combined transformation is still a transformation, so it too is fixed entirely by where it sends the two basis vectors. We do not need to think about the infinitely many vectors in the plane. We need to follow two.

So take the basis vector $[1, 0]$ and push it through both moves in turn. First apply the earlier transformation and see where it goes. Then take that result and apply the later transformation to it. Wherever it ends up, after both moves, is where the *combined* transformation sends $[1, 0]$. Do the same for $[0, 1]$: push it through the first move, then the second, and note where it lands. Now we have both landing spots for the combined transformation, and by the rule from the last chapter, that is the combined transformation, completely. Written as columns, those two landing spots are the product matrix.

That is genuinely all matrix multiplication is. To multiply two matrices is to ask where the basis ends up after both transformations have acted, first one then the other, and to record those final landing spots as the columns of a new matrix. Everything mechanical about the operation, every product and sum in the rule people memorise, is just the arithmetic of tracing the basis through twice.

The order is baked in and cannot be swapped, exactly as the last chapter warned. “First this transformation, then that one” is a different journey for the basis than “first that one, then this one,” and the two journeys generally end in different places. So the two orders give different combined transformations, and different product matrices. When we write the product of two matrices, the order we write them in records which transformation happens first, and getting it backwards computes the wrong journey.

The picture

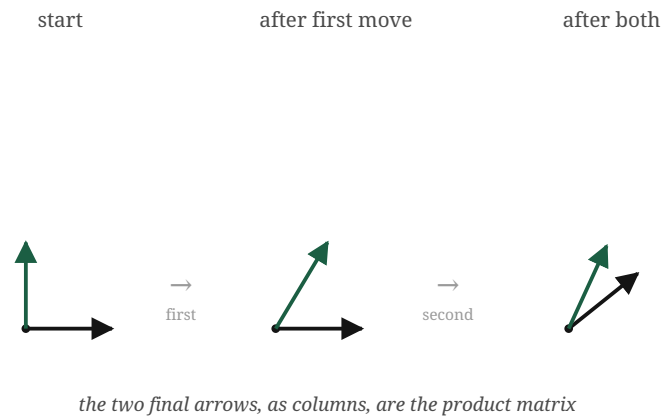


Figure 6.1: To compose two transformations, send each basis vector through the first, then the second. The two final landing spots, as columns, are the combined transformation.

The math, built up

Let us turn “trace the basis through both transformations” into arithmetic, and watch the famous rule assemble itself.

Call the first transformation B and the second A , so we do B first, then A . We want the combined matrix, and we know it column by column: its first column is where $[1, 0]$ ends up after both moves, its second column is where $[0, 1]$ ends up.

Where does $[1, 0]$ go? First B acts on it. Applying a matrix to $[1, 0]$ just picks out that matrix’s first column, because $[1, 0]$ is one of the first column and none of the second. So after B , our basis vector sits at B ’s first column. Then A acts on *that* vector. And applying A to a vector is the operation we built last chapter: combine A ’s columns using the vector’s entries. So the first column of the combined matrix is A applied to B ’s first column. The second column is the same story with $[0, 1]$: it becomes B ’s second column, then A acts on that.

So the combined matrix is A applied to each column of B , one column at a time. That is the entire rule, stated meaningfully:

$$AB = [A(\text{first column of } B) \quad A(\text{second column of } B)]$$

Everything usually memorised is the arithmetic of those applications spelled out. Let us make it concrete. Take

$$A = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}, \quad B = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}$$

where A is a quarter-turn and B is the shear from before. We do B then A . Take B ’s first column, $[1, 0]$, and apply A by combining A ’s columns: $1 \cdot [0, 1] + 0 \cdot [-1, 0] = [0, 1]$. Take B ’s second column, $[1, 1]$, and apply A :

Build it yourself

$1 \cdot [0, 1] + 1 \cdot [-1, 0] = [-1, 1]$. Those two results are the columns of the product:

$$AB = \begin{bmatrix} 0 & -1 \\ 1 & 1 \end{bmatrix}$$

Now compare this to the rule as it is usually stated: each entry of the product is a row of A dotted with a column of B . Check the top-left entry, first row of A dotted with first column of B : $[0, -1] \cdot [1, 0] = 0$. It matches. That row-times-column recipe is not a separate fact to learn. It is exactly what “apply A to each column of B ” works out to, entry by entry, because applying a matrix produces each output entry as a dot product of a row with the input, which we saw last chapter. The dreaded rule is just our column-tracing, written out one number at a time.

There is one honest bookkeeping point the tracing makes obvious that the memorised rule hides. For the combination to make sense at all, A has to be able to act on the columns of B . A column of B lives in whatever space B outputs into, and A has to take vectors from exactly that space. If they do not match, there is no combined journey to speak of, and the product simply does not exist. This is the real reason two matrices can only be multiplied when the first one’s width matches the second one’s height. It is not an arbitrary shape rule. It is the requirement that the output of one transformation is a valid input to the next.

Build it yourself

The product is $\mathcal{Q}$, the same operator we used to apply a matrix to a vector, now between two matrices. But the point of this chapter is that the product is column-tracing, so we build it that way first and check the operator agrees.

Here are the quarter-turn and the shear from the last section:

6 Matrix Multiplication as Composition

```
import numpy as np

A = np.array([[0, -1],
              [1,  0]])    # quarter turn

B = np.array([[1, 1],
              [0, 1]])    # shear
```

We do B first, then A , so we want A applied to each column of B . Pull out B 's columns and send each one through A :

```
col1 = A @ B[:, 0]    # A applied to B's first column
col2 = A @ B[:, 1]    # A applied to B's second column

combined = np.column_stack([col1, col2])
print(combined)
```

```
[[ 0 -1]
 [ 1  1]]
```

The two landing spots, stacked as columns, give the combined matrix. Now the built-in product:

```
print(A @ B)
```

```
[[ 0 -1]
 [ 1  1]]
```

The same matrix. $A @ B$ is doing exactly what we did by hand: applying A to each column of B and collecting the results as columns. We can put them side by side to be sure nothing differs:

```
print(np.array_equal(combined, A @ B))
```

True

True. The operator and the column-tracing are one operation.

Now see the order dependence directly. We do the two transformations in the other order, A first then B , which is the product $B@A$, and compare:

```
print(A @ B)
print(B @ A)
```

```
[[ 0 -1]
 [ 1  1]]
[[ 1 -1]
 [ 1  0]]
```

The two are different matrices. Doing the shear then the quarter-turn is a different combined transformation from the quarter-turn then the shear, and the arithmetic shows it plainly: swap the order and you get a different grid of numbers, because you asked for a different journey. This is the socks-and-shoes fact from the last chapter, now something we can watch NumPy confirm in two lines.

And, as everywhere else, none of this cares about size. Multiply a matrix that is three hundred wide by one that is three hundred tall and `@` still traces every basis vector through both transformations, composing two moves through a space we cannot picture into a single matrix, on the same one line.

Where it lives in ML

This chapter is the reason a network can be deep. A deep network is, structurally, a long chain of transformations: the input vector enters, the first matrix moves it, the result is handed to the second matrix, which moves it again, and on down through the layers. “Deep” means many transformations composed one after another, and composition is the operation from this chapter. Every time a model passes data from one layer to the next, it is doing exactly the chaining we just built.

Seeing a network as composed transformations explains what the depth is *for*. One matrix can only reshape space in one linear way. But a sequence of transformations, each acting on the last one’s output, can carry a point through a long, elaborate journey, reshaping the space in stages. The early layers make coarse moves, the later ones refine, and the composition of all of them is a single overall transformation from raw input to useful output. Depth is just a long composition, and this chapter is how the pieces join.

There is a practical payoff hiding in the fact that composition produces a single matrix. When a fixed sequence of linear transformations is applied to millions of data points, we do not have to push every point through every step. We can compose the whole sequence once, into one matrix, and then apply that single matrix to every point. The many-step journey and the one-step journey give identical results, because that is what composition means, but the one-step version does far less work. Collapsing a chain of transformations into one is a standard efficiency move, and it is only possible because the product of matrices is itself a matrix.

But this same collapsibility is the wall we have circled since the second chapter, and now we can state it exactly. If every layer were only a matrix, the entire network, however deep, would compose down to one matrix. All that depth would collapse into a single linear transformation, and the network could do no more than one layer could. Composition, the very thing that makes depth possible, is also what would make depth *pointless*

if layers were purely linear. A thousand composed linear moves are still just one linear move.

This is the precise reason a real network places a small non-linear step between its matrices. That step breaks the collapsibility. With a bend between each pair of matrices, the layers can no longer be composed away into one, and the depth becomes real: each layer adds something the others cannot undo. The matrices supply the movement, composition chains them, and the non-linear steps are what stop the chain from collapsing. We have now met this bend three times as a promise. When we build it in Part 4, this is the collapse it exists to prevent.

Common misunderstandings

Composition is a clean idea, but its notation is the most misleading we have met, and a couple of traps come with it.

In AB , the right-hand matrix acts first. This catches almost everyone. We read left to right, so AB looks like it should mean “ A , then B .” It means the opposite. The product AB applied to a vector is $A(B\mathbf{v})$: the vector meets B first, and A acts on the result. The reason is that the vector sits on the right, $AB\mathbf{v}$, so the matrix nearest it, B , reaches it first. When you read a chain of matrices, read the order of *action* from right to left, even though the symbols run left to right. Getting this backwards computes a different journey and a different answer, with nothing to warn you.

You cannot multiply any two matrices. Ordinary numbers multiply in any combination, but matrices carry shapes, and the shapes have to meet. The product AB exists only when the output space of B is the input space A expects, which shows up as a dimension that has to match between them. When it does not match, there is no combined journey and no product; the operation is simply undefined. This is not a rule to

6 Matrix Multiplication as Composition

memorise so much as a consequence of what composition is: the second transformation has to be able to receive what the first one produced.

A product of matrices is a single matrix, not a stored to-do list.

Once we compute AB , the result is one matrix, one transformation, with no memory that it came from two. The two-step origin has been fully absorbed into one grid of numbers. This is exactly why the collapse from the last section works and why it saves computation. But it cuts both ways: given only the product, you cannot in general recover which two matrices were multiplied to make it, any more than knowing a sum is twelve tells you it was seven plus five. The composition flows one way, into the product, and does not come back out.

Composition is not commutative, but it is associative. We have leaned hard on order mattering, $AB \neq BA$, and that stays true. But a different rearrangement is always safe: when three or more transformations are chained, how you *group* them does not change the result. Doing A after the combination of B and C is the same as doing the combination of A and B before C , as long as the left-to-right order is untouched. Order is fixed; grouping is free. This is why we can write a long chain of matrices with no brackets at all and compose them in whatever grouping is most convenient, which the efficiency trick from the last section quietly relied on.

Check your intuition

Try each before opening the answers. These ask you to reason about composition, not to grind out products.

1. A is a quarter-turn counterclockwise and B is a stretch that doubles the horizontal axis. In the product AB , which transformation acts on a vector first?

Check your intuition

2. You compose a transformation with the identity matrix, the one that does nothing. What is AI , and what is IA ? Does order matter here?
3. A transformation R rotates the plane by ninety degrees. What is R composed with itself four times, $RRRR$, as a transformation? You should be able to answer without multiplying anything.
4. Someone computes AB and BA for two matrices and gets the same result both times. Have they necessarily made a mistake?
5. You have a fixed chain of three matrices that you must apply to a million data points. Describe two different orders of operations that give the same answers, and say which does less total work.

 Answers

1. B acts first. In the product AB applied to a vector, the vector sits on the right as $AB\mathbf{v}$, so the nearest matrix, B , reaches it first, and A acts on the result. The vector is stretched horizontally, then the stretched vector is turned a quarter-turn. Reading the action right to left is the habit to keep.
2. Both AI and IA equal A . Composing with the do-nothing transformation leaves the other transformation unchanged, whichever side it sits on, because doing nothing before a move or nothing after a move both leave you with just the move. So this is a case where order does not matter, and the identity is exactly the matrix that commutes with everything. It is the rare exception to order-dependence, and it earns that by doing nothing.
3. It is the identity, the transformation that leaves everything where it is. A quarter-turn done four times is a full turn of three hundred and sixty degrees, which brings every vector back to exactly where it started. No arithmetic is needed: four quarter-turns is one full rotation, and a full rotation is indistinguishable from doing nothing. This is a small taste of how thinking in transformations can outrun

6 Matrix Multiplication as Composition

computing in numbers.

4. No. Order usually matters, but not always, and some pairs of transformations genuinely commute. Two rotations about the same centre commute, because turning by one angle then another lands you at the summed angle regardless of order. A stretch and the identity commute. Two stretches along the same axes commute. So $AB = BA$ is unusual but perfectly possible, and finding it does not imply an error; it means these two particular transformations happen not to interfere with each other.

5. Applying the chain point by point, you could push each of the million points through all three matrices in turn, which is three matrix-vector applications per point, three million in total. Or you could first compose the three matrices into a single matrix, one small piece of work done once, and then apply that single matrix to each of the million points, which is one application per point, a million in total. Both give identical answers, because composition preserves the result, but the second does roughly a third of the work. This is the collapse from the fifth section, and it relies on grouping being free even though order is fixed.

7 Independence, Basis, and Rank

The itch

We have been leaning on a word we never defined. Back when matrices arrived, we said every vector is built from the basis vectors $[1, 0]$ and $[0, 1]$, and that a transformation is pinned down by where those two land. We called them the basis and moved on. It was a loan, and this chapter pays it back.

The question underneath is one of redundancy, and it is worth feeling before we make it precise. Suppose someone hands you a set of vectors and says “these are the building blocks; everything you care about is made from combinations of these.” A natural worry: are they all actually needed? Could one of them be thrown away without losing any reach, because it was already buildable from the others? Or worse, could the set be missing something, unable to build a region you needed?

Think of it in the terms from the span chapter. We saw that two vectors pointing different ways span the whole plane, but two vectors pointing the same way span only a line, because the second added no new direction. That second vector was redundant. It was already reachable from the first. So a pile of vectors can contain passengers, members that ride along without contributing any new reach, and it can also be too small to cover the space we care about.

This chapter is about getting that pile exactly right. We want to know when a set of vectors has no passengers, when it is the right size to build a space with nothing wasted and nothing missing, and how to count how

7 Independence, Basis, and Rank

much reach a set actually has. Those three questions have three names, independence, basis, and rank, and they are really one question about redundancy asked three ways.

The picture

Start with the worry made visual. Two vectors that point different ways: between them they reach the whole plane, every point is some combination of the two. Neither is redundant, because neither can be built from the other. Take away either one and you drop from the whole plane down to a single line. Both are pulling their weight.

Now two vectors that point the same way, one just a stretched copy of the other. Together they reach only the line they both lie on. The second brings nothing new, because it was already a scaling of the first, already reachable. It is a passenger. A set with a passenger in it is called **dependent**: at least one member depends on the others, in the sense that it can be built from them and adds no reach. A set with no passengers, where every member contributes something the others cannot, is **independent**.

There is a clean way to see the difference that will matter for the arithmetic later. Take a dependent set, the two vectors on the same line. Because the second is a scaling of the first, we can combine them, with amounts that are not both zero, and arrive back at the origin: walk out along one and back along the other, and you cancel to nothing. For an independent set, this is impossible. The only way to combine two vectors pointing genuinely different ways and land back exactly at the origin is to use nothing of either, both amounts zero. So independence has a sharp test hiding in it: can a non-trivial combination reach zero, or can only the trivial all-zero combination do it?

With independence in hand, a basis is the natural next step: it is an independent set that also reaches everything. Enough vectors to build the whole space, and not one more than needed. In the plane, any two

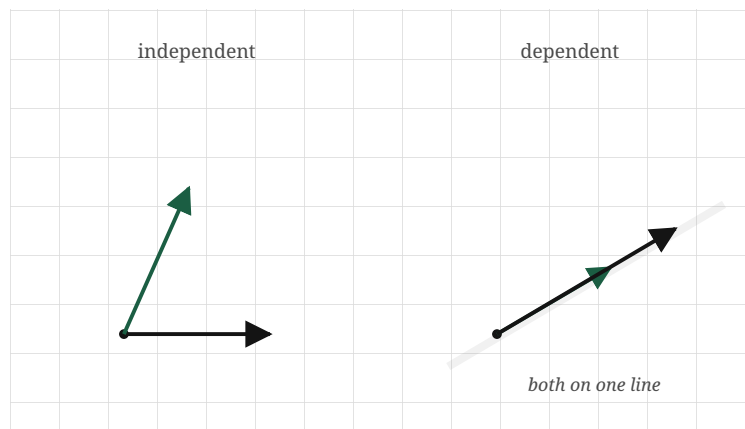


Figure 7.1: Independent vectors reach out into the plane, each adding a direction. Dependent vectors lie along one line, the second a passenger that adds nothing the first did not already cover.

independent vectors form a basis, because two independent vectors already span the whole plane. The familiar $[1, 0]$ and $[0, 1]$ are one basis, the simplest one, but they are not the only one; any two arrows pointing different ways will do. A basis is a complete set of building blocks with no waste: independent, so no passengers, and spanning, so nothing out of reach.

Rank is the counting version of the same idea. Given a set of vectors, or the matrix whose columns they are, the rank is how many independent directions they actually reach, the number of building blocks that are genuinely pulling their weight once the passengers are set aside. Three vectors that all lie in a plane have rank two, not three, because for all their number they only reach a plane; one of them was a passenger. Rank ignores the head-count and measures the true reach.

7 Independence, Basis, and Rank

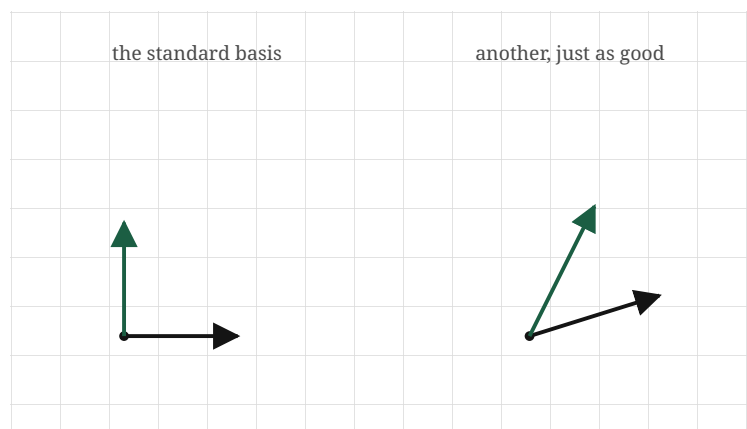


Figure 7.2: A basis: independent and spanning. The standard basis is the simplest, but any two vectors pointing different ways build the plane just as well.

The math, built up

The picture gives us a sharp definition of independence, so let us state it. A set of vectors $\mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_k$ is **independent** when the only linear combination of them that equals the zero vector is the one where every amount is zero:

$$c_1 \mathbf{v}_1 + c_2 \mathbf{v}_2 + \dots + c_k \mathbf{v}_k = \mathbf{0} \quad \text{forces} \quad c_1 = c_2 = \dots = c_k = 0.$$

Read it through the picture. A combination reaching the zero vector means walking out along the vectors, scaled by the amounts, and arriving back at the origin. If the only way to do that is to use none of any of them, no vector was secretly cancelling against the others, so none was a passenger, and the set is independent. If some non-zero amounts also land at zero, then one vector was expressible through the rest, and the set is dependent.

The whole notion of redundancy is captured by this one equation about reaching zero.

A **basis** for a space is then an independent set that spans it. Two conditions, and they pull against each other in a useful way. Spanning wants enough vectors to reach everything; independence wants few enough that none is wasted. A basis is where those two meet: exactly enough. This forces a fact that feels surprising at first and then obvious. Every basis of a given space has the same number of vectors. You cannot build the plane with three independent vectors, there is no room, the third is always dependent on the other two; and you cannot span it with one. The plane's bases all have exactly two vectors. That fixed number is the **dimension** of the space, the word we have used loosely finally given a precise meaning: the dimension is the size of any basis.

Rank is the same count applied to whatever set of vectors you actually have, which need not be a basis. The rank of a set of vectors, or of the matrix holding them as columns, is the size of the largest independent subset, the number of genuinely independent directions among them. A rank equal to the number of vectors means no passengers, the set is independent. A rank below the number of vectors means some are redundant, and the gap is how many. For a matrix, rank measures how many independent directions its columns can reach, which is the true dimensionality of the space the transformation can produce, regardless of how many columns are written down.

Build it yourself

Independence, rank, and dimension all come down to counting genuinely independent directions, and NumPy will count them for us. But first let us catch a dependent set in the act, the way the definition describes.

Here are two vectors on the same line, the second a clean multiple of the first:

7 Independence, Basis, and Rank

```
import numpy as np

v1 = np.array([1.0, 2.0])
v2 = np.array([3.0, 6.0])    # exactly 3 * v1
```

The definition said a set is dependent when some combination, with amounts not all zero, reaches the zero vector. Here we can see the amounts by eye: three of v_1 minus one of v_2 cancels to nothing.

```
print(3 * v1 - 1 * v2)
```

```
[0. 0.]
```

Zero, and the amounts were 3 and -1 , not both zero. That non-trivial path to the origin is dependence, caught in the act. For an independent pair no such non-zero amounts exist; only 0 and 0 would land on zero.

Counting the independent directions in a set is exactly what rank measures, and NumPy computes it directly:

```
dependent = np.column_stack([v1, v2])
print(np.linalg.matrix_rank(dependent))
```

```
1
```

Rank one. Two vectors written down, but only one independent direction between them, because the second was a passenger. Now an independent pair:

```
w1 = np.array([1.0, 2.0])
w2 = np.array([2.0, 1.0])    # not a multiple of w1
independent = np.column_stack([w1, w2])
print(np.linalg.matrix_rank(independent))
```

2

Rank two. Two vectors, two genuine directions, no passengers, so these two form a basis for the plane. The rank matching the number of vectors is the signal that a set is independent.

The same tool sees through a bigger, more deceptive case. Here are three vectors in three-dimensional space, and at a glance they look independent, three different-looking lists:

```
a = np.array([1.0, 0.0, 0.0])
b = np.array([0.0, 1.0, 0.0])
c = np.array([1.0, 1.0, 0.0])      # a + b, a hidden passenger
M = np.column_stack([a, b, c])
print(np.linalg.matrix_rank(M))
```

2

Rank two, not three. The third vector is $a + b$, buildable from the first two, so for all its different appearance it adds no new direction. All three lie in the same flat plane inside the space, and the rank reports the truth the eye missed: two independent directions, not three. This is why rank matters. In high dimensions we cannot see which vectors are passengers, and a set can look full while secretly collapsing into a smaller space. Rank counts what is really there.

Where it lives in ML

Redundancy among features is one of the most common quiet problems in real data, and rank is how it is detected. Suppose a dataset has a column for temperature in Celsius and another for the same temperature

7 Independence, Basis, and Rank

in Fahrenheit. Two columns, but one is a fixed transformation of the other, a passenger in the exact sense of this chapter. They add no independent information. A model handed both is carrying a redundant direction, and the matrix of features is rank-deficient: fewer independent directions than columns.

This is not a curiosity; it breaks things. Several standard methods assume the feature directions are independent, and when they are not, the methods become unstable or fail outright. The clean example is the one we deferred back at the dot product: fitting a linear model involves, under the hood, undoing a transformation, and that undoing is only possible when the relevant vectors are independent. Feed in redundant features and the undoing has no unique answer, because a dependent set does not pin down a single solution. The model cannot decide how to split influence between two columns that say the same thing, and the result is wild, arbitrary weights. Detecting and removing the redundancy, checking the rank, is part of preparing data honestly.

Rank also names something we will meet as a goal rather than a warning. Often a dataset has a thousand features but a much lower true rank: the data looks high-dimensional but really lives in a far smaller number of independent directions, with the rest built from those. This is the hope behind dimensionality reduction. If the true rank is low, we can throw away the redundant directions and keep the few that carry the information, shrinking a thousand numbers to a few dozen with almost no loss. The entire idea rests on the gap between how many features are written down and how many independent directions the data actually spans, which is to say, on rank. When we reach PCA and SVD, this gap is the thing we exploit.

And there is a deeper appearance in the study of neural networks, close to the questions this book was written alongside. The transformations a network learns have ranks, and a low-rank transformation is one that collapses its input into a smaller space of directions, discarding the rest. Whether that collapse is helping the model generalise or quietly destroying

information it needed is an active question, and reading it requires exactly the notion of rank built here: how many independent directions survive a transformation, and how many are lost.

Common misunderstandings

Independence is about the whole set, not about pairs. It is tempting to check independence by looking at vectors two at a time: no vector is a multiple of any other, so the set must be independent. This fails. Three vectors can be pairwise non-multiples and still be dependent, if one is a combination of the other two. The vector $[1, 1]$ is neither a multiple of $[1, 0]$ nor of $[0, 1]$, yet it is their sum, so the three together are dependent. Independence is a property of the entire set at once, whether any combination of all of them reaches zero, not a fact you can verify by pairs.

A basis is not unique. The standard basis $[1, 0]$ and $[0, 1]$ is so familiar that it can seem like *the* basis, the true coordinates of the plane. It is one basis among infinitely many. Any two independent vectors form a basis, and each choice is a different, equally valid set of coordinates for the same space. The standard basis is convenient, not privileged. Much of the power in later chapters comes from deliberately choosing a different basis in which a problem looks simpler, which only makes sense once you accept the standard one holds no special claim.

Rank counts directions, not vectors, and not non-zero entries. Rank is easy to confuse with simpler counts. It is not the number of vectors, which can exceed the rank when passengers are present. It is not the number of non-zero entries, nor the number of columns that happen to look different. Rank is strictly the number of independent directions the set reaches, which requires seeing through combinations, not just scanning the vectors. This is exactly why we need a real computation for it in high dimensions, where the redundancy is invisible.

Dimension is a property of the space, rank is a property of your particular set. These blur together because in the nicest case they coincide, an independent set spanning the whole plane has rank two and the plane has dimension two. But they answer different questions. Dimension asks how many directions the space has room for; rank asks how many your specific vectors actually reach. A set of vectors in three-dimensional space can have rank one, two, or three. The space's dimension is fixed at three; your set's rank depends on what you brought.

Check your intuition

Try each before opening the answers.

1. Are the vectors $[2, 0]$ and $[0, 5]$ independent? Are $[2, 0]$ and $[4, 0]$? Answer from the picture, not a computation.
2. A set contains the zero vector along with two other, genuinely different vectors. Can the set be independent? Think about what combination could reach the origin.
3. Three vectors live in the plane, ordinary two-dimensional space. Can they be independent? What does this say about their rank?
4. A matrix has four columns, but its rank is two. In plain terms, what does that tell you about the columns?
5. You measure the rank of a 1000×1000 matrix of data and find it is 40. What does this suggest about the data, and why might you be pleased to discover it?

Answers

1. The first pair is independent: $[2, 0]$ points along one axis and $[0, 5]$ along the other, two different directions, neither buildable from the

other. The second pair is dependent: $[4, 0]$ is just $[2, 0]$ scaled by two, the same direction, a passenger. Both vectors of the second pair lie on the horizontal axis and reach only that line.

2. No, it cannot be independent. The zero vector is always a passenger, because you can take some amount of it, any amount at all, and none of the others, and still sit at the origin: five times the zero vector is zero. That is a non-trivial combination reaching zero, one amount non-zero, which is exactly what dependence means. Any set containing the zero vector is automatically dependent.

3. No, three vectors in the plane can never be independent. The plane has dimension two, so any basis has two vectors, and no independent set can be larger than a basis. A third vector in the plane is always buildable from two independent ones already there. So three vectors in the plane have rank at most two: at least one is a passenger, no matter which three you pick.

4. Two of the four columns are redundant. The columns reach only two independent directions, so whatever the other two columns are, they are combinations of those two, buildable from them and adding no new reach. The matrix's transformation, despite having four columns to work with, can only produce a two-dimensional space of outputs.

5. The data has a thousand features but only forty independent directions; the other nine hundred and sixty are, in effect, combinations of those forty. The data looks high-dimensional but truly lives in a small subspace. You would be pleased because it means the data is highly compressible: you could keep forty directions and discard the rest with little loss, turning a thousand-number representation into something far smaller and easier to work with. A low rank in a large matrix is an opportunity, and finding it is the premise of dimensionality reduction.

8 The Determinant

The itch

A matrix moves space. It stretches, rotates, shears, flips. Watching the basis vectors land tells us exactly where everything goes, but it does not, at a glance, tell us one simple thing we often want to know: did the transformation make things bigger or smaller, and by how much?

Picture a transformation acting not on a single vector but on a whole region, say the unit square, the little square with corners at the origin, $[1, 0]$, $[0, 1]$, and $[1, 1]$. Apply a matrix and that square becomes some new shape, a parallelogram, tilted and resized. It has an area. The question is how that area compares to the square we started with. Did the transformation double the area, halve it, leave it alone? A single number answers this for the whole transformation, and it turns out to answer far more than the question that produced it.

That number is the determinant. It is the factor by which a transformation scales area, and packed into that one idea are a surprising number of the things we most want to know about a matrix: whether it flips space over, whether it flattens space into something lower-dimensional, and whether the transformation can be undone at all. This short chapter builds the determinant from the one question of area, and then reads those other answers straight off it.

The picture

Begin with the unit square, area one. Apply a transformation and it becomes a parallelogram, the one whose two sides are the landing spots of the basis vectors, the two columns of the matrix. The determinant is simply the area of that parallelogram, compared against the square we started with.

If the parallelogram has area three, the determinant is three: the transformation triples areas. Not just the area of this one square, but every area in the plane, because the transformation treats space uniformly. A region of area five would become area fifteen under the same matrix. The determinant is a single scaling factor that applies everywhere at once. If the parallelogram has area one half, the determinant is one half, and the transformation shrinks every area to half its size.

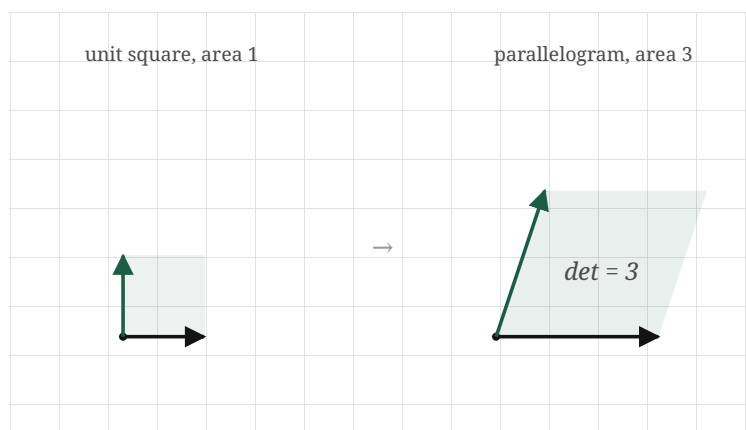


Figure 8.1: The unit square becomes a parallelogram under a transformation. The determinant is the area of that parallelogram: the factor by which every area in the plane is scaled.

Now the case that carries the most meaning. Suppose the transformation sends both basis vectors onto the same line, as it would if its two columns

pointed the same way. The unit square is flattened completely, squashed onto a line, and a line has no area. Its area is zero, so the determinant is zero. A determinant of zero is the signature of a transformation that collapses space, flattening the plane onto a line or onto a single point, destroying a dimension in the process.

This should connect immediately to the last chapter. A transformation whose columns point the same way is one whose columns are dependent, a rank-deficient transformation. So a zero determinant and rank deficiency are the same event seen from two angles: the columns fail to be independent, the space they reach collapses to something lower-dimensional, and the area of that collapsed image is zero. The determinant gives us a single number that detects the collapse we learned to count with rank.

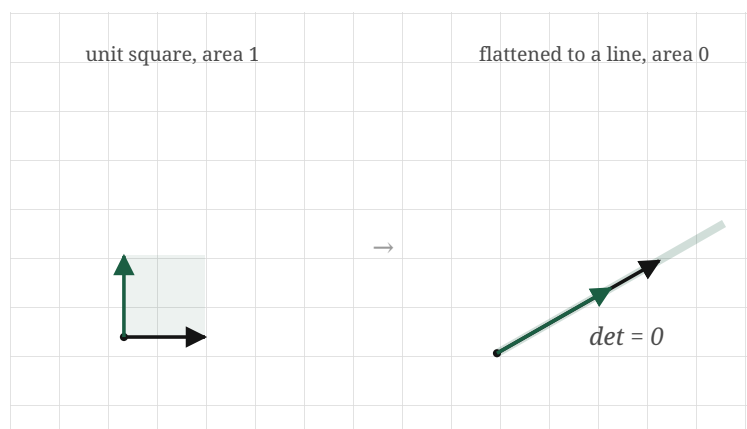


Figure 8.2: When a transformation collapses the square onto a line, the parallelogram has zero area. A zero determinant is the signature of collapse: the same event as rank deficiency.

There is one more thing the area quietly carries, a sign. If a transformation flips the plane over, like turning a page, reflecting left and right, it reverses orientation, and we record this by letting the determinant go negative. A determinant of -2 means areas are doubled and space is flipped. The

8 The Determinant

size of the determinant is the area factor; the sign tells us whether the transformation preserved the plane's handedness or turned it over. For now the magnitude is the main story, but the sign is a genuine part of the number, not an accident.

The math, built up

For a two-by-two matrix, the area of the parallelogram has a short formula, and it is the one piece of determinant arithmetic worth knowing by heart. If

$$A = \begin{bmatrix} a & b \\ c & d \end{bmatrix},$$

then its determinant is

$$\det(A) = ad - bc.$$

The columns are $[a, c]$ and $[b, d]$, the two sides of the parallelogram, and $ad - bc$ is exactly the area that parallelogram encloses, carrying its sign. We will not grind through the geometric derivation of why this particular combination gives the area; it comes from the base-times-height of the parallelogram worked out in coordinates, and the details are in the appendix. What matters is reading it and trusting what it measures.

Test it against the cases we can already see. The identity matrix, which leaves everything alone, is $a = 1, b = 0, c = 0, d = 1$, giving $\det = 1 \cdot 1 - 0 \cdot 0 = 1$. Area unchanged, exactly right for a transformation that does nothing. A matrix that doubles both axes, $a = 2, d = 2$, others zero, gives $\det = 4$: doubling each of two dimensions quadruples area, which matches doubling the side of a square. And a collapse, columns $[1, 2]$ and

$[2, 4]$ where the second is twice the first, gives $\det = 1 \cdot 4 - 2 \cdot 2 = 0$: the zero that signals the square flattened onto a line.

In three dimensions the determinant measures a volume rather than an area, the volume of the parallelepiped that the unit cube becomes, and the formula grows longer, but the meaning is identical: the factor by which the transformation scales volume, zero when it collapses the cube flat. In any number of dimensions the determinant is the single number for how the transformation scales content, whatever the word for content is in that many dimensions, and zero always means collapse. The formula gets more elaborate; the idea does not move.

Build it yourself

The determinant is one function call, but let us confirm the geometry first: the two-by-two formula really does give the parallelogram's scaling, and zero really does mark collapse.

Start with the hand formula on a matrix that doubles areas:

```
import numpy as np

A = np.array([[2.0, 0.0],
              [0.0, 2.0]])

a, b = A[0, 0], A[0, 1]
c, d = A[1, 0], A[1, 1]
print(a * d - b * c)
```

4.0

Four, as we reasoned: doubling both axes scales area by four. NumPy has it built in:

8 The Determinant

```
print(np.linalg.det(A))
```

4.0

The same four. Now watch a collapse. Here is a matrix whose second column is twice the first, so its columns are dependent:

```
collapse = np.array([[1.0, 2.0],  
                    [2.0, 4.0]])  
print(np.linalg.det(collapse))
```

0.0

Zero, give or take the faint floating-point dust these computations leave. The determinant has detected that this transformation flattens the plane onto a line. And it agrees with what rank says about the same matrix:

```
print(np.linalg.matrix_rank(collapse))
```

1

Rank one: one independent direction, the collapse counted the other way. The zero determinant and the deficient rank are the same fact, reported by two different tools.

Finally the sign, the orientation flip. Here is a matrix that swaps the two axes, sending $[1, 0]$ to $[0, 1]$ and $[0, 1]$ to $[1, 0]$, which reflects the plane:

```
flip = np.array([[0.0, 1.0],  
                [1.0, 0.0]])  
print(np.linalg.det(flip))
```

-1.0

Negative one. The magnitude one says areas are preserved, nothing is stretched, and the negative sign says the plane was turned over in the process. The determinant reported both the size and the flip in a single number.

Where it lives in ML

The determinant's largest role in machine learning is as an alarm bell. A zero determinant means a transformation has collapsed space, and collapse is exactly the condition under which many operations fail. The clearest case is inverting a matrix, undoing a transformation. If a transformation flattened the plane onto a line, there is no way to undo it, because everything that landed on a given point of the line came from a whole set of different starting points, and undoing cannot know which one to return to. Information was destroyed in the collapse, and no inverse can recover it. The determinant being zero is the precise signal that a transformation cannot be undone, which we will lean on heavily in the next chapter when we ask when a system of equations has a unique answer.

In practice the danger is rarely a determinant of exactly zero. It is a determinant very close to zero, which signals a transformation that nearly collapses space, squashing it almost but not quite flat. Undoing such a transformation is possible in principle but treacherous in practice: tiny errors in the input get magnified enormously, because the near-collapse has to be violently un-squashed. A matrix with a near-zero determinant is called ill-conditioned, and it is a well-known source of numerical trouble in machine learning, where models that require inverting such a matrix produce wildly unstable results. Watching the determinant, or quantities related to it, is part of knowing whether a computation can be trusted.

There is a second appearance, more specialised, in models that transform probability distributions. Some modern generative models work by taking a simple distribution and pushing it through a transformation to produce a complicated one. When you stretch or squash space, you stretch or squash the probability density living in it, and the factor by which the density changes is exactly the determinant of the transformation. These models have to compute determinants constantly, to keep track of how the probability mass is being redistributed as it flows through each step. The area-scaling idea from a unit square turns out to be the accounting that keeps such a model honest about where its probability went.

Common misunderstandings

The determinant is a property of a transformation, not a number you read off the entries. The formula $ad - bc$ makes the determinant look like just an arithmetic combination of the matrix entries. It is really the area-scaling factor of the transformation, and the arithmetic is only how we compute it. Two matrices with quite different entries can have the same determinant, because they scale area by the same factor while doing different things to space. Read the determinant as what the transformation does to content, not as a sum of products.

A zero determinant does not mean the transformation sends everything to zero. Collapse means the plane is flattened onto a line, or three-dimensional space onto a plane, not that everything is sent to the origin. A great many points survive a collapse; they just all get squashed onto a lower-dimensional space. The image is a line full of points, not a single point. Zero determinant means zero *area*, not zero output.

Never test a determinant for exact equality with zero in code. The collapse case in our own experiment did not print a clean zero; it printed a speck of floating-point dust, something like -2×10^{-16} . This is the approximate arithmetic we have met before, and it means a test like

Check your intuition

`det == 0` will almost always answer `False` even for a genuinely collapsed matrix, because the computed value is a hair off. The honest check is whether the determinant is *close* to zero, small compared to the scale of the matrix, not exactly equal to it. Exact-zero tests on computed determinants are a reliable way to be quietly wrong.

A large determinant does not mean a “big” or “important” matrix. The determinant measures area scaling, nothing more. A matrix can have enormous entries and a determinant of zero, if it collapses space, and a matrix can have a modest determinant while stretching one direction enormously and squashing another, since the determinant reports only the *product* of the stretches, the net effect on area. It is not a measure of size or significance. For how much a transformation stretches individual directions, we will need the singular values of a later chapter; the determinant only ever reports their combined effect on area.

Check your intuition

Try each before opening the answers.

1. A transformation has determinant 5. You apply it to a shape of area 2. What is the area of the result?
2. A two-by-two matrix has columns $[3, 1]$ and $[6, 2]$. Without computing $ad - bc$, what do you expect its determinant to be, and why?
3. A transformation has determinant -1 . What does it do to areas, and what does the sign tell you?
4. Matrix A has determinant 3 and matrix B has determinant 4. You apply B first, then A . By what factor does the combined transformation scale area?

8 The Determinant

5. You are about to invert a matrix as part of fitting a model, and you notice its determinant is 0.0000001, very close to zero. Should you be comfortable? What does this tell you about the computation ahead?

Answers

1. Ten. The determinant is the factor by which every area is scaled, so an area of two becomes $2 \times 5 = 10$. The determinant applies uniformly to every region, whatever its shape or size.

2. Zero. The second column $[6, 2]$ is exactly twice the first column $[3, 1]$, so the two columns point the same way, are dependent, and the parallelogram they form is flattened onto a line with no area. A collapse like this always has determinant zero, and you can see it from the columns without touching the formula. (The arithmetic confirms it: $3 \cdot 2 - 6 \cdot 1 = 0$.)

3. It preserves areas, since the magnitude of the determinant is one, and it flips the plane over, since the sign is negative. Nothing is stretched or shrunk; the shape keeps its size but its orientation is reversed, like a reflection in a mirror. Size from the magnitude, handedness from the sign.

4. Twelve. Composing transformations multiplies their area-scaling factors: the first scales area by four, and the second scales that already-scaled area by three, for a total of $4 \times 3 = 12$. Determinants of composed transformations multiply, which is exactly what you would expect from scaling an area and then scaling it again.

5. You should be wary. A determinant that close to zero means the matrix nearly collapses space, and inverting a near-collapse is numerically dangerous: small errors in your data will be magnified dramatically in the result, and the model's output may be unstable and untrustworthy. The matrix is ill-conditioned. It is not strictly impossible to proceed, but the near-zero determinant is a warning that the computation ahead is fragile and the answer may not mean what you hope.

9 Systems of Equations, Geometrically

The itch

Somewhere in school most of us met equations like $2x + 3y = 12$ and $x - y = 1$, two equations, two unknowns, and were taught to grind them into a solution by elimination or substitution. It worked, but it was arithmetic in the dark. Nothing about the procedure said what we were really doing or why an answer existed. We have spent six chapters building the machinery to see it clearly, and now the whole subject snaps into a single picture.

Here is the reframing. Stack those equations into a matrix acting on a vector of unknowns. The two equations above are exactly

$$\begin{bmatrix} 2 & 3 \\ 1 & -1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} 12 \\ 1 \end{bmatrix},$$

which is just $A\mathbf{x} = \mathbf{b}$. Read it as a transformation. The matrix A is a transformation, $\mathbf{x}$ is some unknown vector, and $\mathbf{b}$ is where $\mathbf{x}$ lands after A acts on it. We know the transformation and we know the destination. The question is where we started: which vector $\mathbf{x}$ does the transformation A send to $\mathbf{b}$?

That is what solving a system of equations is. Not grinding through elimination, but running a transformation backwards: given where something

landed, find where it began. And once we see it this way, everything we already know about transformations, whether they collapse space, whether they can be undone, what their determinant and rank say, tells us immediately whether the system has one answer, no answer, or infinitely many. The dark arithmetic becomes a question about undoing a known motion.

The picture

Think of $A\mathbf{x} = \mathbf{b}$ as a search. The transformation A takes every vector somewhere. We are told that some particular vector $\mathbf{x}$ was taken to $\mathbf{b}$, and we want that $\mathbf{x}$ back. If we could run A in reverse, we would apply the reverse to $\mathbf{b}$ and arrive at $\mathbf{x}$ directly. So solving the system is really asking whether A can be run in reverse, and if so, running it.

For a well-behaved transformation, one that does not collapse space, running it in reverse always works, and works uniquely. Every output came from exactly one input, because a non-collapsing transformation never sends two different vectors to the same place. So there is a single vector that lands on $\mathbf{b}$, and the system has exactly one solution. Geometrically, A stretched and rotated and sheared space without flattening it, so we can perfectly undo those moves and walk $\mathbf{b}$ back to its unique origin.

Now the case where it breaks, and it is the collapse from the last two chapters. Suppose A flattens the plane onto a line, a rank-deficient, zero-determinant transformation. Then many different input vectors all get squashed onto the same output. Running the transformation backwards is no longer well-defined: given a point on the line, we cannot say which of the many vectors that landed there was our $\mathbf{x}$. And here the destination $\mathbf{b}$ decides between two fates.

If $\mathbf{b}$ does not even lie on the line that A squashes everything onto, then nothing maps to $\mathbf{b}$ at all. No input lands there, so there is no solution. The equations are asking for a vector that cannot exist, like asking which

vector a flattening sends to a point floating off the line, when the flattening only ever produces points on the line.

If $\mathbf{b}$ does lie on that line, the opposite problem appears. Not one vector maps to it but a whole infinite family of them, the entire set of inputs that got squashed onto that point. The system has infinitely many solutions, a whole line of them, and no single answer can be singled out. Collapse turns one clean answer into either none or endlessly many, and which one depends on whether the target survived the collapse.

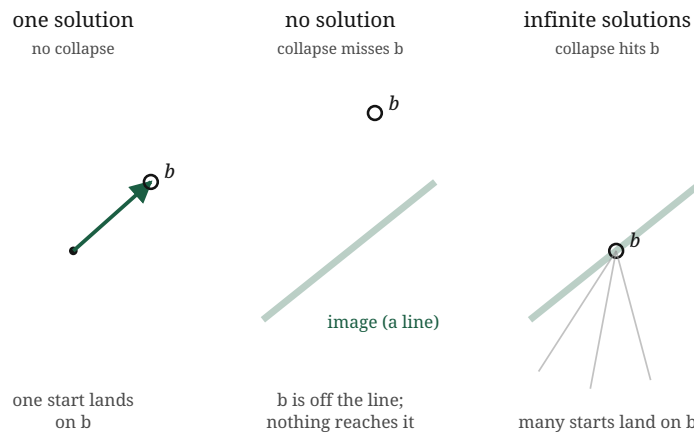


Figure 9.1: Three fates for a system. A non-collapsing transformation runs backward to one unique start. A collapse either misses the target entirely, giving no solution, or lands on it from infinitely many starts.

So the three possibilities of a linear system, the one solution, no solution, and infinitely many that the old elimination method produced as mysterious special cases, are not mysterious at all. They are whether the transformation collapses, and if it does, whether the target lies inside the

collapsed image. Everything is read off the geometry.

The math, built up

Running a transformation backwards has a name and a notation. When A does not collapse space, there is another transformation that undoes it, called the **inverse** of A and written A^{-1} . It is the transformation that takes every landing spot back to where it came from, so that applying A and then A^{-1} returns every vector unchanged. With it, the solution to the system is immediate:

$$A\mathbf{x} = \mathbf{b} \implies \mathbf{x} = A^{-1}\mathbf{b}.$$

Apply the undoing transformation to the destination, and you arrive at the unique start. That is the entire solution when the inverse exists, compressed into one line: to solve the system, run the transformation backwards on $\mathbf{b}$.

The inverse exists exactly when the transformation does not collapse space, which we now have several equivalent ways to state, all saying the same thing. The inverse exists when the determinant is non-zero, because a zero determinant is collapse. It exists when the columns are independent, when the matrix has full rank, because dependence is collapse counted another way. A matrix with an inverse is called **invertible**, and invertible, non-zero determinant, full rank, and independent columns are four names for one property: the transformation preserves space instead of flattening it, so it can be undone.

When that property fails, the inverse does not exist, and the notation A^{-1} simply has nothing to refer to. There is no single transformation that undoes a collapse, because the collapse destroyed the information needed to undo it. This is why a system with a collapsing A has no clean solution: the very object that would produce the answer, A^{-1} , cannot be formed.

Build it yourself

The system might still have solutions, the infinitely many of the previous section, but they cannot be reached by inverting, because there is nothing to invert.

We will not write out the formula for computing an inverse by hand. For the two-by-two case it is short, a rearrangement of the entries divided by the determinant, which already shows the trouble: the determinant sits in the denominator, so a zero determinant means dividing by zero, and the inverse blows up exactly when the transformation collapses. That single fact, the determinant in the denominator, is the arithmetic shadow of everything the picture told us. In practice the inverse is computed by machine, and often not computed at all, because there are cheaper ways to solve $A\mathbf{x} = \mathbf{b}$ than by building the whole inverse transformation. But the meaning is what matters: solving is undoing, and undoing is possible exactly when the transformation did not collapse.

Build it yourself

Let us solve a system by running the transformation backwards, and then watch what happens when the transformation collapses.

Here is the system from the start of the chapter, as a matrix and a target:

```
import numpy as np

A = np.array([[2.0, 3.0],
              [1.0, -1.0]])
b = np.array([12.0, 1.0])
```

The determinant is non-zero, so A does not collapse space and can be run backwards. We can form the inverse and apply it to $\mathbf{b}$, exactly as the formula $\mathbf{x} = A^{-1}\mathbf{b}$ says:

9 Systems of Equations, Geometrically

```
A_inv = np.linalg.inv(A)
x = A_inv @ b
print(x)
```

```
[3.  2.]
```

The solution is $[3, 2]$: the transformation A sends the vector $[3, 2]$ to $[12, 1]$, and we recovered it by undoing A . We can check by going forwards again, applying A to our answer and confirming it lands on $\mathbf{b}$:

```
print(A @ x)
```

```
[12.  1.]
```

Back to $[12, 1]$, the destination we were given. The undoing was correct.

In real work you rarely build the inverse; you ask the solver for $\mathbf{x}$ directly, which is faster and steadier:

```
x = np.linalg.solve(A, b)
print(x)
```

```
[3.  2.]
```

The same $[3, 2]$. `np.linalg.solve` runs the transformation backwards without constructing the whole inverse transformation, which is the method to reach for in practice.

Now a collapsing system. Here the two rows describe the same direction, the second just twice the first, so A flattens the plane:

```
A_bad = np.array([[1.0, 2.0],
                  [2.0, 4.0]])
b_bad = np.array([3.0, 99.0])

print(np.linalg.det(A_bad))
```

0.0

A determinant of zero, the signature of collapse. Asking to undo it fails, because there is nothing to undo:

```
try:
    np.linalg.inv(A_bad)
except np.linalg.LinAlgError as e:
    print("no inverse:", e)
```

no inverse: Singular matrix

The library refuses, reporting a singular matrix, which is the formal word for a collapsing, non-invertible transformation. The geometry predicted this exactly: a flattened transformation cannot be run backwards, so the system it defines has no clean solution to hand back.

Where it lives in ML

Solving $A\mathbf{x} = \mathbf{b}$ is not a side-topic in machine learning; it is close to the centre of it. An enormous amount of what models do, especially the classical ones, comes down to setting up a system of linear equations and solving it. The most direct case is linear regression, the workhorse we will build in Part 4. Fitting a straight relationship to data turns into a system

9 Systems of Equations, Geometrically

of exactly this form, and the best-fit answer is found by solving it, by running a transformation backwards to recover the weights that produced the data.

But real data makes the story richer than the clean three-fates picture, and the enrichment is worth seeing now. When we fit a model, we usually have far more equations than unknowns, one equation per data point and only a handful of weights to find, thousands of demands on a few numbers. Such a system almost never has an exact solution. The equations contradict each other slightly, because real data is noisy, and no single choice of weights satisfies all of them at once. In the strict sense of this chapter, there is no solution: the target $\mathbf{b}$ does not lie exactly in the reachable image.

Rather than give up, we change the question. Instead of demanding a vector that lands exactly on $\mathbf{b}$, we look for the one that lands as close to $\mathbf{b}$ as possible, closest in the straight-line distance we built back with the L2 norm. This is the least-squares solution, and it is what fitting a model actually computes: not the impossible exact answer, but the nearest reachable one. The whole of linear regression is this move, and it leans on two ideas from earlier chapters at once, the reachable image from this one and the notion of closeness from the norms chapter. We are not ready to build it fully, but notice that the machinery is nearly all in hand.

The collapse case matters here too, as a practical warning rather than a curiosity. When the features of a dataset are redundant, the Celsius-and-Fahrenheit problem from two chapters ago, the system that fitting produces is collapsing or nearly so, and the transformation we would need to invert is singular or ill-conditioned. The model cannot find a unique answer, or finds a wildly unstable one, for exactly the reason this chapter lays out: there is no clean way to run a collapse backwards. Recognising a rank-deficient system, and repairing it before solving, is a routine part of making a model actually work, and it is the geometry of this chapter applied with care.

Common misunderstandings

Solving a system is undoing a transformation, not manipulating rows. The elimination procedures taught first, swapping and scaling and subtracting rows, are one method for computing the answer, but they are not what solving *means*. What we are doing is running a transformation backwards to recover an input from its output. Holding the geometric meaning is what lets you predict, before any computation, whether a solution exists and whether it is unique. The row manipulations are a means; the undoing is the meaning.

No solution and infinitely many solutions both come from collapse, but they are opposite situations. It is easy to lump the two non-unique cases together as “the system went wrong.” They are precisely opposite. No solution means the target lies outside the reachable image, so nothing maps to it. Infinitely many means the target lies inside a collapsed image that many inputs share. One is a target too far out to reach; the other is a target reachable in too many ways. Both stem from a collapsing transformation, but which one occurs depends entirely on where the target sits.

An inverse existing is a property of the matrix, not of the particular target. Whether A^{-1} exists depends only on A , on whether the transformation collapses space. It does not depend on $\mathbf{b}$. The target only decides, in the collapsing case, between no solution and infinitely many. When A is invertible, every possible target has exactly one solution, no matter what $\mathbf{b}$ is. Keep straight what depends on the transformation, invertibility, and what depends on the target, which non-unique fate you land in when invertibility fails.

In practice you rarely want the inverse itself. Beginners often reach to compute A^{-1} and then multiply, because the formula $\mathbf{x} = A^{-1}\mathbf{b}$ suggests it. Building the entire inverse transformation is more work and numerically worse than solving for $\mathbf{x}$ directly, which is why the tools offer a `solve` that never forms the inverse. Reserve the inverse for when you genuinely need

9 Systems of Equations, Geometrically

the whole undoing transformation, not just the answer to one system. For a single system, solve directly.

Check your intuition

Try each before opening the answers.

1. A transformation A does not collapse space. How many solutions does $A\mathbf{x} = \mathbf{b}$ have, and does the answer depend on which $\mathbf{b}$ you pick?
2. You are told $\det(A) = 0$. What are the possible numbers of solutions to $A\mathbf{x} = \mathbf{b}$, and what decides between them?
3. A system $A\mathbf{x} = \mathbf{b}$ has infinitely many solutions. What does this tell you about the transformation A , and about where $\mathbf{b}$ sits?
4. You try to solve a system and the library reports the matrix is “singular.” In the language of this book, what has it found?
5. A dataset has one thousand data points and three weights to fit, giving a system with a thousand equations and three unknowns. Do you expect an exact solution? If not, what do we look for instead?

Answers

1. Exactly one solution, and this holds for every $\mathbf{b}$. A non-collapsing transformation is invertible, so it can be run backwards uniquely: every target came from exactly one start. Because invertibility is a property of A alone, the guarantee of a single solution applies no matter which target you choose. There is never no solution and never more than one.
2. A zero determinant means A collapses space, so the system has either no solution or infinitely many, never exactly one. Which of the two occurs is decided by the target $\mathbf{b}$: if $\mathbf{b}$ lies within the

collapsed image, the reachable line or plane, there are infinitely many solutions; if $\mathbf{b}$ lies outside it, there are none. The determinant tells you uniqueness is gone; the target tells you which way.

3. The transformation A collapses space, it is singular, rank-deficient, with a zero determinant. And $\mathbf{b}$ lies inside the collapsed image, on the line or plane that A squashes everything onto, which is why it is reachable at all. Infinitely many inputs get mapped onto that image, and since $\mathbf{b}$ is on it, a whole family of them land exactly on $\mathbf{b}$. Reachable target plus collapsing transformation gives an infinite family of solutions.

4. It has found that the matrix collapses space: it is non-invertible, its determinant is zero, its columns are dependent, its rank is deficient. “Singular” is the formal name for all of these at once, the transformation that cannot be run backwards. The library is telling you there is no clean answer to hand back, because the undoing it would need does not exist.

5. No exact solution should be expected. A thousand equations pressing on three unknowns will almost always contradict one another, because real data carries noise, so no three weights can satisfy every equation at once, and the target does not lie exactly in the reachable image. Instead of an exact answer we look for the least-squares solution, the weights that land as close to the target as possible in straight-line distance. That nearest-reachable answer is what fitting a model actually finds.

10 Eigenvectors and Eigenvalues

The itch

When a transformation acts on the plane, it does something different to almost every vector. It turns them. An arrow pointing one way gets swung to point another, its direction changed along with its length. Watch a shear or a rotation act on a fan of arrows and they all pivot, each ending up aimed somewhere new. Direction is not preserved; that is most of what a transformation does.

But not every arrow turns. For most transformations there are a few special directions that come through unturned. A vector pointing exactly along one of these special directions gets stretched or squashed, made longer or shorter, but it keeps pointing exactly the way it did. The transformation moved it along its own line without ever knocking it off that line. These survivor directions are rare, usually just one or two in the plane, and everything else pivots around them.

Those special directions are the deep structure of a transformation. They are the axes the transformation is, in a sense, built around, the directions it treats most simply, by pure stretching with no turning. If we can find them, a complicated-looking transformation resolves into something far easier to understand: along these special directions it merely scales, and its whole behaviour can be read through what it does to them. This chapter is about finding those directions and the amounts they get stretched by, because they are among the most useful things a matrix can tell us, and they are the foundation of the flagship chapter that follows.

The picture

Take a transformation and watch what it does to arrows pointing in every direction. Nearly all of them get turned: the arrow and its transformed version point different ways, the transformation having swung it off its original line. Picture the original arrow and its image as two arrows from the origin, and for most starting directions those two make an angle, evidence that the direction was not preserved.

Now hunt for the exceptions. Is there a direction where the arrow and its transformed image still lie along the very same line? Where the transformation, whatever else it did, did not swing the arrow off course but only slid it along its own line, longer or shorter or reversed, but never turned? For most transformations such directions exist, and they are special enough to name. A direction that survives a transformation unturned is an **eigenvector** of that transformation, and the factor by which the arrow along it gets stretched is the **eigenvalue** for that direction.

An example makes the two ideas concrete. Consider a transformation that stretches everything horizontally by three and leaves the vertical untouched. A vector pointing straight along the horizontal axis gets tripled in length but stays horizontal: it is an eigenvector, with eigenvalue three. A vector pointing straight up is left completely alone, unmoved: it too is an eigenvector, with eigenvalue one, since it is stretched by a factor of one, which is to say not at all. But a vector pointing diagonally gets pulled flatter as its horizontal part triples and its vertical part stays put, so it ends up aimed differently than it began. The diagonal direction is turned; it is not an eigenvector. The two axes survive; the diagonal does not.

Eigenvalues carry sign and size, and both mean something plain. An eigenvalue larger than one means the eigenvector is stretched; between zero and one, it is squashed toward the origin; exactly one, it is left unchanged. A negative eigenvalue means the arrow is flipped to point backward along its own line, which still counts as unturned in the sense that matters, since it stays on the same line through the origin, just now facing the other way.

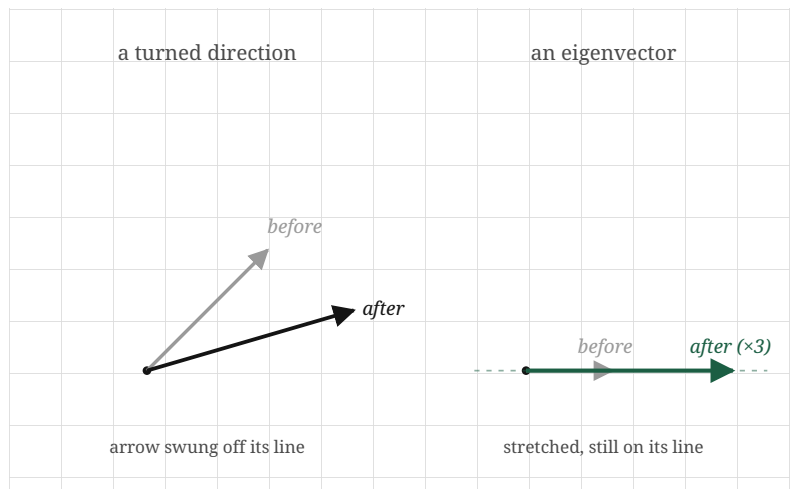


Figure 10.1: Most directions are turned by a transformation. An eigenvector is a rare direction that survives unturned, merely stretched along its own line. The stretch factor is its eigenvalue.

10 Eigenvectors and Eigenvalues

And an eigenvalue of zero means the eigenvector is squashed all the way to nothing, collapsed onto the origin, which is the direction a collapsing transformation destroys.

Not every transformation has these survivor directions in the plane. A pure rotation, which turns every arrow by the same angle, turns all of them, and so has no real eigenvectors at all, no direction survives a quarter-turn unturned. This is not a flaw in the idea; it is the idea correctly reporting that a rotation preserves no direction. When survivor directions do exist, though, they expose the transformation's skeleton.

The math, built up

The picture translates into one compact equation. An eigenvector is a vector $\mathbf{v}$ that the transformation A sends to a scaled copy of itself, stretched by the eigenvalue λ but not turned:

$$A\mathbf{v} = \lambda\mathbf{v}.$$

Read each side through the picture. The left side, $A\mathbf{v}$, is the transformation acting on the vector, sending it wherever it goes. The right side, $\lambda\mathbf{v}$, is the same vector merely scaled, still on its own line. The equation says these are equal: applying the whole transformation to $\mathbf{v}$ does no more than stretch it by λ . That is the entire definition, and it is just the survivor-direction idea written in symbols. Everything about eigenvectors is unpacking this one line.

There is a subtlety the equation makes clear. If $\mathbf{v}$ is an eigenvector, so is any scaling of it, because stretching a survivor direction gives another arrow on the same line, equally unturned. So eigenvectors come as whole lines, not single arrows; the direction is what matters, and we usually pick one representative arrow along it, often scaled to length one. The eigenvalue,

by contrast, is a single definite number: how much that direction gets stretched, the same whichever representative we chose.

Finding eigenvectors means finding the $\mathbf{v}$ and λ that satisfy the equation, and the route to them runs straight through the last two chapters. Rearrange the equation so one side is zero. We want $A\mathbf{v} - \lambda\mathbf{v} = \mathbf{0}$, which, treating $\lambda\mathbf{v}$ as a transformation that scales $\mathbf{v}$ by λ , becomes $(A - \lambda I)\mathbf{v} = \mathbf{0}$, where I is the identity. This says the transformation $A - \lambda I$ sends the non-zero vector $\mathbf{v}$ to the origin. But a transformation that sends a non-zero vector to zero is a collapsing one, and we know its signature exactly: its determinant is zero. So the eigenvalues are precisely the numbers λ for which

$$\det(A - \lambda I) = 0.$$

This is the bridge from the picture to a computation. Each λ that makes $A - \lambda I$ collapse is an eigenvalue, and for each such λ , the directions that get sent to zero are its eigenvectors. We will not turn this into a hand procedure, solving the resulting equation for λ and back-substituting, because for anything beyond the smallest matrices it is done by machine. What matters is that eigenvalues are the stretch factors, eigenvectors are the survivor directions, and the whole search reduces to asking when a related transformation collapses, which ties eigenvectors directly to the determinant and rank we already built.

Build it yourself

Finding eigenvectors by hand is laborious, but NumPy does it in one call, and we can verify the survivor-direction picture directly.

Take the horizontal stretch from the picture, tripling the first axis and leaving the second:

10 Eigenvectors and Eigenvalues

```
import numpy as np

A = np.array([[3.0, 0.0],
              [0.0, 1.0]])

values, vectors = np.linalg.eig(A)
print(values)
print(vectors)
```

```
[3.+0.j 1.+0.j]
[[1.+0.j 0.+0.j]
 [0.+0.j 1.+0.j]]
```

The eigenvalues come back as three and one, the two stretch factors we reasoned out. The eigenvectors are returned as the columns of the second array, and they are the horizontal and vertical directions, each written as a unit-length arrow. The transformation's survivor directions are exactly the two axes, exactly as the picture promised.

Now confirm the defining property directly: applying A to an eigenvector should equal simply scaling that eigenvector by its eigenvalue. Take the first eigenvector and its eigenvalue and check both sides of $A\mathbf{v} = \lambda\mathbf{v}$:

```
v = vectors[:, 0]          # first eigenvector
lam = values[0]             # its eigenvalue

print(A @ v)               # the transformation acting on v
print(lam * v)              # v merely scaled by the eigenvalue
```

```
[3.+0.j 0.+0.j]
[3.+0.j 0.+0.j]
```


The two lines print the same vector. Applying the whole transformation to this direction did no more than scale it, which is what makes it an eigenvector. The arrow was not turned, only stretched.

Watch a turned direction fail the same test. A diagonal vector is not an eigenvector of this transformation, so applying A to it should not give a mere scaling:

```
d = np.array([1.0, 1.0])    # a diagonal direction
print(A @ d)                # becomes [3, 1]
```

```
[3.  1.]
```

The diagonal $[1, 1]$ becomes $[3, 1]$, which does not point the same way as $[1, 1]$; the direction was turned, pulled flatter. No single number scales $[1, 1]$ into $[3, 1]$, which is precisely what it means for a direction to fail to be an eigenvector.

And the rotation with no survivors behaves as the picture warned. A quarter-turn turns every direction, so it should have no real eigenvectors:

```
R = np.array([[0.0, -1.0],
               [1.0,  0.0]])
print(np.linalg.eig(R).eigenvalues)
```

```
[0.+1.j 0.-1.j]
```

The eigenvalues come back as complex numbers, not real ones, which is the arithmetic's way of reporting that no real direction survives the rotation unturned. When a transformation preserves no direction, the real survivor directions simply are not there to find.

Where it lives in ML

Eigenvectors matter in machine learning because they find the directions that matter in data, and the most important instance is a technique we have been pointing at for several chapters: principal component analysis, or PCA. Its whole job is to look at a cloud of data and find the directions along which the data varies most, the axes the data is really organised around, so that we can keep those few directions and discard the rest. Those directions of greatest variation are eigenvectors, of a particular matrix built from the data, and their eigenvalues say how much variation each one carries. PCA is, at heart, an eigenvector computation, and dimensionality reduction is keeping the eigenvectors with the largest eigenvalues and throwing away the small ones.

The reason this works is the survivor-direction idea applied to data. A dataset spread through a high-dimensional space usually has a few directions along which it stretches out a great deal and many along which it barely varies. The high-eigenvalue directions are where the information lives; the low-eigenvalue directions are nearly flat, carrying almost nothing. Reducing the data to its top few eigenvectors keeps almost all of the structure while collapsing the description from thousands of numbers to a handful. It is the low-rank hope from the independence chapter, now made actionable: the eigenvalues tell us exactly which directions are worth keeping.

Eigenvalues also diagnose the behaviour of repeated transformations, which appear throughout machine learning whenever a process is applied over and over. When a transformation is applied many times in succession, its eigenvalues govern what happens in the long run. Directions with eigenvalue larger than one grow with each application, exploding if the process runs long enough; directions with eigenvalue smaller than one shrink away to nothing. This is the mathematics behind a famous difficulty in training deep and recurrent networks, where signals passed through many layers either blow up or vanish depending on whether the relevant eigenvalues sit above or below one. The exploding and vanishing gradients

that plague deep learning are, underneath, a story about eigenvalues of a repeated transformation, and understanding them starts exactly here.

There is a further appearance close to the systems this book was written alongside. The stability of many iterative methods, the way a training process either settles toward an answer or spirals away from it, is read through the eigenvalues of the transformation that each step applies. Whether a method converges is often precisely the question of whether certain eigenvalues lie inside a safe range. The survivor directions and their stretch factors, met here as a piece of geometry, turn out to decide whether some of the most important processes in the field work at all.

Common misunderstandings

An eigenvector is a direction, not a single arrow. Because any scaling of an eigenvector is also an eigenvector, lying on the same unturned line, it makes no sense to speak of *the* eigenvector as one specific arrow. The eigenvector is the whole line through the origin, the direction that survives. Tools return a single representative, usually scaled to length one, but that particular arrow is a choice of representative, not the eigenvector itself. What is fixed and meaningful is the direction and its eigenvalue, not the length or sign of the arrow the computer happened to hand back.

Not every transformation has real eigenvectors. It is tempting to assume the survivor directions always exist, but a rotation turns every direction and preserves none, so it has no real eigenvectors at all. The computation reports this by returning complex numbers rather than real ones. This is not a failure; it is the honest answer that no real direction comes through unturned. When you expect eigenvectors and get complex values, the transformation is doing something rotational, mixing directions together rather than merely stretching along fixed axes.

A large eigenvalue is not the same as an important eigenvector, unless the matrix is the right kind. For the symmetric matrices

that arise in PCA, built from data in a particular way, the eigenvalues genuinely rank the directions by importance, and keeping the largest is sound. But for a general matrix, an eigenvalue only tells you the stretch along its direction, and stretch is not always importance. The clean “keep the biggest eigenvalues” story belongs to the special, well-behaved matrices that dimensionality reduction uses, not to every matrix. Which is part of why the next chapter exists: it gives a version of this idea that works for *every* matrix, not only the well-behaved ones.

Eigenvalues can be negative or zero, and both mean something specific. A common assumption is that eigenvalues are positive stretch factors. A negative eigenvalue flips its eigenvector to point backward along the same line, still unturned in the sense that matters. A zero eigenvalue squashes its direction entirely to the origin, and its presence means the transformation collapses that direction, connecting eigenvalues straight back to the determinant: a transformation has a zero eigenvalue exactly when it collapses, exactly when its determinant is zero. Sign and zero are not edge cases to ignore; they carry the flip and the collapse.

Check your intuition

Try each before opening the answers.

1. A transformation stretches the vertical axis by four and leaves the horizontal axis unchanged. What are its eigenvectors and eigenvalues?
2. A vector $\mathbf{v}$ satisfies $A\mathbf{v} = -2\mathbf{v}$. Is $\mathbf{v}$ an eigenvector? What does the transformation do to it, geometrically?
3. A transformation has an eigenvalue of zero. What does this tell you about the transformation, in the language of earlier chapters?
4. Why does a pure rotation of the plane have no real eigenvectors? Answer from the picture, not the arithmetic.

5. A process applies the same transformation over and over to a vector. One direction has eigenvalue 1.5 and another has eigenvalue 0.5. What happens to each component as the process runs many times?

 Answers

1. The eigenvectors are the two axes. The vertical direction has eigenvalue four, since a vertical arrow is stretched fourfold while staying vertical. The horizontal direction has eigenvalue one, since a horizontal arrow is left completely unchanged, stretched by a factor of one. These two axes are the survivor directions; every other direction gets turned as its vertical part quadruples and its horizontal part stays fixed.

2. Yes, $\mathbf{v}$ is an eigenvector, with eigenvalue -2 . The equation says the transformation sends $\mathbf{v}$ to $-2\mathbf{v}$, which is $\mathbf{v}$ scaled by two and flipped to point the opposite way along the same line. It is unturned in the sense that matters, staying on its own line through the origin, but it is doubled in length and reversed in direction. Negative eigenvalues are exactly this flip-and-scale.

3. The transformation collapses space. A zero eigenvalue means some direction is squashed entirely to the origin, which is what a collapsing, rank-deficient transformation does, and it means the determinant is zero. So a zero eigenvalue is another face of the collapse we have met as zero determinant, deficient rank, and singularity. The transformation cannot be undone, because a direction has been destroyed.

4. A rotation turns every arrow by the same angle. An eigenvector would have to be a direction the transformation leaves on its own line, merely stretched, but a rotation by anything other than a whole half-turn swings every direction off its line, none excepted. There is simply no arrow that a genuine rotation leaves pointing along its original line, so there are no real eigenvectors. The transformation preserves no direction, and the absence of eigenvectors reports that faithfully.

5. The direction with eigenvalue 1.5 grows: each application multiplies it by 1.5, so after many rounds it has been multiplied by 1.5 many times over and becomes enormous. The direction with eigenvalue 0.5 shrinks: each application halves it, so it fades toward zero. Run long enough, the growing direction dominates completely and the shrinking one all but vanishes. This is the seed of exploding and vanishing behaviour in repeated transformations, decided entirely by whether an eigenvalue sits above or below one.

11 The Singular Value Decomposition

The itch

We ended the eigenvector chapter with an honest crack. Eigenvectors find the directions a transformation merely stretches, which is a powerful thing, but they are picky. They need square transformations, they can come out complex and refuse to point anywhere real, and even when they exist their stretch factors do not, in general, rank directions by importance. For the well-behaved matrices of PCA they work beautifully, but “well-behaved” is a real restriction, and most matrices in the world are not.

This chapter removes the restriction. It turns out that every matrix, without exception, square or not, well-behaved or not, can be broken into a sequence of the simplest possible transformations. Not just some matrices. Every single one. Whatever tangled thing a matrix does, stretching and shearing and rotating and collapsing all at once, it can always be pulled apart into three plain moves done in order: a rotation, then a stretch along perpendicular axes, then another rotation. Nothing more exotic than turning and stretching, and every transformation there is decomposes into exactly that.

This decomposition is called the singular value decomposition, and it is the most useful single result in this part of the book. It gives us, for any matrix at all, the true directions it stretches and the exact amounts, ranked cleanly from most to least. It makes the rank of a matrix visible at a

11 The Singular Value Decomposition

glance. It powers the compression of images and data, the recommendation systems that suggest what to watch next, and the dimensionality reduction that makes high-dimensional data tractable. Everything the part has built, transformations, rank, collapse, eigenvectors, comes together here into one construction that works on everything.

The picture

Here is the claim in full, and it is worth stating plainly because it sounds too strong to be true. Take any transformation whatsoever. However complicated its effect on space, it can always be performed in three steps: first rotate space, then stretch it along the horizontal and vertical axes by fixed amounts, then rotate again. Turn, stretch, turn. Every matrix is secretly just these three moves composed.

Walk through why that is remarkable. A general transformation can shear, can squash diagonally, can do things that seem to mix stretching and turning inseparably. The claim is that all of that apparent complexity is an illusion of viewpoint. If you rotate to the right starting orientation first, then the transformation's action becomes a plain stretch along the axes, pure scaling with no turning, and a final rotation sets the result into its true orientation. The messiness was only ever a matter of looking at the transformation in the wrong frame. Rotate into the right frame and it is simple.

The stretch in the middle is where the important information lives. The amounts by which the middle step stretches the axes are called the **singular values** of the matrix, and they are the heart of the decomposition. Each singular value is how much the transformation stretches along one of its special perpendicular directions, and unlike eigenvalues they are always real, always present, and always ranked cleanly from largest to smallest. The largest singular value is the transformation's strongest stretch, the direction along which it pulls space out the most; the smallest is its weakest.

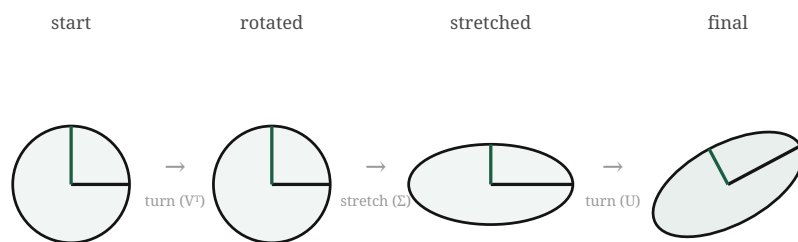


Figure 11.1: Every transformation is three moves in sequence: a rotation, a stretch along perpendicular axes, and a final rotation. Whatever a matrix does, it does it this way.

These singular values expose the rank we defined abstractly two chapters ago, and now we can see it. If a transformation collapses space, one of its stretch amounts is zero: the middle step squashes a direction flat rather than stretching it. So the number of non-zero singular values is exactly the rank, the number of directions that survive with real reach. A matrix that looks full but secretly has rank two shows it openly here: two non-zero singular values and the rest zero. The rank we had to compute abstractly becomes something you can read straight off the stretches.

And the ranking is what makes the decomposition so useful. Because the singular values come ordered from strongest to weakest, they tell us which directions carry most of what the transformation does, and which barely matter. A direction stretched enormously is doing most of the work; a direction stretched by almost nothing is nearly irrelevant. Keep the few strongest and discard the rest, and you have an approximation of the transformation that captures nearly all of its effect with a fraction of the information. This single idea, keep the big stretches and drop the small ones, is the engine behind compression and dimensionality reduction, and we will see it work shortly.

The math, built up

The three moves, turn then stretch then turn, become three matrices multiplied together. Since each move is a transformation and doing them in sequence is composition, any matrix A can be written as

$$A = U\Sigma V^T,$$

where the three factors are exactly the three moves, read right to left as composition demands. The rightmost factor V^T is the first rotation, turning space into the right frame. The middle factor Σ is the stretch, and it is a particularly simple kind of matrix: zero everywhere except along its diagonal, and those diagonal entries are the singular values, the stretch amounts. The leftmost factor U is the final rotation, setting the stretched result into place.

Each piece has a clean meaning, all of it built from things we already have. The columns of V are the special input directions, the perpendicular set that the first rotation lines up with the axes, the directions the transformation treats simply. The columns of U are where those directions end up pointing after the transformation, the output directions. And Σ holds the singular values on its diagonal, in descending order, each one pairing an input direction from V with an output direction in U and recording how much that direction is stretched between them. The whole transformation is: line up the special input directions with the axes, stretch each by its singular value, and rotate into the output directions.

The rotations U and V have a special property worth naming, because it is what makes them pure rotations with no stretching of their own. Their columns are perpendicular unit vectors, directions all at right angles to one another and each of length one. A matrix whose columns are perpendicular unit vectors is called **orthogonal**, and an orthogonal transformation is a rigid motion: it turns space without stretching or squashing any part of it, preserving all lengths and angles. So the only stretching in the entire

decomposition is quarantined into the middle factor Σ , where the singular values live. The two rotations turn; the diagonal stretches; nothing else happens.

We will not derive how the three factors are computed, which is a genuinely involved piece of numerical work, done always by machine and never by hand for any real matrix. What matters is what the decomposition means and guarantees: that it exists for every matrix without exception, that the singular values are real and ordered and expose the rank, and that the whole of a transformation's behaviour is laid out as turn, stretch, turn, with the stretches ranked from most to least important. That guarantee is what the rest of the chapter puts to work.

Build it yourself

NumPy computes the whole decomposition in one call, and we can confirm both that it rebuilds the original matrix and that the singular values say what we claimed.

Take a matrix and decompose it:

```
import numpy as np

A = np.array([[3.0, 1.0],
              [1.0, 3.0]])

U, S, Vt = np.linalg.svd(A)
print("singular values:", S)
print("U:\n", U)
print("Vt:\n", Vt)
```

```
singular values: [4. 2.]
U:
```

11 The Singular Value Decomposition

```
[[ -0.70710678 -0.70710678]
 [ -0.70710678  0.70710678]]
Vt:
[[ -0.70710678 -0.70710678]
 [ -0.70710678  0.70710678]]
```

The singular values come back in **S**, ordered largest first. **U** and **Vt** are the two rotations, the output directions and the transposed input directions. Notice **S** is returned as just the list of singular values, not the full diagonal matrix, since storing the zeros would be wasteful; we rebuild the diagonal when we need it.

Confirm the three moves really do reassemble the original transformation. We put the singular values onto a diagonal and multiply the three factors back together, turn then stretch then turn:

```
Sigma = np.diag(S)
reconstructed = U @ Sigma @ Vt
print(reconstructed)
print(np.allclose(reconstructed, A))
```

```
[[3.  1.]
 [1.  3.]]
True
```

The product returns the original matrix, and **np.allclose** confirms it, allowing for the faint floating-point dust these computations always leave. The three simple moves, composed, are exactly the matrix we started with.

Now watch the singular values expose rank. Here is a matrix that secretly collapses space, its second column twice the first:

```
collapse = np.array([[1.0, 2.0],
                     [2.0, 4.0]])
U, S, Vt = np.linalg.svd(collapse)
print(S)
```

```
[5.00000000e+00 1.98602732e-16]
```

One singular value is substantial and the other is essentially zero, a speck of floating-point dust where a true zero should be. One non-zero stretch means rank one: the transformation stretches a single direction and squashes the other flat, exactly the collapse we detected before with determinant and rank, now visible as a stretch amount that fell to zero. The singular values saw the collapse the same way every other tool did.

Finally, a first taste of compression, keeping the strong stretch and discarding the weak one. We take a matrix, decompose it, and rebuild it using only its largest singular value, throwing the smaller away:

```
A = np.array([[4.0, 1.0],
              [2.0, 3.0]])
U, S, Vt = np.linalg.svd(A)

# keep only the largest singular value, zero the rest
S_approx = S.copy()
S_approx[1:] = 0

approx = U @ np.diag(S_approx) @ Vt
print("original:\n", A)
print("rank-1 approximation:\n", approx)
```

```
original:
[[4. 1.]
```

11 The Singular Value Decomposition

```
[2. 3.]]
rank-1 approximation:
[[3.34164079 2.06524758]
 [2.78885438 1.7236068 ]]
```

The approximation is not the original, but it is a recognisable shadow of it, rebuilt from a fraction of the information by keeping only the strongest stretch. On a small two-by-two matrix the saving is trivial, but the same move on a large matrix, keeping a handful of the strongest stretches out of thousands, is exactly how images and data are compressed. The strong directions carry the picture; the weak ones were details we can afford to lose.

Where it lives in ML

The singular value decomposition is one of the most heavily used tools in all of applied machine learning, because the move it enables, keep the strong stretches and discard the weak ones, is exactly what so much of the field needs. We can finally build principal component analysis properly, the technique we have pointed at since the independence chapter. PCA takes a cloud of data, finds the directions along which it varies most, and keeps only those. Those directions are the top singular directions of the data, and how much the data varies along each is given by the singular values. PCA is, essentially, an SVD of the data with the small singular values thrown away, which is why it reduces a thousand features to a handful while losing almost nothing: the discarded directions were the ones the data barely used.

Compression works by the same logic, applied to a single object rather than a dataset. An image is a grid of numbers, which is to say a matrix, and that matrix has an SVD. Most images have a few strong singular values carrying the broad structure and a long tail of tiny ones carrying fine detail.

Keep the strongest few hundred stretches and discard the thousands of weak ones, and you can rebuild a picture nearly indistinguishable from the original using a small fraction of the numbers. The rank-one approximation we built by hand was the crudest version of this; real compression keeps more than one stretch, but the principle is identical. The strong directions are the image; the weak ones were details you can afford to lose.

Recommendation systems lean on the decomposition in a way that is almost magical when you first see it. Arrange what every user thought of every item into an enormous matrix, users down the side, items across the top, most of it blank because most people have rated almost nothing. The SVD of that sparse, gigantic matrix uncovers a small number of strong hidden directions, latent patterns of taste that explain most of the ratings, things like a leaning toward a genre that no one labelled but that falls out of the stretches. Reconstructing the matrix from its strongest singular directions fills in the blanks with startlingly good guesses about what a user would think of something they have never seen. The recommendation you get for the next thing to watch is, underneath, an SVD quietly reconstructing a missing entry.

And there is a use tied closely to the concerns this book was written alongside, in understanding and controlling what models learn. The transformations inside a trained network have singular values, and studying them reveals how the network stretches and compresses the information passing through it, which directions it amplifies and which it discards. Techniques that shrink or fine-tune large models work by keeping only the strongest singular directions of the transformations, the same keep-the-big-stretches move applied to a network's weights rather than to data. The decomposition built here is, increasingly, a tool for looking inside models and deciding what in them matters, which is close to the heart of making powerful systems safe and efficient.

Common misunderstandings

Singular values are not eigenvalues, though they are related.

After the last chapter it is tempting to treat the two as the same. They are not. Every matrix has singular values, always real and non-negative and ordered; only some matrices have real eigenvalues, and eigenvalues can be negative or complex. For the special well-behaved matrices where both exist, they line up closely, which is why the two get confused, but the singular value decomposition works on every matrix precisely because singular values do not carry the fragility eigenvalues do. When you need something guaranteed to exist and behave, reach for singular values.

The decomposition is exact, not an approximation. It is easy to come away thinking of the SVD as an approximation method, because we use it to approximate by dropping small singular values. But the full decomposition, keeping every singular value, is exact: the three moves multiply back to precisely the original matrix, to the last digit the arithmetic allows. The approximation is a separate, optional second step, throwing away the weak stretches on purpose. The decomposition itself hides nothing and loses nothing; only the deliberate truncation does.

Small singular values are not always noise to discard. The keep-the-strong-drop-the-weak move is powerful, but it rests on an assumption: that the weak directions carry little that matters. Usually true, sometimes badly false. In some problems the small singular values carry exactly the subtle signal you care about, and discarding them throws away the answer. Truncating is a judgement about your data, not a mechanical rule, the same lesson we met with cosine similarity and with norms: the tool does not decide for you whether the thing it removes was worth keeping.

A matrix is not literally three matrices. The decomposition writes A as a product of three factors, but A remains one transformation; the three factors are a way of *seeing* it as turn, stretch, turn, not a claim that it was secretly assembled from three separate things. This is the flip side of a fact from the composition chapter, where we noted you generally cannot

recover the two matrices that were multiplied to make a product. The SVD is the remarkable exception: for this particular into-three factorisation, there is a canonical answer, always available, which is part of what makes it so valuable. Composition usually hides its factors; the SVD is where they can always be recovered.

Check your intuition

Try each before opening the answers.

1. In the decomposition $A = U\Sigma V^T$, which factor holds the stretch amounts, and which two are pure rotations?
2. A matrix has singular values 5, 3, and 0. What is its rank, and what does the zero tell you about the transformation?
3. Every matrix has a singular value decomposition, but not every matrix has real eigenvectors. Why is this a reason to prefer singular values for general work?
4. You compress an image by keeping only its ten largest singular values out of five hundred. What have you kept, and what have you thrown away?
5. A transformation's singular values are 10, 9.8, 9.5, and 0.01. Which direction could you discard with the least loss, and what does the spread of values suggest about the data?

Answers

1. The middle factor Σ holds the stretch amounts, the singular values, along its diagonal and zeros elsewhere. The outer two factors, U and V^T , are the pure rotations: V^T turns space into the right frame first, and U turns the stretched result into its final orientation. All the

stretching is quarantined in the middle; the outer two only turn.

2. The rank is two, the number of non-zero singular values. The zero singular value means the transformation completely squashes one direction flat rather than stretching it, collapsing that dimension to nothing. So this transformation is rank-deficient, singular, non-invertible, with a zero determinant, all the faces of collapse we have gathered, now shown as a stretch amount that fell to zero. Two directions survive with real reach; the third is destroyed.

3. Because a tool that always exists and always behaves is safer to build on than one that sometimes fails. Eigenvectors can come out complex or fail to point anywhere real, and their stretch factors do not reliably rank importance for general matrices. Singular values are always real, always present, always ordered from strongest to weakest, for every matrix without exception. For general work, where you cannot guarantee your matrix is the well-behaved kind eigenvectors need, the singular value decomposition gives you the structure you wanted without the fragility.

4. You have kept the ten strongest stretches, the directions along which the image varies most, which carry its broad structure and most of its recognisable content. You have thrown away four hundred and ninety weak directions, the fine details the image barely used. The result is a recognisable version of the picture rebuilt from a fraction of the numbers, sharp enough to see clearly even though most of the information, by count, is gone. The strong directions were the picture; the discarded ones were details.

5. The fourth direction, with singular value 0.01, could be discarded with almost no loss, since the transformation barely stretches it at all, so it carries almost nothing. The spread is telling: three large, close singular values and then a sudden drop to nearly zero means the data is essentially three-dimensional, spread richly through three directions and flat in the fourth. The sharp fall from 9.5 to 0.01 is the signature of data that lives in fewer dimensions than it appears to,

Check your intuition

exactly the low-rank structure that makes dimensionality reduction pay off.

12 High-Dimensional Space

The itch

Every picture in this book has been drawn in two dimensions, occasionally three. We have leaned on that relentlessly, and it has served us well: arrows, shadows, parallelograms, the circle becoming an ellipse. And at every turn we reassured ourselves that the arithmetic carries into higher dimensions untouched, that the formulas do not care how many numbers a vector has. That reassurance was true, and it still is. The formulas carry perfectly.

But we quietly promised something else along the way, or at least let the reader assume it: that the *intuition* carries too. That three hundred dimensional space is just like the plane, only roomier, and that the pictures in our heads keep meaning what they meant. This is where we admit that this second promise does not hold. The formulas travel into high dimensions unharmed. The intuitions do not. They break, sometimes gently and sometimes violently, and a mind trained only on two and three dimensions will confidently expect things that are simply false up there.

This closing chapter is about that gap, the difference between arithmetic that keeps working and intuition that stops being reliable. It is a strange chapter, less about building a new tool than about dismantling false expectations before they cost us, because nearly all real machine learning lives in the high dimensions where our instincts mislead us. Knowing exactly how our low-dimensional intuition fails is not a curiosity. It is protection against a specific set of mistakes that the pictures we have been drawing quietly set us up to make.

The picture

The trouble is that we cannot picture high dimensions at all, and so we do the only thing we can: we imagine the plane, or a room, and silently assume the pattern continues. Most of the time we do not even notice we are doing it. When someone says a data point lives in three hundred dimensional space, we picture a dot somewhere in a room and let that stand in. The stand-in is where the errors come from, because the room is lying to us about what three hundred dimensions are like.

Consider one concrete way the pattern breaks, chosen because it is easy to state and genuinely surprising. In the plane, if you scatter points at random in a square, plenty of them land near the middle. In three dimensions, the same is true of a cube. But as the number of dimensions climbs, something odd happens: the points drift away from the centre and pile up near the edges and corners, until in high dimensions almost every random point is out near the boundary and almost none is in the middle. The centre, which felt so central in the plane, becomes nearly empty. The room in our head, where the middle is a perfectly ordinary place to be, has misled us completely.

Here is another, stranger still. Take two random points in the plane and measure the distance between them; sometimes they are close, sometimes far, with plenty of variety. Do the same in very high dimensions and the distances stop varying. Almost every pair of random points ends up almost exactly the same distance apart, as if the notion of “close” and “far” had quietly collapsed into a single “medium” that applies to nearly everyone. In the plane, some points are near neighbours and some are distant; in three hundred dimensions, everyone is roughly equidistant from everyone else, and the idea of a nearest neighbour, so natural on a page, becomes almost meaningless.

These are not tricks or pathological setups. They are how high-dimensional space genuinely behaves, and they contradict the room in our heads directly. We cannot fix this by picturing harder, because the picturing is exactly the

problem; our visual imagination runs on three dimensions and there is no fourth to add. What we can do is learn the specific ways the intuition fails, hold them as explicit warnings, and trust the arithmetic over the picture whenever the two disagree. Up there, the arithmetic is the only reliable guide we have.

The math, built up

This chapter has less arithmetic to build than the others, because its lessons are about phenomena rather than formulas. But one calculation shows precisely where the intuition cracks, and it uses only the length we defined in the very first chapter.

Think about the corners of a cube compared to its faces, measured from the centre. In two dimensions, take a square whose sides sit one unit from the centre. The middle of an edge is one unit away, straightforward. A corner, though, is further, by Pythagoras: $\sqrt{1^2 + 1^2} = \sqrt{2} \approx 1.41$. The corner is about forty percent further from the centre than the edge. Not dramatic, but already the corners reach out further than the flat sides, which is the diamond-and-circle asymmetry from the norms chapter seen once more.

Now let the dimensions climb, keeping the faces one unit from the centre. In three dimensions the corner distance is $\sqrt{1^2 + 1^2 + 1^2} = \sqrt{3} \approx 1.73$. In ten dimensions it is $\sqrt{10} \approx 3.16$. In three hundred dimensions the corners sit $\sqrt{300} \approx 17.3$ units from the centre, while the faces remain stubbornly at one. The corners have raced away to seventeen times the distance of the faces, and there are vastly more corners than faces to begin with. The cube, which in our imagination is a compact and roughly round sort of thing, is in high dimensions overwhelmingly spikes: almost all of it is out in the corners, absurdly far from the centre, with the familiar middle a tiny and nearly empty core.

12 High-Dimensional Space

That single calculation, corner distance growing as the square root of the dimension while face distance stays fixed, is the arithmetic behind the whole phenomenon. It is not exotic mathematics; it is the length formula from chapter one, applied honestly in three hundred dimensions and refusing to match the picture. The formula never lied. It kept computing lengths exactly as it always had. It was our mental image of a cube, borrowed from three dimensions, that was never a fair likeness of the high-dimensional thing.

Measure it yourself

Rather than build a tool, let us watch the intuition break with our own code, by measuring high-dimensional space and comparing what we find to what the room in our heads expects.

First, the claim that random points drift to the edges. We scatter points at random inside a cube in growing numbers of dimensions and measure their distance from the centre. If the room in our head were right, plenty should sit near the middle, near distance zero:

```
import numpy as np

rng = np.random.default_rng(0)

for dim in [2, 10, 100, 1000]:
    points = rng.random((5000, dim)) - 0.5      # centred cube, sides at ±0.5
    distances = np.linalg.norm(points, axis=1)
    print(f"{dim:4d} dimensions: average distance from centre = {distances.mean():.2f}")
```

```
2 dimensions: average distance from centre = 0.38
10 dimensions: average distance from centre = 0.90
100 dimensions: average distance from centre = 2.88
1000 dimensions: average distance from centre = 9.13
```


In two dimensions the average distance from the centre is small; the middle is a normal place to be. As the dimensions climb, the average distance grows steadily, and the fraction of points anywhere near the centre falls toward nothing. The points have drifted outward exactly as promised, and the higher we go the emptier the middle becomes.

Now the stranger claim, that distances between points stop varying. We take many random points in high dimensions, measure all the distances between pairs, and look at how much those distances differ from one another:

```
for dim in [2, 10, 100, 1000]:
    points = rng.random((500, dim))
    # distances from the first point to all others
    d = np.linalg.norm(points[1:] - points[0], axis=1)
    spread = d.std() / d.mean()      # variation relative to the average
    print(f"{dim:4d} dimensions: relative spread of distances = {spread:.3f}")
```

```
2 dimensions: relative spread of distances = 0.470
10 dimensions: relative spread of distances = 0.182
100 dimensions: relative spread of distances = 0.057
1000 dimensions: relative spread of distances = 0.017
```

The relative spread shrinks as dimensions grow. In two dimensions distances vary a good deal, some points genuinely near and others far. By a thousand dimensions the spread has collapsed: nearly every point is almost exactly the average distance from the first, and the difference between the nearest and the farthest has all but vanished. The concept of a nearest neighbour, so solid on a page, has quietly dissolved. We did not derive this; we measured it, and the measurement contradicts the room in our heads as flatly as a measurement can.

Where it lives in ML

The curse of dimensionality is not an abstraction in machine learning; it is a daily constraint that shapes what does and does not work, precisely because machine learning lives almost entirely in the high dimensions where intuition fails. Data routinely has hundreds or thousands of features, which is to say hundreds or thousands of dimensions, and every strangeness of this chapter is in force.

The collapse of distance is the sharpest example. A whole family of methods works by finding the nearest points to a given one, on the reasonable-sounding idea that nearby points are similar. This is the logic of recommending what similar users liked, of classifying a thing by what its neighbours are. But we have just seen that in high dimensions nearly every point is almost the same distance from every other, so “nearest” stops being meaningful, and methods built on it degrade badly as the dimensions climb. The comfortable intuition that near means similar, so trustworthy on a page, quietly stops holding, and a practitioner who does not know this will build a method that works in their small tests and fails on real, high-dimensional data for reasons they cannot see.

The emptiness of high-dimensional space causes a second, related problem: data becomes desperately sparse. To cover the plane with sample points you need only so many; to cover a cube, more; to cover a three hundred dimensional space at the same density you would need more points than there are atoms in anything. Real datasets, however large they feel, are unimaginably sparse in the spaces they inhabit, a scattering of points in a vast emptiness. This is why models need so much data and still see only a thin sample of the space they are meant to handle, and why generalising beyond the sampled points, filling the enormous gaps, is so much of the difficulty of the field.

And yet high dimensions are not only a curse, which is the hopeful turn this book ends the part on. The reason machine learning works at all, despite the emptiness, is that real data almost never fills the high-dimensional

space it nominally lives in. It clusters on a much lower-dimensional surface within that space, a sheet or ribbon folded through the high dimensions, occupying far fewer true dimensions than the feature count suggests. This is the low-rank hope made physical, the same structure the singular value decomposition finds and exploits. The curse says the ambient space is impossibly vast and empty; the saving grace says the data does not actually spread through it but huddles on something small enough to learn. Nearly every successful method is, in some form, an attempt to find and work on that hidden low-dimensional structure rather than the hostile high-dimensional space it is embedded in.

Common misunderstandings

The formulas do not break in high dimensions; the intuitions do. The single most important thing to carry out of this chapter is that nothing went wrong with the mathematics. The length formula, the dot product, the norms, every tool still computes exactly what it always computed, correctly, in any number of dimensions. What fails is the mental picture we attach to the numbers, the room in our heads that we borrowed from three dimensions. When the picture and the arithmetic disagree in high dimensions, the arithmetic is right. Distrust the picture, never the formula.

High dimensions are not just “more room.” The natural assumption is that adding dimensions simply gives space more room, the way a cube has more room than a square, and that everything otherwise behaves the same. It does not. Adding dimensions changes the qualitative behaviour of space: where the mass sits, how distances distribute, how full a region can be. It is not the plane with extra elbow room; it is a genuinely different regime where the familiar relationships no longer hold. Treating high-dimensional space as a roomier version of the plane is exactly the mistake this chapter exists to prevent.

The curse does not mean high-dimensional machine learning is hopeless. Reading only the warnings, one might conclude that learning in high dimensions cannot work, yet it plainly does. The resolution is that real data does not occupy the full high-dimensional space; it lies on a much lower-dimensional structure within it, where the curse is far weaker. The methods that succeed are the ones that find and exploit that structure. The curse describes the hostile space data is embedded in, not the friendlier shape the data actually takes. Hopelessness would follow only if data truly filled the space, and it essentially never does.

Dimensionality reduction is not merely throwing information away. Because reducing dimensions sounds like loss, it is tempting to see it as a regrettable compromise. Seen through this chapter, it is closer to the opposite: it is moving from the hostile, empty, intuition-breaking high-dimensional space down onto the smaller true structure where the data actually lives and where distance and nearness mean something again. Done well, it discards the empty dimensions the data was not using and keeps the ones it was, which is why it so often makes models work better rather than worse. It is less a sacrifice than a return to a space our tools and intuitions can handle.

Check your intuition

Try each before opening the answers.

1. In high dimensions, are randomly scattered points more likely to be found near the centre of a region or near its edges? Why does this contradict our intuition from the plane?
2. A method classifies a new data point by looking at its nearest neighbours. Why might this method work well in low dimensions but poorly in very high ones?

Check your intuition

3. In a cube whose faces sit one unit from the centre, how far is a corner from the centre in one hundred dimensions? Use the length formula.
4. If high-dimensional space is so hostile and empty, why does machine learning on high-dimensional data work at all?
5. In high dimensions the arithmetic of the length formula and the picture in your head disagree about how a cube behaves. Which do you trust, and what is the general lesson?

 Answers

1. Near the edges, overwhelmingly. As dimensions grow, random points drift away from the centre and pile up near the boundary, until in high dimensions almost none are anywhere near the middle. This contradicts the plane, where scattering points in a square leaves plenty near the centre, a perfectly ordinary place to be. Our intuition, trained on two and three dimensions, expects a populated middle, and high dimensions empty it out, because there is vastly more room near the boundary than near the centre once the dimensions climb.
2. In low dimensions, nearest neighbours are genuinely meaningful: some points are close and some are far, so “nearest” picks out truly similar points. In very high dimensions, distances between points collapse toward a single common value, so nearly every point is about the same distance from the new one, and the nearest neighbour is barely nearer than the farthest. The method’s core assumption, that nearby means similar, quietly stops holding, and its accuracy degrades as the dimensions rise even though the procedure itself is unchanged.
3. The corner is $\sqrt{100} = 10$ units from the centre, while the faces remain one unit away. Each of the hundred coordinates contributes one to the sum under the square root, giving $\sqrt{1 + 1 + \dots + 1} = \sqrt{100} = 10$. The corner is ten times further from the centre than the face, a cube that in our imagination is compact but in a hundred dimensions is almost entirely distant spikes with a tiny empty core.

4. Because real data does not fill the high-dimensional space it lives in. It clusters on a much lower-dimensional structure within that space, a folded surface occupying far fewer true dimensions than the feature count suggests, where the curse is far weaker and distances mean something again. The methods that work are the ones that find and operate on that hidden low-dimensional structure. The space is hostile; the shape the data actually takes within it is not, and that gap is what makes learning possible.

5. Trust the arithmetic, always. The length formula computes correctly in any number of dimensions; it is the mental picture, borrowed from three dimensions, that fails to match. The general lesson, and a fitting one to end the part on, is that our geometric intuition is a tool with a limited range, superb in the low dimensions it was built for and unreliable beyond them, while the arithmetic we derived from that intuition outlives it and keeps working where the pictures cannot follow. When they disagree, the formula is the one that has earned our trust.